Guide SBCL Cross-Compiling

Guide SBCL Cross-Compiling

by CMU CL team, Fred, Earl Kent, many others

Guide SBCL Cross-Compiling

Copyright © blah blah blah

Misc. Published by the: blah blah blah

ISBN blah blah

Permission is granted to copy, distribute and/or modify this document under the terms of the MIT License

Short Contents

Preface 1
1 Compiler Overview 11
The Implicit Continuation Representation 14

Compiler Front End
2 Front End: ICR Conversion 25

Compiler Middle Phase
3 Virtual Machine Representation Introduction 43
4 Global TN Assignment 44

Compiler Back End
5 Copy propagation 57
6 Representation Selection 58

The SBCL Lisp Virtual Machine
7 Introduction to the VM 71
8 Data Types and Storage Resources 75
9 Characters 78
10 Symbols 79

User Interface to the Compiler
11 User Interface of the Compiler 95

Retargeting the Compile
12 Retargeting the Compiler 100
13 Storage bases and classes 102
14 Introduction 131
15 [Data Types and Object Formats] 133
16 [Runtime Environment] 147
17 [Storage Management] 152
18 [Assembler Support Routines] 154

19 [Control Conventions] 175
A [Fasload File Format] 185
20 Appendix – Building CMU Common Lisp 196
Index 208

Table of Contents

Preface 1
On Reading this Text 1
SBCL History (???) 1

1 Compiler Overview 11

The Implicit Continuation Representation 14
Tail sets 16
Hairy function representation 17
ICR representation of non-local exits 18
Block compilation 19
Entry points 20

Compiler Front End

2 Front End: ICR Conversion 25
2.1 Canonical forms 25
2.1.1 Array hackery 25

Compiler Middle Phase

3 Virtual Machine Representation Introduction 43

4 Global TN Assignment 44

Compiler Back End

5 Copy propagation 57

6 Representation Selection 58

The SBCL Lisp Virtual Machine

7 Introduction to the VM 71
7.0.1 VOP base name rules 73
7.0.2 VOP name prefixes and suffixes 73

8 Data Types and Storage Resources ... 75
8.1 Lisp Objects ... 75
8.2 Type VOPs ... 76
8.3 Type Sub-primitives ... 77

9 Characters ... 78

10 Symbols ... 79
10.1 Sub-primitives ... 79
10.2 Accessor VOPs ... 79
10.3 Special Binding ... 80
10.4 Property Lists ... 80

User Interface to the Compiler

11 User Interface of the Compiler ... 95
11.1 Error Message Utilities ... 95
11.2 Source Paths ... 95

Retargeting the Compile

12 Retargeting the Compiler ... 100

13 Storage bases and classes ... 102

14 Introduction ... 131
14.1 Scope and Purpose ... 131
14.2 Notational Conventions ... 131

15 [Data Types and Object Formats] ... 133
15.1 [Lisp Objects] ... 133
15.2 [Table of Type Codes] ... 133
15.3 [Table of Space Codes] ... 134
15.4 Immediate Data Type Descriptions ... 134
15.5 [Pointer-Type Objects and Spaces] ... 135
15.6 [Forwarding Pointers] ... 137
15.7 [System and Stack Spaces] ... 138
15.8 [Vectors and Arrays] ... 138
15.8.1 [General Vectors] ... 138
15.8.2 [Integer Vectors] ... 139
15.8.3 [Arrays] ... 140
15.9 [Symbols Known to the Assembler Routines] ... 141

16 [Runtime Environment] ... 147
16.1 [Register Allocation] ... 147
16.2 [Function Object Format] ... 148
16.3 [Defined-From String Format] ... 149
16.4 [Control-Stack Format] ... 149
16.4.1 [Call Frames] ... 149
16.4.2 [Catch Frames] ... 150
16.5 [Binding-Stack Format] ... 151

17 [Storage Management] ... 152
17.1 [The Transporter] ... 152
17.2 [The Scavenger] The scavenger looks through an area of ... 153
17.3 [Purification] ... 153

18 [Assembler Support Routines] ... 154
18.1 [Miscop Descriptions] ... 154
18.1.1 [Allocation] ... 154
18.1.2 [Stack Manipulation] ... 156
18.1.3 [List Manipulation] ... 157
18.1.4 [Symbol Manipulation] ... 157
18.1.5 [Array Manipulation] ... 158
18.1.6 [Type Predicates] ... 161
18.1.7 [Arithmetic] ... 163
18.1.8 [Branching] ... 166
18.1.9 [Function Call and Return] ... 166
18.1.10 [Miscellaneous] ... 171
18.1.11 [System Hacking] ... 171

19 [Control Conventions] ... 175
19.1 [Function Calls] ... 175
19.2 (Returning from a Function Call) ... 177
19.2.1 [Returning Multiple-Values] ... 178
19.3 [Non-Local Exits] ... 179
19.4 [Escaping to Lisp code] ... 179
19.5 [Errors] ... 180
19.6 [Trapping to the Mach Kernel] ... 184
19.7 [Interrupts] ... 184

Appendix A [Fasload File Format] ... 185
A.1 [General] ... 185
A.2 [Strategy] ... 185
A.3 [Fasload Language] ... 186

20 Appendix – Building CMU Common Lisp 196
20.1 (Introduction) 196
20.2 (Installing Source Code) 197
20.3 (Compiling the Lisp Startup Program) 199
20.4 (Assembling Assembler routines) 199
20.5 (Compiling the Compiler) 199
20.6 (Compiling the Lisp Sources) 200
20.7 (Compiling Hemlock) 200
20.8 (Compiling Matchmaker) 200
20.9 (Generating Lisp Source Files from Matchmaker Definition Files) ... 201
20.10 (Compiling Matchmaker Generated Lisp Files) 201
20.11 (Compiling the Common Lisp Object System) 202
20.12 (Compiling Genesis) 202
20.13 (Building a Cold Core File) 202
20.14 (Building a Full Common Lisp) 203
20.15 (Debugging) 204
20.16 (Running the Soar Benchmark) 204
20.17 (Summary) 205

Index 208

Preface

At least initially this is mostly a project plan for porting SBCL to WASM which is an unusual port that needs indepth integration with SBCL's cross-building capabilities, with considerable novel re-working of the process, making it an ideal project to document the proecss and its assumptions.

On Reading this Text

Some general pointers on how to use this text:

1. to understanding he SBCL compiler as an independent facility separate from integration into an SBCL system.
2. To understand, in depth the cross-compilation process for SBCL.
3. To provide a checklist for porting SBCL to other architectures, and operating environments
4. To get up to speed tweaking existing architectures.
5. Getting the cross compilation process running.

SBCL History (???)

Maclisp -> Spice Lisp -> CMUCL -> SBCL

;;;; Terminology.

OBJECT An object is one of the following: a single word containing immediate data (characters, fixnums, etc) a single word pointing to an object (structures, conses, etc.) These are tagged with three low-tag bits as described in the section "Tagging". This is synonymous with DESCRIPTOR.

LISP OBJECT A Lisp object is a high-level object discussed as a data type in Common Lisp: The Language.

DATA-BLOCK A data-block is a dual-word aligned block of memory that either manifests a Lisp object (vectors, code, symbols, etc.) or helps manage a Lisp object on the heap (array header, function header, etc.).

DESCRIPTOR A descriptor is a tagged, single-word object. It either contains immediate data or a pointer to data. This is synonymous with OBJECT.

WORD A word is a 32-bit quantity.

;;;; Tagging.

The following is a key of the three bit low-tagging scheme: 000 even fixnum 001 function pointer 010 other-immediate (header-words, characters, symbol-value trap value, etc.) 011 list pointer 100 odd fixnum 101 structure pointer 110 unused 111 other-pointer to data-blocks (other than conses, structures, and functions)

This taging scheme forces a dual-word alignment of data-blocks on the heap, but this can be pretty negligible: RATIOS and COMPLEX must have a header-word anyway since they are not a major type. This wastes one word for these infrequent data-blocks since they require two words for the data. BIGNUMS must have a header-word and probably contain only one other word anyway, so we probably don't waste any words here. Most bignums just barely overflow fixnums, that is by

a bit or two. Single and double FLOATS? no waste one word wasted SYMBOLS are dual-word aligned with the header-word. Everything else is vector-like including code, so these probably take up so many words that one extra one doesn't matter.

;;;; GC Comments.

Data-Blocks comprise only descriptors, or they contain immediate data and raw bits interpreted by the system. GC must skip the latter when scanning the heap, so it does not look at a word of raw bits and interpret it as a pointer descriptor. These data-blocks require headers for GC as well as for operations that need to know how to interpret the raw bits. When GC is scanning, and it sees a header-word, then it can determine how to skip that data-block if necessary. Header-Words are tagged as other-immediates. See the sections "Other-Immediates" and "Data-Blocks and Header-Words" for comments on distinguishing header-words from other-immediate data. This distinction is necessary since we scan through data-blocks containing only descriptors just as we scan through the heap looking for header-words introducing data-blocks.

Data-Blocks containing only descriptors do not require header-words for GC since the entire data-block can be scanned by GC a word at a time, taking whatever action is necessary or appropriate for the data in that slot. For example, a cons is referenced by a descriptor with a specific tag, and the system always knows the size of this data-block. When GC encounters a pointer to a cons, it can transport it into the new space, and when scanning, it can simply scan the two words manifesting the cons interpreting each word as a descriptor. Actually there is no cons tag, but a list tag, so we make sure the cons is not nil when appropriate. A header may still be desired if the pointer to the data-block does not contain enough information to adequately maintain the data-block. An example of this is a simple-vector containing only descriptor slots, and we attach a header-word because the descriptor pointing to the vector lacks necessary information – the type of the vector's elements, its length, etc.

There is no need for a major tag for GC forwarding pointers. Since the tag bits are in the low end of the word, a range check on the start and end of old space tells you if you need to move the thing. This is all GC overhead.

;;;; Structures.

Structures comprise a word for each slot in the definition in addition to one word, a type slot which is a pointer descriptor. This points to a structure describing the data-block as a structure, a defstruct-descriptor object. When operating on a structure, doing a structure test can be done by simply checking the tag bits on the pointer descriptor referencing it. As described in section "GC Comments", data-blocks such as those representing structures may avoid having a header-word since they are GC-scanable without any problem. This saves two words for every structure instance.

;;;; Fixnums.

A fixnum has one of the following formats in 32 bits: ——- | 30 bit 2's complement even integer | 0 0 0 | ——- or ——- | 30 bit 2's complement odd integer | 1 0 0 | ——-

Effectively, there is one tag for immediate integers, two zeros. This buys one more bit for fixnums, and now when these numbers index into simple-vectors or offset into memory, they point to word boundaries on 32-bit, byte-addressable machines. That is, no shifting need occur to use the number directly as an offset.

This format has another advantage on byte-addressable machines when fixnums are offsets into vector-like data-blocks, including structures. Even though we previously mentioned data-blocks are dual-word aligned, most indexing and slot accessing is word aligned, and so are fixnums with effectively two tag bits.

Two tags also allow better usage of special instructions on some machines that can deal with two low-tag bits but not three.

Since the two bits are zeros, we avoid having to mask them off before using the words for arithmetic, but division and multiplication require special shifting.

;;;; Other-immediates.

An other-immediate has the following format: ——————————————

Using a separate word allows for much larger vectors, and it allows LENGTH to simply access a single word without masking or shifting. Similarly, the header for complex arrays and vectors has a second word, following the header-word, the system uses for the fill pointer, so computing the length of any array is the same code sequence.

;;;; Data-Blocks and Other-immediates Typing.

These are the other-immediate types. We specify them including all low eight bits, including the other-immediate tag, so we can think of the type bits as one type – not an other-immediate major type and a subtype. Also, fetching a byte and comparing it against a constant is more efficient than wasting even a small amount of time shifting out the other-immediate tag to compare against a five bit constant.

Number (< 30) 00000 010 bignum 10 00000 010 ratio 14 00000 010 single-float 18 00000 010 double-float 22 00000 010 complex 26

Array (>= 30 code 86) Simple-Array (>= 20 code 70) 00000 010 simple-array 30 Vector (>= 34 code 82) 00000 010 simple-string 34 00000 010 simple-bit-vector 38 00000 010 simple-vector 42 00000 010 (simple-array (unsigned-byte 2) (*)) 46 00000 010 (simple-array (unsigned-byte 4) (*)) 50 00000 010 (simple-array (unsigned-byte 8) (*)) 54 00000 010 (simple-array (unsigned-byte 16) (*)) 58 00000 010 (simple-array (unsigned-byte 32) (*)) 62 00000 010 (simple-array single-float (*)) 66 00000 010 (simple-array double-float (*)) 70 00000 010 complex-string 74 00000 010 complex-bit-vector 78 00000 010 (array * (*)) – general complex vector. 82 00000 010 complex-array 86

00000 010 code-header-type 90 00000 010 function-header-type 94 00000 010 closure-header-type 98 00000 010 funcallable-instance-header-type 102 00000 010 unused-function-header-1-type 106 00000 010 unused-function-header-2-type 110 00000 010 unused-function-header-3-type 114 00000 010 closure-function-header-type 118 00000 010 return-pc-header-type 122 00000 010 value-cell-header-type 126 00000 010 symbol-header-type 130 00000 010 base-character-type 134 00000 010 system-area-pointer-type (header type) 138 00000 010 unbound-marker 142 00000 010 weak-pointer-type 146

;;;; Strings.

All strings in the system are C-null terminated. This saves copying the bytes when calling out to C. The only time this wastes memory is when the string contains a multiple of eight characters, and then the system allocates two more words (since Lisp objects are dual-word aligned) to hold the C-null byte. Since the system will make heavy use of C routines for systems calls and libraries that save reimplementation of higher level operating system functionality (such as pathname resolution or current directory computation), saving on copying strings for C should make C call out more efficient.

The length word in a string header, see section "Data-Blocks and Header-Word Format", counts only the characters truly in the Common Lisp string. Allocation and GC will have to know to handle the extra C-null byte, and GC already has to deal with rounding up various objects to dual-word alignment.

;;;; Symbols and NIL.

Symbol data-block has the following format: ———————————————————————————- | 5 (data-block words) | Symbol Type (8 bits) | ———————————————————————————————————- | Value Descriptor | ———————————————————————————————————- | Function Pointer | ———————————————————————————————————- | Raw Function Address | ———————————————————————————————————- | Setf Function | ———————————————————————————————————- | Property List | ———————————————————————————————————- | Print Name | ———————————————————————————————————- | Package | ———————————————————————————————————-

Most of these slots are self-explanatory given what symbols must do in Common Lisp, but a couple require comments. We added the Raw Function Address slot to speed up named call which is the most common calling convention. This is a non-descriptor slot, but since objects are dual word aligned, the value inherently has fixnum low-tag bits. The GC method for symbols must know to update this slot. The Setf Function slot is currently unused, but we had an extra slot due to adding Raw Function Address since objects must be dual-word aligned.

The issues with nil are that we want it to act like a symbol, and we need list operations such as CAR and CDR to be fast on it. CMU Common Lisp solves this by putting nil as the first object in static space, where other global values reside, so it has a known address in the system: ———————————————————————————————————- <– start static | 0 | space ———————————————————————————————————- | 5 (data-block words) | Symbol Type (8 bits) | ———————————————————————————————————- <– nil | Value/CAR | ———————————————————————————————————- | Definition/CDR | ———————————————————————————————————- | Raw Function Address | ———————————————————————————————————- | Setf Function | ———————————————————————————————————- | Property List | ———————————————————————————————————- | Print Name | ———————————————————————————————————- | Package | ———————————————————————————————————- | ... | ———————————————————————————————————- In addition, we make the list typed pointer to nil actually point past the header word of the nil symbol data-block. This has usefulness explained below. The value and definition of nil are nil. Therefore, any reference to nil used as a list has quick list type checking, and CAR and CDR can go right through the first and second words as if nil were a cons object.

When there is a reference to nil used as a symbol, the system adds offsets to the address the same as it does for any symbol. This works due to a combination of nil pointing past the symbol header-word and the chosen list and other-pointer type tags. The list type tag is four less than the other-pointer type tag, but nil points four additional bytes into its symbol data-block.

;;;; Array Headers.

The array-header data-block has the following format: ———————————————————————————————————- | Header Len (24 bits) = Array Rank +5 | Array Type (8 bits) | ———————————————————————————————————- | Fill Pointer (30 bits) | 0 0 | ———————————————————————————————————- | Available Elements (30 bits) | 0 0 | ———————————————————————————————————- | Data Vector (29 bits) | 1 1 1 | ———————————————————————————————————- | Displacement (30 bits) | 0 0 | ———————————————————————————————————- | Displacedp (29 bits) – t

or nil | 1 1 1 | ————————————————————————————- | Range of First Index (30 bits) | 0 0 | ————————————————————————————- . . .

The array type in the header-word is one of the eight-bit patterns from section "Data-Blocks and Other-immediates Typing", indicating that this is a complex string, complex vector, complex bit-vector, or a multi-dimensional array. The data portion of the other-immediate word is the length of the array header data-block. Due to its format, its length is always five greater than the array's number of dimensions. The following words have the following interpretations and types: Fill Pointer This is a fixnum indicating the number of elements in the data vector actually in use. This is the logical length of the array, and it is typically the same value as the next slot. This is the second word, so LENGTH of any array, with or without an array header, is just four bytes off the pointer to it. Available Elements This is a fixnum indicating the number of elements for which there is space in the data vector. This is greater than or equal to the logical length of the array when it is a vector having a fill pointer. Data Vector This is a pointer descriptor referencing the actual data of the array. This a data-block whose first word is a header-word with an array type as described in sections "Data-Blocks and Header-Word Format" and "Data-Blocks and Other-immediates Typing" Displacement This is a fixnum added to the computed row-major index for any array. This is typically zero. Displacedp This is either t or nil. This is separate from the displacement slot, so most array accesses can simply add in the displacement slot. The rare need to know if an array is displaced costs one extra word in array headers which probably aren't very frequent anyway. Range of First Index This is a fixnum indicating the number of elements in the first dimension of the array. Legal index values are zero to one less than this number inclusively. IF the array is zero-dimensional, this slot is non-existent. ... (remaining slots) There is an additional slot in the header for each dimension of the array. These are the same as the Range of First Index slot.

;;;; Bignums.

Bignum data-blocks have the following format: ——- | Length (24 bits) | Bignum Type (8 bits) | ——- | least significant bits | ——- . . .

The elements contain the two's complement representation of the integer with the least significant bits in the first element or closer to the header. The sign information is in the high end of the last element.

;;;; Code Data-Blocks.

A code data-block is the run-time representation of a "component". A component is a connected portion of a program's flow graph that is compiled as a single unit, and it contains code for many functions. Some of these functions are callable from outside of the component, and these are termed "entry points".

Each entry point has an associated user-visible function data-block (of type FUNCTION). The full call convention provides for calling an entry point specified by a function object.

Although all of the function data-blocks for a component's entry points appear to the user as distinct objects, the system keeps all of the code in a single code

data-block. The user-visible function object is actually a pointer into the middle of a code data-block. This allows any control transfer within a component to be done using a relative branch.

Besides a function object, there are other kinds of references into the middle of a code data-block. Control transfer into a function also occurs at the return-PC for a call. The system represents a return-PC somewhat similarly to a function, so GC can also recognize a return-PC as a reference to a code data-block.

It is incorrect to think of a code data-block as a concatenation of "function data-blocks". Code for a function is not emitted in any particular order with respect to that function's function-header (if any). The code following a function-header may only be a branch to some other location where the function's "real" definition is.

The following are the three kinds of pointers to code data-blocks: Code pointer (labeled A below): A code pointer is a descriptor, with other-pointer low-tag bits, pointing to the beginning of the code data-block. The code pointer for the currently running function is always kept in a register (CODE). In addition to allowing loading of non-immediate constants, this also serves to represent the currently running function to the debugger. Return-PC (labeled B below): The return-PC is a descriptor, with other-pointer low-tag bits, pointing to a location for a function call. Note that this location contains no descriptors other than the one word of immediate data, so GC can treat return-PC locations the same as instructions. Function (labeled C below): A function is a descriptor, with function low-tag bits, that is user callable. When a function header is referenced from a closure or from the function header's self-pointer, the pointer has other-pointer low-tag bits, instead of function low-tag bits. This ensures that the internal function data-block associated with a closure appears to be uncallable (although users should never see such an object anyway).

Information about functions that is only useful for entry points is kept in some descriptors following the function's self-pointer descriptor. All of these together with the function's header-word are known as the "function header". GC must be able to locate the function header. We provide for this by chaining together the function headers in a NIL terminated list kept in a known slot in the code data-block.

A code data-block has the following format: ++++++

| ——————————————————————————————————————- | First instruction after return | ——————————————————————————————————————- | ... more code and return-PC header-words | ——————————————————————————————————————- | (pad to dual-word boundary if necessary) | ++++++

Function entry point header-word: The immediate header-word data is the word offset from the enclosing code data-block's header-word to this word. This is the same as for the retrun-PC header-word. Self-pointer back to header-word: In a non-closure function, this self-pointer to the previous header-word allows the call sequence to always indirect through the second word in a user callable function. See section "Closure Format". With a closure, indirecting through the second word gets you a function header-word. The system ignores this slot in the function header for a closure, since it has already indirected once, and this slot could be some random thing that causes an error if you jump to it. This pointer has an other-pointer tag instead of a function pointer tag, indicating it is not a user callable Lisp object. The system accesses this slot with the system constant, %Function-Code-Slot, offset from the function header-word. Pointer to next function: This is the next link in the thread of entry point functions found in this component. This value is NIL when the current header is the last entry point in the component. The system accesses this slot with the system constant, %Function-Header-Next-Slot, offset from the function header-word. Function name: This function's name (for printing). If the user defined this function with DEFUN, then this is the defined symbol, otherwise it is a descriptive string. The system accesses this slot with the system constant, %Function-Header-Name-Slot, offset from the function header-word. Function debug arglist: A printed string representing the function's argument list, for human readability. If it is a macroexpansion function, then this is the original DEFMACRO arglist, not the actual expander function arglist. The system accesses this slot with the system constant, %Function-Header-Debug-Arglist-Slot, offset from the function header-word. Function type: A list-style function type specifier representing the argument signature and return types for this function. For example, (FUNCTION (FIXNUM FIXNUM FIXNUM) FIXNUM) or (FUNCTION (STRING &KEY (:START UNSIGNED-BYTE)) STRING) This information is intended for machine readablilty, such as by the compiler. The system accesses this slot with the system constant, %Function-Header-Type-Slot, offset from the function header-word.

;;;; Closure Format.

A closure data-block has the following format: ——————————————————————————————————- | Word size (24 bits) | %Closure-Type (8 bits) | ——- | Pointer to function header (other-pointer low-tag) | ——————————————————————————————————————- | . | | Environment information | | . | ——————————————————————————————————————-

A closure descriptor has function low-tag bits. This means that a descriptor with function low-tag bits may point to either a function header or to a closure. The idea is that any callable Lisp object has function low-tag bits. Insofar as call is concerned, we make the format of closures and non-closure functions compatible. This is the reason for the self-pointer in a function header. Whenever you have a callable object, you just jump through the second word, offset some bytes, and go.

;;;; Function call.

Due to alignment requirements and low-tag codes, it is not possible to use a hardware call instruction to compute the return-PC. Instead the return-PC for a call is computed by doing an add-immediate to the start of the code data-block.

An advantage of using a single data-block to represent both the descriptor and non-descriptor parts of a function is that both can be represented by a single pointer. This reduces the number of memory accesses that have to be done in a full call. For example, since the constant pool is implicit in a return-PC, a call need only save the return-PC, rather than saving both the return PC and the constant pool.

;;;; Memory Layout.

CMU Common Lisp has four spaces, read-only, static, dynamic-0, and dynamic-1. Read-only contains objects that the system never modifies, moves, or reclaims. Static space contains some global objects necessary for the system's runtime or performance (since they are located at a known offset at a know address), and the system never moves or reclaims these. However, GC does need to scan static space for references to moved objects. Dynamic-0 and dynamic-1 are the two heap areas for stop-and-copy GC algorithms.

What global objects are at the head of static space??? NIL eval::*top-of-stack* lisp::*current-catch-block* lisp::*current-unwind-protect* FLAGS (RT only) BSP (RT only) HEAP (RT only)

In addition to the above spaces, the system has a control stack, binding stack, and a number stack. The binding stack contains pairs of descriptors, a symbol and its previous value. The number stack is the same as the C stack, and the system uses it for non-Lisp objects such as raw system pointers, saving non-Lisp registers, parts of bignum computations, etc.

;;;; System Pointers.

The system pointers reference raw allocated memory, data returned by foreign function calls, etc. The system uses these when you need a pointer to a non-Lisp block of memory, using an other-pointer. This provides the greatest flexibility by relieving contraints placed by having more direct references that require descriptor type tags.

A system area pointer data-block has the following format: ——————————————————————- | 1 (data-block words) | SAP Type (8 bits) | ——————————————————————————- | system area pointer | ——————————————————————————————-

"SAP" means "system area pointer", and much of our code contains this naming scheme. We don't currently restrict system pointers to one area of memory, but if they do point onto the heap, it is up to the user to prevent being screwed by GC or whatever.

1 Compiler Overview

The structure of the compiler may be broadly characterized by describing the compilation phases and the data structures that they manipulate. The steps in the compilation are called phases rather than passes since they don't necessarily involve a full pass over the code. The data structure used to represent the code at some point is called an *Intermediate representation* (IR).

Two major intermediate representations are used in the compiler:

- The *implicit continuation representation* (ICR) represents the lisp-level semantics of the source code during the initial phases. Partial evaluation and semantic analysis are done on this representation. ICR is roughly equivalent to a subset of Common Lisp, but is represented as a flow-graph rather than a syntax tree. Phases which only manipulate ICR comprise the "front end". It would be possible to use a different back end such as one that directly generated code for a stack machine.
- The *virtual machine representation* (VMR) represents the implementation of the source code on a virtual machine. The virtual machine may vary depending on the the target hardware, but VMR is sufficiently stylized that most of the phases which manipulate it are portable.

Each phase is briefly described here. The phases from *local call analysis* to *constraint propagation* all interact; for maximum optimization, they are generally repeated until nothing new is discovered. The source files which primarily contain each phase are listed after "Files: ".

ICR conversion
: Convert the source into ICR, doing macroexpansion and simple source-to-source transformation. All names are resolved at this time, so we don't have to worry about name conflicts later on.

  Files:

  `ir1opt.lisp, ir1report.lisp, ir1tran-lambda.lisp, ir1tran.lisp, ir1-translators.lisp, ir1util.lisp, srctran, typetran`

Local call analysis
: Find calls to local functions and convert them to local calls to the correct entry point, doing keyword parsing, etc. Recognize once-called functions as lets. Create *external entry points* for entry-point functions. Files: `locall.lisp`

Find components
: Find flow graph components and compute depth-first ordering. Separate top-level code from run-time code, and determine which components are top-level components. Files: `dfo.lisp`

ICR optimize
: A grab-bag of all the non-flow ICR optimizations. Fold constant functions, propagate types and eliminate code that computes unused values. Special-case calls to some known global functions by replacing

them with a computed function. Merge blocks and eliminate IF-IFs. Substitute let variables.

Files:

`ir1opt.lisp, ir1tran.lisp, typetran.lisp, seqtran.lisp, generic/vm-tran.lisp`

Type constraint propagation
: Use global flow analysis to propagate information about lexical variable types. Eliminate unnecessary type checks and tests. Files: `constraint.lisp`

Type check generation
: Emit explicit ICR code for any necessary type checks that are too complex to be easily generated on the fly by the back end. Files: `checkgen.lisp`

Event driven operations
: Various parts of ICR are incrementally recomputed, either eagerly on modification of the ICR, or lazily, when the relevant information is needed.

- Check that type assertions are satisfied, marking places where type checks need to be done.
- Locate let calls.
- Delete functions and variables with no references

Files: `ir1util.lisp`, `ir1opt.lisp`

ICR finalize
: This phase is run after all components have been compiled. It scans the global variable references, looking for references to undefined variables and incompatible function redefinitions. Files: `ir1final.lisp`, `main.lisp`.

Environment analysis
: Determine which distinct environments need to be allocated, and what context needed to be closed over by each environment. We detect non-local exits and set closure variables. We also emit cleanup code as funny function calls. This is the last pure ICR pass. Files: `envanal.lisp`

Global TN allocation (GTN)
: Iterate over all defined functions, determining calling conventions and assigning TNs to local variables. Files: `gtn.lisp`

Local TN allocation (LTN)
: Use type and policy information to determine which VMR translation to use for known functions, and then create TNs for expression evaluation temporaries. We also accumulate some random information needed by VMR conversion. Files: `ltn.lisp`

Control analysis
: Linearize the flow graph in a way that minimizes the number of branches. The block-level structure of the flow graph is basically frozen at this point. Files: `control`

Stack analysis
: Maintain stack discipline for unknown-values continuation in the presence of local exits. Files: `stack.lisp`

Entry analysis
: Collect some back-end information for each externally callable function.

VMR conversion
: Convert ICR into VMR by translating nodes into VOPs. Emit type checks. Files: `ir2tran.lisp`, `vmdef.lisp`

Copy propagation
: Use flow analysis to eliminate unnecessary copying of TN values. Files: `copyprop.lisp`

Representation selection
: Look at all references to each TN to determine which representation has the lowest cost. Emit appropriate move and coerce VOPs for that representation.

Lifetime analysis
: Do flow analysis to find the set of TNs whose lifetimes overlap with the lifetimes of each TN being packed. Annotate call VOPs with the TNs that need to be saved. Files: `life.lisp`

Pack
: Find a legal register allocation, attempting to minimize unnecessary moves. Files: `pack.lisp`

Code generation
: Call the VOP generators to emit assembly code. Files: `codegen.lisp`

Pipeline reorganization
: On some machines, move memory references backward in the code so that they can overlap with computation. On machines with delayed branch instructions, locate instructions that can be moved into delay slots. Files: `assem-opt`

Assembly
: Resolve branches and convert into object code and fixup information. Files: `assembler.lisp`

Dumping
: Convert the compiled code into an object file or in-core function. Files: `debug-dump.lisp`, `dump.lisp`, `generic/core.lisp`

The Implicit Continuation Representation

The set of special forms recognized is exactly that specified in the Common Lisp manual. Everything that is described as a macro in CLTL is a macro. (EEK: can we say the same about the ANSI standard?)

Large amounts of syntactic information is thrown away by the conversion to an anonymous flow graph representation. The elimination of names eliminates the need to represent most environment manipulation special forms. The explicit representation of control eliminates the need to represent BLOCK and GO, and makes flow analysis easy. The full Common Lisp LAMBDA is implemented with a simple fixed-arg lambda, which greatly simplifies later code.

The elimination of syntactic information eliminates the need for most of the *beta transformation* optimizations in Rabbit. There are no PROGNs, no TAGBODYs and no RETURNs. There are no "close parens" which get in the way of determining which node receives a given value.

<Add reference for: Rabbit: A Compiler for Scheme)

In ICR, computation is represented by Nodes. These are the node types:

if Represents all conditionals.

set Represents a ⟨undefined⟩ "setq", page ⟨undefined⟩.

ref Represents a constant or variable reference.

combination
Represents a normal function call.

mv-combination
Represents a ⟨undefined⟩ "multiple-value-call", page ⟨undefined⟩. This is used to implement all multiple value receiving forms except for ⟨undefined⟩ "multiple-value-prog1", page ⟨undefined⟩, which is implicit.

bind This represents the allocation and initialization of the variables in a lambda.

return This collects the return value from a lambda and represents the control transfer on return.

entry Marks the start of a dynamic extent that can have non-local exits to it. Dynamic state can be saved at this point for restoration on re-entry.

exit Marks a potentially non-local exit. This node is interposed between the non-local uses of a continuation and the *dest* so that code to do a non-local exit can be inserted if necessary.

Some slots are shared between all node types (via *defstruct* inheritance.) This information held in common between all nodes often makes it possible to avoid special-casing nodes on the basis of type. This shared information is primarily concerned with the order of evaluation and destinations and properties of results. This

control and value flow is indicated in the node primarily by pointing to continuations.

The `continuation` structure represents information sufficiently related to the normal notion of a continuation that naming it so seems sensible. Basically, a continuation represents a place in the code, or alternatively the destination of an expression result and a transfer of control. These two notions are bound together for the same reasons that they are related in the standard functional continuation interpretation.

A continuation may be deprived of either or both of its value or control significance. If the value of a continuation is unused due to evaluation for effect, then the continuation will have a null *dest*. If the *next* node for a continuation is deleted by some optimization, then *next* will be *:none*.

[\#\#\# Continuation kinds...]

The *block* structure represents a basic block, in the the normal sense. Control transfers other than simple sequencing are represented by information in the block structure. The continuation for the last node in a block represents only the destination for the result.

It is very difficult to reconstruct anything resembling the original source from ICR, so we record the original source form in each node. The location of the source form within the input is also recorded, allowing for interfaces such as commandEdit Compiler Warnings. See section See Chapter 14 “source-paths”, page 131.

Forms such as `special-bind` and catch need to have cleanup code executed at all exit points from the form. We represent this constraint in ICR by annotating the code syntactically within the form with a cleanup structure describing what needs to be cleaned up. Environment analysis determines the cleanup locations by watching for a change in the cleanup between two continuations. We can't emit cleanup code during ICR conversion, since we don't know which exits will be local until after ICR optimizations are done.

Special binding is represented by a call to the funny function *%special-bind*. The first argument is the `global-var` structure for the variable bound and the second argument is the value to bind it to.

Some subprimitives are implemented using a macro-like mechanism for translating `%primitive` forms into arbitrary lisp code. Subprimitives 'special-cased' by VMR conversion are represented by a call to the funny function `%%primitive`. The corresponding `template` structure is passed as the first argument.

We check global function calls for syntactic legality with respect to any defined function type function. If the call is illegal or we are unable to tell if it is legal due to non-constant keywords, then we give a warning and mark the function reference as `:notinline` to force a full call and cause subsequent phases to ignore the call. If the call is legal and is to a known function, then we annotate the Combination node with the Function-Info structure that contains the compiler information for the function.

Tail sets

<EEK: was this done?> Probably want to have a GTN-like function result equivalence class mechanism for ICR type inference. This would be like the return value propagation being done by Propagate-From-Calls, but more powerful, less hackish, and known to terminate. The ICR equivalence classes could probably be used by GTN, as well.

What we do is have local call analysis eagerly maintain the equivalence classes of functions that return the same way by annotating functions with a Tail-Info structure shared between all functions whose value could be the value of this function. We don't require that the calls actually be tail-recursive, only that the call deliver its value to the result continuation. [\#\#\# Actually now done by `ir1-optimize-return`, which is currently making ICR optimize mandatory.]

We can then use the `tail-set` during ICR type inference. It would have a type that is the union across all equivalent functions of the types of all the uses other than in local calls. This type would be recomputed during optimization of `return` nodes. When the type changes, we would propagate it to all calls to any of the equivalent functions. How do we know when and how to recompute the type for a tail-set? Recomputation is driven by type propagation on the result continuation.

This is really special-casing of `return` nodes. The `return` node has the type which is the union of all the non-call uses of the result. The tail-set is found though the lambda. We can then recompute the overall union by taking the union of the type per return node, rather than per-use.

How do result type assertions work? We can't intersect the assertions across all functions in the equivalence class, since some of the call combinations may not happen (or even be possible). We can intersect the assertion of the result with the derived types for non-call uses.

When we do a tail call, we obviously can't check that the returned value matches our assertion. Although in principle, we would like to be able to check all assertions, to preserve system integrity, we only need to check assertions that we depend on. We can afford to lose some assertion information as long as we entirely lose it, ignoring it for type inference as well as for type checking.

Things will work out, since the caller will see the tail-info type as the derived type for the call, and will emit a type check if it needs a stronger result.

A remaining question is whether we should intersect the assertion with per-`return` derived types from the very beginning (i.e. before the type check pass). I think the answer is yes. We delay the type check pass so that we can get our best guess for the derived type before we decide whether a check is necessary. But with the function return type, we aren't committing to doing any type check when we intersect with the type assertion; the need to type check is still determined in the type check pass by examination of the result continuation.

What is the relationship between the per-`return` types and the types in the result continuation? The assertion is exactly the `continuation-asserted-type` (note that the asserted type of result continuations will never change after ICR conversion). <EEK: Also note that it seems `use-union` is used instead> The per-`return` derived type is different than the Continuation-Derived-Type, since it is intersected

with the asserted type even before `Type Check` runs. Ignoring the `continuation-asserted-type` <struct accessor defined in `node.lisp` in CMUCL but not SBCL> probably makes life simpler anyway, since this breaks the potential circularity of the `tail-info-type` <EEK: in SBCL the only thing we can find is `return-info-types`. In CMUCL there is the `tail-set` struct in `node.lisp` which has `type` and `info` slots> will affecting the `continuation-derived-type` <EEK: No longer found in SBCL. In CMUCL was in `ir1opt.lisp` *%continuation-derived-type*>, which affects...

When a given return has no non-call uses, we represent this by using `*empty-type*`. This is consistent with the interpretation that a return type of `nil` means the function can't return.

Hairy function representation

Non-`fixed-arg` functions are represented using `optional-dispatch`. An `optional-dispatch` has an entry-point function for each legal number of optionals, and one for when extra args are present. Each entry point function is a simple lambda. The entry point function for an optional is passed the arguments which were actually supplied; the entry point function is expected to default any remaining parameters and evaluate the actual function body.

If no `supplied-p` arg is present, then we can do this fairly easily by having each entry point supply its default and call the next entry point, with the last entry point containing the body. If there are `supplied-p` args, then entry point function is replaced with a function that calls the original entry function with `t`'s inserted at the position of all the supplied args with `supplied-p` parameters.

We want to be a bit clever about how we handle arguments declared special when doing optional defaulting, or we will emit really gross code for special optionals. If we bound the arg specially over the entire entry-point function, then the entry point function would be caused to be non-tail-recursive. What we can do is only bind the variable specially around the evaluation of the default, and then read the special and store the final value of the special into a lexical variable which we then pass as the argument. In the common case where the default is a constant, we don't have to special-bind at all, since the computation of the default is not affected by and cannot affect any special bindings.

Keyword and rest args are both implemented using a LEXPR-like 'more args' convention. The `more-entry` takes two arguments in addition to the fixed and optional arguments: the argument context and count. `(ARG <context> <n>)` accesses the N'th additional argument. Keyword args are implemented directly using this mechanism. Rest args are created by calling `%listify-rest-args` with the context and count.

The `more-entry` parses the keyword arguments and passes the values to the main function as positional arguments. If a keyword default is not constant, then we pass a `supplied-p` parameter into the main entry and let it worry about defaulting the argument. Since the main entry accepts keywords in parsed form, we can parse keywords at compile time for calls to known functions. We keep around the original

parsed lambda-list and related information so that people can figure out how to call the main entry.

ICR representation of non-local exits

All exits are initially represented by `exit` nodes:

How about an `exit` node:

```
(defstruct (exit (:include node))
  value)
```

The `exit` node uses the continuation that is to receive the thrown Value. During optimization, if we discover that the Cont's home-lambda is the same as the `exit` node's, then we can delete the `exit` node, substituting the Cont for all of the Value's uses.

The successor block of an exit is the entry block in the entered environment. So we use the `exit` node to mark the place where exit code is inserted. During environment analysis, we need only insert a single block containing the entry point stub.

We ensure that all Exits that aren't for a *non-local exit* NLX don't have any Value, so that local exits never require any value massaging.

The Entry node marks the beginning of a block or tagbody:

```
(defstruct (entry (:include node))
  (continuations nil :type list))
```

It contains a list of all the continuations that the body could exit to. The `entry` node is used as a marker for the place to snapshot state, including the control stack pointer. Each lambda has a list of its Entries so that environment analysis can figure out which continuations are really being closed over. There is no reason for optimization to delete `entry` nodes, since they are harmless in the degenerate case: we just emit no code (like a no-var `let`).

We represent CATCH using the lexical exit mechanism. We do a transformation like this:

```
(catch 'foo xxx)  ==>
(block #:foo
  (%catch #'(lambda () (return-from #:foo (%unknown-values))) 'foo)
  (%within-cleanup :catch
    xxx))
```

`%catch` just sets up the catch frame which points to the exit function. `%catch` is an ordinary function as far as ICR is concerned. The fact that the catcher needs to be cleaned up is expressed by the Cleanup slots in the continuations in the body. `%unknown-values` is a dummy function call which represents the fact that we don't know what values will be thrown.

`%within-cleanup` is a special special form that instantiates its first argument as the current cleanup when converting the body. In reality, the lambda is also created by the special special form `%escape-function`, which gives the lambda a special `:escape` kind so that the back end knows not to generate any code for it.

We use a similar hack in `unwind-protect` to represent the fact that the cleanup forms can be invoked at arbitrarily random times.

```
        (unwind-protect p c)  ==>
        (flet ((#:cleanup () c))
          (block #:return
    (multiple-value-bind
        (#:next #:start #:count)
        (block #:unwind
                  (%unwind-protect #'(lambda (x) (return-from #:unwind x)))
                  (%within-cleanup :unwind-protect
    (return-from #:return p)))
      (#:cleanup)
              (%continue-unwind #:next #:start #:count))))
```

We use the block `#:unwind` to represent the entry to cleanup code in the case where we are non-locally unwound. Calling of the cleanup function in the drop-through case (or any local exit) is handled by cleanup generation. We make the cleanup a function so that cleanup generation can add calls at local exits from the protected form. `#:next`, `#:start` and `#:count` are state used in the case where we are unwound. They indicate where to go after doing the cleanup and what values are being thrown. The cleanup encloses only the protected form. As in `catch`, the escape function is specially tagged as `:escape`. The cleanup function is tagged as `:cleanup` to inhibit let conversion (since references are added in environment analysis.)

Notice that implementing these forms using closures over continuations eliminates any need to special-case ICR flow analysis. Obviously we don't really want to make heap-closures here. In reality these functions are special-cased by the back-end according to their KIND.

Block compilation

One of the properties of ICR is that it supports *block compilation* by allowing arbitrarily large amounts of code to be converted at once, with actual compilation of the code being done at will.

In order to preserve the normal semantics we must recognize that proclamations (possibly implicit) are scoped. A proclamation is in effect only from the time of appearance of the proclamation to the time it is contradicted. The current global environment at the end of a block is not necessarily the correct global environment for compilation of all the code within the block. We solve this problem by closing over the relevant information in the ICR at the time it is converted. For example, each functional variable reference is marked as inline, notinline or don't care. Similarly, each node contains a structure known as a "cookie" which contains the appropriate settings of the compiler policy switches.

We actually convert each form in the file separately, creating a separate "initial component" for each one. Later on, these components are merged as needed. The main reason for doing this is to cause `eval-when` processing to be interleaved with reading.

Entry points

Since we need to evaluate potentially arbitrary code in the *External Entry Point* (XEP) argument forms (for type checking), we can't leave the arguments in the wired passing locations. Instead, it seems better to give the XEP max-args fixed arguments, with the passing locations being the true passing locations. Instead of using `%xep-arg`, we reference the appropriate variable.

Also, it might be a good idea to do argument count checking and dispatching with explicit conditional code in the XEP. This would simplify both the code that creates the XEP and the VMR conversion of XEPs. Also, argument count dispatching would automatically benefit from any cleverness in compilation of case-like forms (jump tables, etc). On the downside, this would push some assumptions about how arg dispatching is done into ICR. But then we are currently violating abstraction at least as badly in VMR conversion, which is also supposed to be implementation independent.

As a side-effect of finding which references to known functions can be converted to local calls, we find any references that cannot be converted. References that cannot be converted to a local call must evaluate to a "function object" (or function-entry) that can be called using the full call convention. A function that can be called from outside the component is called an "entry-point".

Lots of stuff that happens at compile-time with local function calls must be done at run-time when an entry-point is called.

It is desirable for optimization and other purposes if all the calls to every function were directly present in ICR as local calls. We cannot directly do this with entry-point functions, since we don't know where and how the entry-point will be called until run-time.

What we do is represent all the calls possible from outside the component by local calls within the component. For each entry-point function, we create a corresponding lambda called the external entry point or XEP. This is a function which takes the number of arguments passed as the first argument, followed by arguments corresponding to each required or optional argument.

If an optional argument is unsupplied, the value passed into the XEP is undefined. The XEP is responsible for doing argument count checking and dispatching.

In the case of a fixed-arg lambda, we emit a call to the `verify-arg-count` funny function (conditional on policy), then call the real function on the passed arguments. Even in this simple case, we benefit several ways from having a separate XEP:

- The argument count checking is factored out, and only needs to be done in full calls.
- Argument type checking happens automatically as a consequence of passing the XEP arguments in a local call to the real function. This type checking is also only done in full calls.
- The real function may use a non-standard calling convention for the benefit of recursive or block-compiled calls. The XEP converts arguments/return values to/from the standard convention. This also requires little special-casing of XEPs.

If the function has variable argument count (represented by an `optional-dispatch`), then the XEP contains a `cond` which dispatches off of the argument count, calling the appropriate entry-point function (which then does defaulting). If there is a more entry (for keyword or rest args), then the XEP obtains the more arg context and count by calling the `%more-arg-context` funny function.

All non-local-call references to functions are replaced with references to the corresponding XEP. ICR optimization may discover a local call that was previously a non-local reference. When we delete the reference to the XEP, we may find that it has no references. In this case, we can delete the XEP, causing the function to no longer be an entry-point.

Compiler Front End

2 Front End: ICR Conversion

2.1 Canonical forms

Would be useful (in fact we know have in SBCL) to have a `freeze-type` proclamation. Its primary use would be to say that the indicated type won't acquire any new subtypes in the future. This allows better open-coding of structure type predicates, since the possible types that would satisfy the predicate will be constant at compile time, and thus can be compiled as a skip-chain of #codeeq tests.

Of course, this is only a big win when the subtypes are few: the most important case is when there are none. If the closure of the subtypes is much larger than the average number of supertypes of an inferior, then it is better to grab the list of superiors out of the object's type, and test for membership in that list.

Should type-specific numeric equality be done by `eql` rather than `=?` i.e. should `=` on two fixnums become `eql` and then convert to `eql/fixnum`? Currently we transform `eql` into `=`, which is complicated, since we have to prove the operands are the class of numeric type before we do it. Also, when `eql` sees one operand is a `fixnum`, it transforms to `eq`, but the generator for `eq` isn't expecting numbers, so it doesn't use an immediate compare.

2.1.1 Array hackery

Array type tests are transformed to `%array-typep`, separation of the implementation-dependent array-type handling. This way we can transform `stringp` to:

```
(or (simple-string-p x)
    (and (complex-array-p x)
         (= (array-rank x) 1)
         (simple-string-p (%array-data x))))
```

In addition to the similar bit-vector-p, we also handle vectorp and any type tests on which the a dimension isn't wild. [Note that we will want to expand into frobs compatible with those that array references expand into so that the same optimizations will work on both.]

These changes combine to convert hairy type checks into hairy typep's, and then convert hairyp typeps into simple typeps.

Do we really need non-VOP templates? It seems that we could get the desired effect through implementation-dependent ICR transforms. The main risk would be of obscuring the type semantics of the code. We could fairly easily retain all the type information present at the time the tranform is run, but if we discover new type information, then it won't be propagated unless the VM also supplies type inference methods for its internal frobs (precluding the use of \verb|%PRIMITIVE|, since primitives don't have derive-type methods.)

I guess one possibility would be to have the call still considered "known" even though it has been transformed. But this doesn't work, since we start doing LET

optimizations that trash the arglist once the call has been transformed (and indeed we want to.)

Actually, I guess the overhead for providing type inference methods for the internal frobs isn't that great, since we can usually borrow the inference method for a Common Lisp function. For example, in our AREF case:

\beginverbatim (aref x y) ==> (let ((#:len (array-dimension x 0))) (%unchecked-aref x (%check-in-bounds y #:len))) \endverbatim

Now in this case, if we made \verb|%UNCHECKED-AREF| have the same derive-type method as AREF, then if we discovered something new about X's element type, we could derive a new type for the entire expression.

Actually, it seems that baring this detail at the ICR level is beneficial, since it admits the possibility of optimizing away the bounds check using type information. If we discover X's dimensions, then \verb|#:LEN| becomes a constant that can be substituted. Then \verb|%CHECK-IN-BOUNDS| can notice that the bound is constant and check it against the type for Y. If Y is known to be in range, then we can optimize away the bounds check.

Actually in this particular case, the best thing to do would be if we discovered the bound is constant, then replace the bounds check with an implicit type check. This way all the type check optimization mechanisms would be brought into the act.

So we actually want to do the bounds-check expansion as soon as possible, rather than later than possible: it should be a source-transform, enabled by the fast-safe policy.

With multi-dimensional arrays we probably want to explicitly do the index computation: this way portions of the index computation can become loop invariants. In a scan in row-major order, the inner loop wouldn't have to do any multiplication: it would only do an addition. We would use normal fixnum arithmetic, counting on * to cleverly handle multiplication by a constant, and appropriate inline expansion.

Note that in a source transform, we can't make any assumptions the type of the array. If it turns out to be a complex array without declared dimensions, then the calls to ARRAY-DIMENSION will have to turn into a VOP that can be affected. But if it is simple, then the VOP is unaffected, and if we know the bounds, it is constant. Similarly, we would have %ARRAY-DATA and %ARRAY-DISPLACEMENT operations. %ARRAY-DISPLACEMENT would optimize to 0 if we discover the array is simple. [This is somewhat inefficient when the array isn't eventually discovered to be simple, since finding the data and finding the displacement duplicate each other. We could make %ARRAY-DATA return both as MVs, and then optimize to (VALUES (%SIMPLE-ARRAY-DATA x) 0), but this would require optimization of trivial VALUES uses.]

Also need (THE (ARRAY * * * ...) x) to assert correct rank.

A bunch of functions have source transforms that convert them into the canonical form that later parts of the compiler want to see. It is not legal to rely on the canonical form since source transforms can be inhibited by a Notinline declaration. This shouldn't be a problem, since everyone should keep their hands off of Notinline calls.

Some transformations: \beginverbatim Endp ==> (NULL (THE LIST ...)) (NOT xxx) or (NULL xxx) => (IF xxx NIL T)

(typep x '<simple type>) => (<simple predicate> x) (typep x '<complex type>) => ...composition of simpler operations... \endverbatim

TYPEP of AND, OR and NOT types turned into conditionals over multiple TYPEP calls. This makes hairy TYPEP calls more digestible to type constraint propagation, and also means that the TYPEP code generators don't have to deal with these cases. [\#\#\# In the case of union types we may want to do something to preserve information for type constraint propagation.]

\beginverbatim (apply \#'foo a b c) ==> (multiple-value-call \#'foo (values a) (values b) (values-list c)) \endverbatim

This way only MV-CALL needs to know how to do calls with unknown numbers of arguments. It should be nearly as efficient as a special-case VMR-Convert method could be.

\beginverbatim Make-String => Make-Array N-arg predicates associated into two-arg versions. Associate N-arg arithmetic ops. Expand CxxxR and FIRST...nTH Zerop, Plusp, Minusp, 1+, 1-, Min, Max, Rem, Mod (Values x), (Identity x) => (Prog1 x)

All specialized aref functions => (aref (the xxx) ...) \endverbatim

Convert (ldb (byte ...) ...) into internal frob that takes size and position as separate args. Other byte functions also...

Change for-value primitive predicates into \verb+(if <pred> t nil)+. This isn't particularly useful during ICR phases, but makes life easy for VMR conversion.

This last can't be a source transformation, since a source transform can't tell where the form appears. Instead, ICR conversion special-cases calls to known functions with the Predicate attribute by doing the conversion when the destination of the result isn't an IF. It isn't critical that this never be done for predicates that we ultimately discover to deliver their value to an IF, since IF optimizations will flush unnecessary IFs in a predicate.

\sectionInline functions

[\#\#\# Inline expansion is especially powerful in the presence of good lisp-level optimization ("partial evaluation"). Many "optimizations" usually done in Lisp compilers by special-case source-to-source transforms can be had simply by making the source of the general case function available for inline expansion. This is especially helpful in Common Lisp, which has many commonly used functions with simple special cases but bad general cases (list and sequence functions, for example.)

Inline expansion of recursive functions is allowed, and is not as silly as it sounds. When expanded in a specific context, much of the overhead of the recursive calls may be eliminated (especially if there are many keyword arguments, etc.)

[Also have MAYBE-INLINE]]

We only record a function's inline expansion in the global environment when the function is in the null lexical environment, since the expansion must be represented as source.

We do inline expansion of functions locally defined by FLET or LABELS even when the environment is not null. Since the appearances of the local function must be nested within the desired environment, it is possible to expand local functions inline even when they use the environment. We just stash the source form and environments in the Functional for the local function. When we convert a call to it, we just reconvert the source in the saved environment.

An interesting alternative to the inline/full-call dichotomy is "semi-inline" coding. Whenever we have an inline expansion for a function, we can expand it only once per block compilation, and then use local call to call this copied version. This should get most of the speed advantage of real inline coding with much less code bloat. This is especially attractive for simple system functions such as Read-Char.

The main place where true inline expansion would still be worth doing is where large amounts of the function could be optimized away by constant folding or other optimizations that depend on the exact arguments to the call.

\sectionCompilation policy

We want more sophisticated control of compilation safety than is offered in CL, so that we can emit only those type checks that are likely to discover something (i.e. external interfaces.)

\sectionNotes

Generalized back-end notion provides dynamic retargeting? (for byte code)

The current node type annotations seem to be somewhat unsatisfactory, since we lose information when we do a THE on a continuation that already has uses, or when we convert a let where the actual result continuation has other uses.

But the case with THE isn't really all that bad, since the test of whether there are any uses happens before conversion of the argument, thus THE loses information only when there are uses outside of the declared form. The LET case may not be a big deal either.

Note also that losing user assertions isn't really all that bad, since it won't damage system integrity. At worst, it will cause a bug to go undetected. More likely, it will just cause the error to be signaled in a different place (and possibly in a less informative way). Of course, there is an efficiency hit for losing type information, but if it only happens in strange cases, then this isn't a big deal.

\chapterLocal call analysis

All calls to local functions (known named functions and LETs) are resolved to the exact LAMBDA node which is to be called. If the call is syntactically illegal, then we emit a warning and mark the reference as :notinline, forcing the call to be a full call. We don't even think about converting APPLY calls; APPLY is not special-cased at all in ICR. We also take care not to convert calls in the top-level component, which would join it to normal code. Calls to functions with rest args and calls with non-constant keywords are also not converted.

We also convert MV-Calls that look like MULTIPLE-VALUE-BIND to local calls, since we know that they can be open-coded. We replace the optional dispatch with a call to the last optional entry point, letting MV-Call magically default the unsupplied values to NIL.

When ICR optimizations discover a possible new local call, they explicitly invoke local call analysis on the code that needs to be reanalyzed.

[\#\#\# Let conversion. What it means to be a let. Argument type checking done by caller. Significance of local call is that all callers are known, so special call conventions may be used.] A lambda called in only one place is called a "let" call, since a Let would turn into one.

In addition to enabling various ICR optimizations, the let/non-let distinction has important environment significance. We treat the code in function and all of the lets called by that function as being in the same environment. This allows exits from lets to be treated as local exits, and makes life easy for environment analysis.

Since we will let-convert any function with only one call, we must be careful about cleanups. It is possible that a lexical exit from the let function may have to clean up dynamic bindings not lexically apparent at the exit point. We handle this by annotating lets with any cleanup in effect at the call site. The cleanup for continuations with no immediately enclosing cleanup is the lambda that the continuation is in. In this case, we look at the lambda to see if any cleanups need to be done.

Let conversion is disabled for entry-point functions, since otherwise we might convert the call from the XEP to the entry point into a let. Then later on, we might want to convert a non-local reference into a local call, and not be able to, since once a function has been converted to a let, we can't convert it back.

A function's return node may also be deleted if it is unreachable, which can happen if the function never returns normally. Such functions are not lets.

\chapterFind components

This is a post-pass to ICR conversion that massages the flow graph into the shape subsequent phases expect. Things done: Compute the depth-first ordering for the flow graph. Find the components (disconnected parts) of the flow graph.

This pass need only be redone when newly converted code has been added to the flow graph. The reanalyze flag in the component structure should be set by people who mess things up.

We create the initial DFO using a variant of the basic algorithm. The initial DFO computation breaks the ICR up into components, which are parts that can be compiled independently. This is done to increase the efficiency of large block compilations. In addition to improving locality of reference and reducing the size of flow analysis problems, this allows back-end data structures to be reclaimed after the compilation of each component.

ICR optimization can change the connectivity of the flow graph by discovering new calls or eliminating dead code. Initial DFO determination splits up the flow graph into separate components, but does so conservatively, ensuring that parts that might become joined (due to local call conversion) are joined from the start. Initial DFO computation also guarantees that all code which shares a lexical environment is in the same component so that environment analysis needs to operate only on a single component at a time.

[This can get a bit hairy, since code seemingly reachable from the environment entry may be reachable from a NLX into that environment. Also, function references

must be considered as links joining components even though the flow graph doesn't represent these.]

After initial DFO determination, components are neither split nor joined. The standard DFO computation doesn't attempt to split components that have been disconnected.

\chapterICR optimize

\bf Somewhere describe basic ICR utilities: continuation-type, constant-continuation-p, etc. Perhaps group by type in ICR description?

We are conservative about doing variable-for-variable substitution in ICR optimization, since if we substitute a variable with a less restrictive type, then we may prevent use of a "good" representation within the scope of the inner binding.

Note that variable-variable substitutions aren't really crucial in ICR, since they don't create opportunities for new optimizations (unlike substitution of constants and functions). A spurious variable-variable binding will show up as a Move operation in VMR. This can be optimized away by reaching-definitions and also by targeting. [\#\#\# But actually, some optimizers do see if operands are the same variable.]

The IF-IF optimization can be modeled as a value driven optimization, since adding a use definitely is cause for marking the continuation for reoptimization. [When do we add uses? Let conversion is the only obvious time.] I guess IF-IF conversion could also be triggered by a non-immediate use of the test continuation becoming immediate, but to allow this to happen would require Delete-Block (or somebody) to mark block-starts as needing to be reoptimized when a predecessor changes. It's not clear how important it is that IF-IF conversion happen under all possible circumstances, as long as it happens to the obvious cases.

[\#\#\# It isn't totally true that code flushing never enables other worthwhile optimizations. Deleting a functional reference can cause a function to cease being an XEP, or even trigger let conversion. It seems we still want to flush code during ICR optimize, but maybe we want to interleave it more intimately with the optimization pass.

Ref-flushing works just as well forward as backward, so it could be done in the forward pass. Call flushing doesn't work so well, but we could scan the block backward looking for any new flushable stuff if we flushed a call on the forward pass.

When we delete a variable due to lack of references, we leave the variable in the lambda-list so that positional references still work. The initial value continuation is flushed, though (replaced with NIL) allowing the initial value for to be deleted (modulo side-effects.)

Note that we can delete vars with no refs even when they have sets. I guess when there are no refs, we should also flush all sets, allowing the value expressions to be flushed as well.

Squeeze out single-reference unset let variables by changing the dest of the initial value continuation to be the node that receives the ref. This can be done regardless of what the initial value form is, since we aren't actually moving the evaluation.

Instead, we are in effect using the continuation's locations in place of the temporary variable.

Doing this is of course, a wild violation of stack discipline, since the ref might be inside a loop, etc. But with the VMR back-end, we only need to preserve stack discipline for unknown-value continuations; this ICR transformation must be already inhibited when the DEST of the REF is a multiple-values receiver (EXIT, RETURN or MV-COMBINATION), since we must preserve the single-value semantics of the let-binding in this case.

The REF and variable must be deleted as part of this operation, since the ICR would otherwise be left in an inconsistent state; we can't wait for the REF to be deleted due to being unused, since we have grabbed the arg continuation and substituted it into the old DEST.

The big reason for doing this transformation is that in macros such as INCF and PSETQ, temporaries are squeezed out, and the new value expression is evaluated directly to the setter, allowing any result type assertion to be applied to the expression evaluation. Unlike in the case of substitution, there is no point in inhibiting this transformation when the initial value type is weaker than the variable type. Instead, we intersect the asserted type for the old REF's CONT with the type assertion on the initial value continuation. Note that the variable's type has already been asserted on the initial-value continuation.

Of course, this transformation also simplifies the ICR even when it doesn't discover interesting type assertions, so it makes sense to do it whenever possible. This reduces the demands placed on register allocation, etc.

There are three dead-code flushing rules:

\beginenumerate \item Refs with no DEST may be flushed.

\item Known calls with no dest that are flushable may be flushed. We null the DEST in all the args.

\item If a lambda-var has no refs, then it may be deleted. The flushed argument continuations have their DEST nulled. \endenumerate

These optimizations all enable one another. We scan blocks backward, looking for nodes whose CONT has no DEST, then type-dispatching off of the node. If we delete a ref, then we check to see if it is a lambda-var with no refs. When we flush an argument, we mark the blocks for all uses of the CONT as needing to be reoptimized.

\sectionGoals for ICR optimizations

<When an optimization is disabled, code should still be correct and not ridiculously inefficient. Phases shouldn't be made mandatory when they have lots of non-required stuff jammed into them.>

This pass is optional, but is desirable if anything is more important than compilation speed.

This phase is a grab-bag of optimizations that concern themselves with the flow of values through the code representation. The main things done are type inference, constant folding and dead expression elimination. This phase can be understood as a walk of the expression tree that propagates assertions down the

tree and propagates derived information up the tree. The main complication is that there isn't any expression tree, since ICR is flow-graph based.

We repeat this pass until we don't discover anything new. This is a bit of feat, since we dispatch to arbitrary functions which may do arbitrary things, making it hard to tell if anything really happened. Even if we solve this problem by requiring people to flag when they changed or by checking to see if they changed something, there are serious efficiency problems due to massive redundant computation, since in many cases the only way to tell if anything changed is to recompute the value and see if it is different from the old one.

We solve this problem by requiring that optimizations for a node only depend on the properties of the CONT and the continuations that have the node as their DEST. If the continuations haven't changed since the last pass, then we don't attempt to re-optimize the node, since we know nothing interesting will happen.

We keep track of which continuations have changed by a REOPTIMIZE flag that is set whenever something about the continuation's value changes.

When doing the bottom up pass, we dispatch to type specific code that knows how to tell when a node needs to be reoptimized and does the optimization. These node types are special-cased: COMBINATION, IF, RETURN, EXIT, SET.

The REOPTIMIZE flag in the COMBINATION-FUN is used to detect when the function information might have changed, so that we know when there are new assertions that could be propagated from the function type to the arguments.

When we discover something about a leaf, or substitute for leaf, we reoptimize the CONT for all the REF and SET nodes.

We have flags in each block that indicate when any nodes or continuations in the block need to be re-optimized, so we don't have to scan blocks where there is no chance of anything happening.

It is important for efficiency purposes that optimizers never say that they did something when they didn't, but this by itself doesn't guarantee timely termination. I believe that with the type system implemented, type inference will converge in finite time, but as a practical matter, it can take far too long to discover not much. For this reason, ICR optimization is terminated after three consecutive passes that don't add or delete code. This premature termination only happens 2\% of the time.

\sectionFlow graph simplification

Things done:

\beginitemize \item Delete blocks with no predecessors. \item Merge blocks that can be merged. \item Convert local calls to Let calls. \item Eliminate degenerate IFs. \enditemize

We take care not to merge blocks that are in different functions or have different cleanups. This guarantees that non-local exits are always at block ends and that cleanup code never needs to be inserted within a block.

We eliminate IFs with identical consequent and alternative. This would most likely happen if both the consequent and alternative were optimized away.

[Could also be done if the consequent and alternative were different blocks, but computed the same value. This could be done by a sort of cross-jumping

optimization that looked at the predecessors for a block and merged code shared between predecessors. IFs with identical branches would eventually be left with nothing in their branches.]

We eliminate IF-IF constructs:

\beginverbatim (IF (IF A B C) D E) ==> (IF A (IF B D E) (IF C D E)) \endverbatim

In reality, what we do is replicate blocks containing only an IF node where the predicate continuation is the block start. We make one copy of the IF node for each use, leaving the consequent and alternative the same. If you look at the flow graph representation, you will see that this is really the same thing as the above source to source transformation.

\sectionForward ICR optimizations

In the forward pass, we scan the code in forward depth-first order. We examine each call to a known function, and:

\beginitemize \item Eliminate any bindings for unused variables.

\item Do top-down type assertion propagation. In local calls, we propagate asserted and derived types between the call and the called lambda.

\item Replace calls of foldable functions with constant arguments with the result. We don't have to actually delete the call node, since Top-Down optimize will delete it now that its value is unused.

\item Run any Optimizer for the current function. The optimizer does arbitrary transformations by hacking directly on the IR. This is useful primarily for arithmetic simplification and similar things that may need to examine and modify calls other than the current call. The optimizer is responsible for recording any changes that it makes. An optimizer can inhibit further optimization of the node during the current pass by returning true. This is useful when deleting the node.

\item Do ICR transformations, replacing a global function call with equivalent inline lisp code.

\item Do bottom-up type propagation/inferencing. For some functions such as Coerce we will dispatch to a function to find the result type. The Derive-Type function just returns a type structure, and we check if it is different from the old type in order to see if there was a change.

\item Eliminate IFs with predicates known to be true or false.

\item Substitute the value for unset let variables that are bound to constants, unset lambda variables or functionals.

\item Propagate types from local call args to var refs. \enditemize

We use type info from the function continuation to find result types for functions that don't have a derive-type method.

\subsectionICR transformation

ICR transformation does "source to source" transformations on known global functions, taking advantage of semantic information such as argument types and constant arguments. Transformation is optional, but should be done if speed or space is more important than compilation speed. Transformations which increase space should pass when space is more important than speed.

A transform is actually an inline function call where the function is computed at compile time. The transform gets to peek at the continuations for the arguments, and computes a function using the information gained. Transforms should be cautious about directly using the values of constant continuations, since the compiler must preserve eqlness of named constants, and it will have a hard time if transforms go around randomly copying constants.

The lambda that the transform computes replaces the original function variable reference as the function for the call. This lets the compiler worry about evaluating each argument once in the right order. We want to be careful to preserve type information when we do a transform, since it may be less than obvious what the transformed code does.

There can be any number of transforms for a function. Each transform is associated with a function type that the call must be compatible with. A transform is only invoked if the call has the right type. This provides a way to deal with the common case of a transform that only applies when the arguments are of certain types and some arguments are not specified. We always use the derived type when determining whether a transform is applicable. Type check is responsible for setting the derived type to the intersection of the asserted and derived types.

If the code in the expansion has insufficient explicit or implicit argument type checking, then it should cause checks to be generated by making declarations.

A transformation may decide to pass if it doesn't like what it sees when it looks at the args. The Give-Up function unwinds out of the transform and deals with complaining about inefficiency if speed is more important than brevity. The format args for the message are arguments to Give-Up. If a transform can't be done, we just record the message where ICR finalize can find it. note. We can't complain immediately, since it might get transformed later on.

\sectionBackward ICR optimizations

In the backward pass, we scan each block in reverse order, and eliminate any effectless nodes with unused values. In ICR this is the only way that code is deleted other than the elimination of unreachable blocks.

\chapterType checking

% Somehow split this section up into three parts: % – Conceptual: how we know a check is necessary, and who is responsible for % doing checks. % – Incremental: intersection of derived and asserted types, checking for % non-subtype relationship. % – Check generation phase.

We need to do a pretty good job of guessing when a type check will ultimately need to be done. Generic arithmetic, for example: In the absence of declarations, we will use the safe variant, but if we don't know this, we will generate a check for NUMBER anyway. We need to look at the fast-safe templates and guess if any of them could apply.

We compute a function type from the VOP arguments and assertions on those arguments. This can be used with Valid-Function-Use to see which templates do or might apply to a particular call. If we guess that a safe implementation will be used, then we mark the continuation so as to force a safe implementation to be chosen. [This will happen if ICR optimize doesn't run to completion, so the ICR

optimization after type check generation can discover new type information. Since we won't redo type check at that point, there could be a call that has applicable unsafe templates, but isn't type checkable.]

[\#\#\# A better and more general optimization of structure type checks: in type check conversion, we look at the *original derived* type of the continuation: if the difference between the proven type and the asserted type is a simple type check, then check for the negation of the difference. e.g. if we want a FOO and we know we've got (OR FOO NULL), then test for (NOT NULL). This is a very important optimization for linked lists of structures, but can also apply in other situations.]

If after ICR phases, we have a continuation with check-type set in a context where it seems likely a check will be emitted, and the type is too hairy to be easily checked (i.e. no CHECK-xxx VOP), then we do a transformation on the ICR equivalent to:

\beginverbatim (... (the hair <foo>) ...) ==> (... (funcall \#'(lambda (\#:val) (if (typep \#:val 'hair) \#:val (%type-check-error \#:val 'hair))) <foo>) ...) \endverbatim This way, we guarantee that VMR conversion never has to emit type checks for hairy types.

[Actually, we need to do a MV-bind and several type checks when there is a MV continuation. And some values types are just too hairy to check. We really can't check any assertion for a non-fixed number of values, since there isn't any efficient way to bind arbitrary numbers of values. (could be done with MV-call of a more-arg function, I guess...)]

[Perhaps only use CHECK-xxx VOPs for types equivalent to a ptype? Exceptions for CONS and SYMBOL? Anyway, no point in going to trouble to implement and emit rarely used CHECK-xxx vops.]

One potential lose in converting a type check to explicit conditionals rather than to a CHECK-xxx VOP is that VMR code motion optimizations won't be able to do anything. This shouldn't be much of an issue, though, since type constraint propagation has already done global optimization of type checks.

This phase is optional, but should be done if anything is more important than compile speed.

Type check is responsible for reconciling the continuation asserted and derived types, emitting type checks if appropriate. If the derived type is a subtype of the asserted type, then we don't need to do anything.

If there is no intersection between the asserted and derived types, then there is a manifest type error. We print a warning message, indicating that something is almost surely wrong. This will inhibit any transforms or generators that care about their argument types, yet also inhibits further error messages, since NIL is a subtype of every type.

If the intersection is not null, then we set the derived type to the intersection of the asserted and derived types and set the Type-Check flag in the continuation. We always set the flag when we can't prove that the type assertion is satisfied, regardless of whether we will ultimately actually emit a type check or not. This is so other phases such as type constraint propagation can use the Type-Check flag to detect an interesting type assertion, instead of having to duplicate much of the

work in this phase. [\#\#\# 7 extremely random values for CONTINUATION-TYPE-CHECK.]

Type checks are generated on the fly during VMR conversion. When VMR conversion generates the check, it prints an efficiency note if speed is important. We don't flame now since type constraint progpagation may decide that the check is unnecessary. [\#\#\# Not done now, maybe never.]

In local function call, it is the caller that is in effect responsible for checking argument types. This happens in the same way as any other type check, since ICR optimize propagates the declared argument types to the type assertions for the argument continuations in all the calls.

Since the types of arguments to entry points are unknown at compile time, we want to do runtime checks to ensure that the incoming arguments are of the correct type. This happens without any special effort on the part of type check, since the XEP is represented as a local call with unknown type arguments. These arguments will be marked as needing to be checked.

\chapterConstraint propagation

New lambda-var-slot:

constraints: a list of all the constraints on this var for either X or Y.

How to maintain consistency? Does it really matter if there are constraints with deleted vars lying around? Note that whatever mechanism we use for getting the constraints in the first place should tend to keep them up to date. Probably we would define optimizers for the interesting relations that look at their CONT's dest and annotate it if it is an IF.

But maybe it is more trouble then it is worth trying to build up the set of constraints during ICR optimize (maintaining consistency in the process). Since ICR optimize iterates a bunch of times before it converges, we would be wasting time recomputing the constraints, when nobody uses them till constraint propagation runs.

It seems that the only possible win is if we re-ran constraint propagation (which we might want to do.) In that case, we wouldn't have to recompute all the constraints from scratch. But it seems that we could do this just as well by having ICR optimize invalidate the affected parts of the constraint annotation, rather than trying to keep them up to date. This also fits better with the optional nature of constraint propagation, since we don't want ICR optimize to commit to doing a lot of the work of constraint propagation.

For example, we might have a per-block flag indicating that something happened in that block since the last time constraint propagation ran. We might have different flags to represent the distinction between discovering a new type assertion inside the block and discovering something new about an if predicate, since the latter would be cheaper to update and probably is more common.

It's fairly easy to see how we can build these sets of restrictions and propagate them using flow analysis, but actually using this information seems a bit more ad-hoc.

Probably the biggest thing we do is look at all the refs. If we have proven that the value is EQ (EQL for a number) to some other leaf (constant or lambda-var),

then we can substitute for that reference. In some cases, we will want to do special stuff depending on the DEST. If the dest is an IF and we proved (not null), then we can substitute T. And if the dest is some relation on the same two lambda-vars, then we want to see if we can show that relation is definitely true or false.

Otherwise, we can do our best to invert the set of restrictions into a type. Since types hold only constant info, we have to ignore any constraints between two vars. We can make some use of negated type restrictions by using TYPE-DIFFERENCE to remove the type from the ref types. If our inferred type is as good as the type assertion, then the continuation's type-check flag will be cleared.

It really isn't much of a problem that we don't infer union types on joins, since union types are relatively easy to derive without using flow information. The normal bottom-up type inference done by ICR optimize does this for us: it annotates everything with the union of all of the things it might possibly be. Then constraint propagation subtracts out those types that can't be in effect because of predicates or checks.

This phase is optional, but is desirable if anything is more important than compilation speed. We use an algorithm similar to available expressions to propagate variable type information that has been discovered by implicit or explicit type tests, or by type inference.

We must do a pre-pass which locates set closure variables, since we cannot do flow analysis on such variables. We set a flag in each set closure variable so that we can quickly tell that it is losing when we see it again. Although this may seem to be wastefully redundant with environment analysis, the overlap isn't really that great, and the cost should be small compared to that of the flow analysis that we are preparing to do. [Or we could punt on set variables...]

A type constraint is a structure that includes sset-element and has the type and variable. [Also a not-p flag indicating whether the sense is negated.]

Each variable has a list of its type constraints. We create a type constraint when we see a type test or check. If there is already a constraint for the same variable and type, then we just re-use it. If there is already a weaker constraint, then we generate both the weak constraints and the strong constraint so that the weak constraints won't be lost even if the strong one is unavailable.

We find all the distinct type constraints for each variable during the pre-pass over the lambda nesting. Each constraint has a list of the weaker constraints so that we can easily generate them.

Every block generates all the type constraints in it, but a constraint is available in a successor only if it is available in all predecessors. We determine the actual type constraint for a variable at a block by intersecting all the available type constraints for that variable.

This isn't maximally tense when there are constraints that are not hierarchically related, e.g. (or a b) (or b c). If these constraints were available from two predecessors, then we could infer that we have an (or a b c) constraint, but the above algorithm would come up with none. This probably isn't a big problem.

[\#\#\# Do we want to deal with \verb+(if (eq <var> '<foo>) ...)+ indicating singleton member type?]

We detect explicit type tests by looking at type test annotation in the IF node. If there is a type check, the OUT sets are stored in the node, with different sets for the consequent and alternative. Implicit type checks are located by finding Ref nodes whose Cont has the Type-Check flag set. We don't actually represent the GEN sets, we just initialize OUT to it, and then form the union in place.

When we do the post-pass, we clear the Type-Check flags in the continuations for Refs when we discover that the available constraints satisfy the asserted type. Any explicit uses of typep should be cleaned up by the ICR optimizer for typep. We can also set the derived type for Refs to the intersection of the available type assertions. If we discover anything, we should consider redoing ICR optimization, since better type information might enable more optimizations.

\chapterICR finalize % -*- Dictionary: design -*-

This pass looks for interesting things in the ICR so that we can forget about them. Used and not defined things are flamed about.

We postpone these checks until now because the ICR optimizations may discover errors that are not initially obvious. We also emit efficiency notes about optimizations that we were unable to do. We can't emit the notes immediately, since we don't know for sure whether a repeated attempt at optimization will succeed.

We examine all references to unknown global function variables and update the approximate type accordingly. We also record the names of the unknown functions so that they can be flamed about if they are never defined. Unknown normal variables are flamed about on the fly during ICR conversion, so we ignore them here.

We check each newly defined global function for compatibility with previously recorded type information. If there is no :defined or :declared type, then we check for compatibility with any approximate function type inferred from previous uses.

\chapterEnvironment analysis

A related change would be to annotate ICR with information about tail-recursion relations. What we would do is add a slot to the node structure that points to the corresponding Tail-Info when a node is in a TR position. This annotation would be made in a final ICR pass that runs after cleanup code is generated (part of environment analysis). When true, the node is in a true TR position (modulo return-convention incompatibility). When we determine return conventions, we null out the tail-p slots in XEP calls or known calls where we decided not to preserve tail-recursion.

In this phase, we also check for changes in the dynamic binding environment that require cleanup code to be generated. We just check for changes in the Continuation-Cleanup on local control transfers. If it changes from an inner dynamic context to an outer one that is in the same environment, then we emit code to clean up the dynamic bindings between the old and new continuation. We represent the result of cleanup detection to the back end by interposing a new block containing a call to a funny function. Local exits from CATCH or UNWIND-PROTECT are detected in the same way.

The primary activity in environment analysis is the annotation of ICR with environment structures describing where variables are allocated and what values the environment closes over.

Each lambda points to the environment where its variables are allocated, and the environments point back. We always allocate the environment at the Bind node for the sole non-let lambda in the environment, so there is a close relationship between environments and functions. Each "real function" (i.e. not a LET) has a corresponding environment.

We attempt to share the same environment among as many lambdas as possible so that unnecessary environment manipulation is not done. During environment analysis the only optimization of this sort is realizing that a Let (a lambda with no Return node) cannot need its own environment, since there is no way that it can return and discover that its old values have been clobbered.

When the function is called, values from other environments may need to be made available in the function's environment. These values are said to be "closed over".

Even if a value is not referenced in a given environment, it may need to be closed over in that environment so that it can be passed to a called function that does reference the value. When we discover that a value must be closed over by a function, we must close over the value in all the environments where that function is referenced. This applies to all references, not just local calls, since at other references we must have the values on hand so that we can build a closure. This propagation must be applied recursively, since the value must also be available in *those* functions' callers.

If a closure reference is known to be "safe" (not an upward funarg), then the closure structure may be allocated on the stack.

Closure analysis deals only with closures over values, while Common Lisp requires closures over variables. The difference only becomes significant when variables are set. If a variable is not set, then we can freely make copies of it without keeping track of where they are. When a variable is set, we must maintain a single value cell, or at least the illusion thereof. We achieve this by creating a heap-allocated "value cell" structure for each set variable that is closed over. The pointer to this value cell is passed around as the "value" corresponding to that variable. References to the variable must explicitly indirect through the value cell.

When we are scanning over the lambdas in the component, we also check for bound but not referenced variables.

Environment analysis emits cleanup code for local exits and markers for non-local exits.

A non-local exit is a control transfer from one environment to another. In a non-local exit, we must close over the continuation that we transfer to so that the exiting function can find its way back. We indicate the need to close a continuation by placing the continuation structure in the closure and also pushing it on a list in the environment structure for the target of the exit. [\#\#\# To be safe, we would treat the continuation as a set closure variable so that we could invalidate

it when we leave the dynamic extent of the exit point. Transferring control to a meaningless stack pointer would be apt to cause horrible death.]

Each local control transfer may require dynamic state such as special bindings to be undone. We represent cleanup actions by funny function calls in a new block linked in as an implicit MV-PROG1.

Compiler Middle Phase

3 Virtual Machine Representation Introduction

4 Global TN Assignment

% Rename this phase so as not to be confused with the local/global TN % representation.

The basic mechanism for closing over values is to pass the values as additional implicit arguments in the function call. This technique is only applicable when:

\beginitemize \item the calling function knows which values the called function wants to close over, and \item the values to be closed over are available in the calling environment. \enditemize

The first condition is always true of local function calls. Environment analysis can guarantee that the second condition holds by closing over any needed values in the calling environment.

If the function that closes over values may be called in an environment where the closed over values are not available, then we must store the values in a "closure" so that they are always accessible. Closures are called using the "full call" convention. When a closure is called, control is transferred to the "external entry point", which fetches the values out of the closure and then does a local call to the real function, passing the closure values as implicit arguments.

In this scheme there is no such thing as a "heap closure variable" in code, since the closure values are moved into TNs by the external entry point. There is some potential for pessimization here, since we may end up moving the values from the closure into a stack memory location, but the advantages are also substantial. Simplicity is gained by always representing closure values the same way, and functions with closure references may still be called locally without allocating a closure. All the TN based VMR optimizations will apply to closure variables, since closure variables are represented in the same way as all other variables in VMR. Closure values will be allocated in registers where appropriate.

Closures are created at the point where the function is referenced, eliminating the need to be able to close over closures. This lazy creation of closures has the additional advantage that when a closure reference is conditionally not done, then the closure consing will never be done at all. The corresponding disadvantage is that a closure over the same values may be created multiple times if there are multiple references. Note however, that VMR loop and common subexpression optimizations can eliminate redundant closure consing. In any case, multiple closures over the same variables doesn't seem to be that common.

\#| Having the Tail-Info would also make return convention determination trivial. We could just look at the type, checking to see if it represents a fixed number of values. To determine if the standard return convention is necessary to preserve tail-recursion, we just iterate over the equivalent functions, looking for XEPs and uses in full calls. |\#

The Global TN Assignment pass (GTN) can be considered a post-pass to environment analysis. This phase assigns the TNs used to hold local lexical variables and pass arguments and return values and determines the value-passing strategy used in local calls.

To assign return locations, we look at the function's tail-set.

If the result continuation for an entry point is used as the continuation for a full call, then we may need to constrain the continuation's values passing convention to the standard one. This is not necessary when the call is known not to be part of a tail-recursive loop (due to being a known function).

Once we have figured out where we must use the standard value passing strategy, we can use a more flexible strategy to determine the return locations for local functions. We determine the possible numbers of return values from each function by examining the uses of all the result continuations in the equivalence class of the result continuation.

If the tail-set type is for a fixed number of values, then we return that fixed number of values from all the functions whose result continuations are equated. If the number of values is not fixed, then we must use the unknown-values convention, although we are not forced to use the standard locations. We assign the result TNs at this time.

We also use the tail-sets to see what convention we want to use. What we do is use the full convention for any function that has a XEP its tail-set, even if we aren't required to do so by a tail-recursive full call, as long as there are no non-tail-recursive local calls in the set. This prevents us from gratuitously using a non-standard convention when there is no reason to.

\chapterLocal TN assignment

[Want a different name for this so as not to be confused with the different local/global TN representations. The really interesting stuff in this phase is operation selection, values representation selection, return strategy, etc. Maybe this phase should be conceptually lumped with GTN as "implementation selection", since GTN determines call strategies and locations.]

\#|

[\#\#\# I guess I believe that it is OK for VMR conversion to dick the ICR flow graph. An alternative would be to give VMR its very own flow graph, but that seems like overkill.

In particular, it would be very nice if a TR local call looked exactly like a jump in VMR. This would allow loop optimizations to be done on loops written as recursions. In addition to making the call block transfer to the head of the function rather than to the return, we would also have to do something about skipping the part of the function prolog that moves arguments from the passing locations, since in a TR call they are already in the right frame.

In addition to directly indicating whether a call should be coded with a TR variant, the Tail-P annotation flags non-call nodes that can directly return the value (an "advanced return"), rather than moving the value to the result continuation and jumping to the return code. Then (according to policy), we can decide to advance all possible returns. If all uses of the result are Tail-P, then LTN can annotate the result continuation as :Unused, inhibiting emission of the default return code.

[\#\#\# But not really. Now there is a single list of templates, and a given template has only one policy.]

In LTN, we use the :Safe template as a last resort even when the policy is unsafe. Note that we don't try :Fast-Safe; if this is also a good unsafe template, then it should have the unsafe policies explicitly specified.

With a :Fast-Safe template, the result type must be proven to satisfy the output type assertion. This means that a fast-safe template with a fixnum output type doesn't need to do fixnum overflow checking. [\#\#\# Not right to just check against the Node-Derived-Type, since type-check intersects with this.]

It seems that it would be useful to have a kind of template where the args must be checked to be fixnum, but the template checks for overflow and signals an error. In the case where an output assertion is present, this would generate better code than conditionally branching off to make a bignum, and then doing a type check on the result.

How do we deal with deciding whether to do a fixnum overflow check? This is perhaps a more general problem with the interpretation of result type restrictions in templates. It would be useful to be able to discriminate between the case where the result has been proven to be a fixnum and where it has simply been asserted to be so.

The semantics of result type restriction is that the result must be proven to be of that type *except* for safe generators, which are assumed to verify the assertion. That way "is-fixnum" case can be a fast-safe generator and the "should-be-fixnum" case is a safe generator. We could choose not to have a safe "should-be-fixnum" generator, and let the unrestricted safe generator handle it. We would then have to do an explicit type check on the result.

In other words, for all template except Safe, a type restriction on either an argument or result means "this must be true; if it is not the system may break." In contrast, in a Safe template, the restriction means "If this is not true, I will signal an error."

Since the node-derived-type only takes into consideration stuff that can be proved from the arguments, we can use the node-derived-type to select fast-safe templates. With unsafe policies, we don't care, since the code is supposed to be unsafe.

|\#

Local TN assignment (LTN) assigns all the TNs needed to represent the values of continuations. This pass scans over the code for the component, examining each continuation and its destination. A number of somewhat unrelated things are also done at the same time so that multiple passes aren't necessary. – Determine the Primitive-Type for each continuation value and assigns TNs to hold the values. – Use policy information to determine the implementation strategy for each call to a known function. – Clear the type-check flags in continuations whose destinations have safe implementations. – Determine the value-passing strategy for each continuation: known or unknown. – Note usage of unknown-values continuations so that stack analysis can tell when stack values must be discarded.

If safety is more important than speed and space, then we consider generating type checks on the values of nodes whose CONT has the Type-Check flag set. If the destination for the continuation value is safe, then we don't need to do a check. We

assume that all full calls are safe, and use the template information to determine whether inline operations are safe.

This phase is where compiler policy switches have most of their effect. The speed/space/safety tradeoff can determine which of a number of coding strategies are used. It is important to make the policy choice in VMR conversion rather than in code generation because the cost and storage requirement information which drives TNBIND will depend strongly on what actual VOP is chosen. In the case of +/FIXNUM, there might be three or more implementations, some optimized for speed, some for space, etc. Some of these VOPS might be open-coded and some not.

We represent the implementation strategy for a call by either marking it as a full call or annotating it with a "template" representing the open-coding strategy. Templates are selected using a two-way dispatch off of operand primitive-types and policy. The general case of LTN is handled by the LTN-Annotate function in the function-info, but most functions are handled by a table-driven mechanism. There are four different translation policies that a template may have: \begindescription \item[Safe] The safest implementation; must do argument type checking.

\item[Small] The (unsafe) smallest implementation.

\item[Fast] The (unsafe) fastest implementation.

\item[Fast-Safe] An implementation optimized for speed, but which does any necessary checks exclusive of argument type checking. Examples are array bounds checks and fixnum overflow checks. \enddescription

Usually a function will have only one or two distinct templates. Either or both of the safe and fast-safe templates may be omitted; if both are specified, then they should be distinct. If there is no safe template and our policy is safe, then we do a full call.

We use four different coding strategies, depending on the policy: \begindescription \item[Safe:] safety $>$ space $>$ speed, or we want to use the fast-safe template, but there isn't one.

\item[Small:] space $>$ (max speed safety)

\item[Fast:] speed $>$ (max space safety)

\item[Fast-Safe (and type check):] safety $>$ speed $>$ space, or we want to use the safe template, but there isn't one. \enddescription

"Space" above is actually the maximum of space and cspeed, under the theory that less code will take less time to generate and assemble. [\#\#\# This could lose if the smallest case is out-of-line, and must allocate many linkage registers.]

\chapterControl optimization

In this phase we annotate blocks with drop-throughs. This controls how code generation linearizes code so that drop-throughs are used most effectively. We totally linearize the code here, allowing code generation to scan the blocks in the emit order.

There are basically two aspects to this optimization:

\beginenumerate \item Dynamically reducing the number of branches taken v.s. branches not taken under the assumption that branches not taken are cheaper.

\item Statically minimizing the number of unconditional branches, saving space and presumably time. \endenumerate

These two goals can conflict, but if they do it seems pretty clear that the dynamic optimization should get preference. The main dynamic optimization is changing the sense of a conditional test so that the more commonly taken branch is the fall-through case. The problem is determining which branch is more commonly taken.

The most clear-cut case is where one branch leads out of a loop and the other is within. In this case, clearly the branch within the loop should be preferred. The only added complication is that at some point in the loop there has to be a backward branch, and it is preferable for this branch to be conditional, since an unconditional branch is just a waste of time.

In the absence of such good information, we can attempt to guess which branch is more popular on the basis of difference in the cost between the two cases. Min-max strategy suggests that we should choose the cheaper alternative, since the percentagewise improvement is greater when the branch overhead is significant with respect to the cost of the code branched to. A tractable approximation of this is to compare only the costs of the two blocks immediately branched to, since this would avoid having to do any hairy graph walking to find all the code for the consequent and the alternative. It might be worthwhile discriminating against ultra-expensive functions such as ERROR.

For this to work, we have to detect when one of the options is empty. In this case, the next for one branch is a successor of the other branch, making the comparison meaningless. We use dominator information to detect this situation. When a branch is empty, one of the predecessors of the first block in the empty branch will be dominated by the first block in the other branch. In such a case we favor the empty branch, since that's about as cheap as you can get.

Statically minimizing branches is really a much more tractable problem, but what literature there is makes it look hard. Clearly the thing to do is to use a non-optimal heuristic algorithm.

A good possibility is to use an algorithm based on the depth first ordering. We can modify the basic DFO algorithm so that it chooses an ordering which favors any drop-thrus that we may choose for dynamic reasons. When we are walking the graph, we walk the desired drop-thru arc last, which will place it immediately after us in the DFO unless the arc is a retreating arc.

We scan through the DFO and whenever we find a block that hasn't been done yet, we build a straight-line segment by setting the drop-thru to the unreached successor block which has the lowest DFN greater than that for the block. We move to the drop-thru block and repeat the process until there is no such block. We then go back to our original scan through the DFO, looking for the head of another straight-line segment.

This process will automagically implement all of the dynamic optimizations described above as long as we favor the appropriate IF branch when creating the DFO. Using the DFO will prevent us from making the back branch in a loop the drop-thru, but we need to be clever about favoring IF branches within loops while

computing the DFO. The IF join will be favored without any special effort, since we follow through the most favored path until we reach the end.

This needs some knowledge about the target machine, since on most machines non-tail-recursive calls will use some sort of call instruction. In this case, the call actually wants to drop through to the return point, rather than dropping through to the beginning of the called function.

\chapterVMR conversion

\#| Single-use let var continuation substitution not really correct, since it can cause a spurious type error. Maybe we do want stuff to prove that an NLX can't happen after all. Or go back to the idea of moving a combination arg to the ref location, and having that use the ref cont (with its output assertion.) This lossage doesn't seem very likely to actually happen, though. [\#\#\# must-reach stuff wouldn't work quite as well as combination substitute in psetq, etc., since it would fail when one of the new values is random code (might unwind.)]

Is this really a general problem with eager type checking? It seems you could argue that there was no type error in this code:

\beginverbatim (+ :foo (throw 'up nil)) \endverbatim

But we would signal an error.

Emit explicit you-lose operation when we do a move between two non-T ptypes, even when type checking isn't on. Can this really happen? Seems we should treat continuations like this as though type-check was true. Maybe LTN should leave type-check true in this case, even when the policy is unsafe. (Do a type check against NIL?)

At continuation use time, we may in general have to do both a coerce-to-t and a type check, allocating two temporary TNs to hold the intermediate results.

\sectionVMR Control representation

We represent all control transfer explicitly. In particular, :Conditional VOPs take a single Target continuation and a Not-P flag indicating whether the sense of the test is negated. Then an unconditional Branch VOP will be emitted afterward if the other path isn't a drop-through.

So we linearize the code before VMR-conversion. This isn't a problem, since there isn't much change in control flow after VMR conversion (none until loop optimization requires introduction of header blocks.) It does make cost-based branch prediction a bit ucky, though, since we don't have any cost information in ICR. Actually, I guess we do have pretty good cost information after LTN even before VMR conversion, since the most important thing to know is which functions are open-coded.

|\#

VMR preserves the block structure of ICR, but replaces the nodes with a target dependent virtual machine (VM) representation. Different implementations may use different VMs without making major changes in the back end. The two main components of VMR are Temporary Names (TNs) and Virtual OPerations (VOPs). TNs represent the locations that hold values, and VOPs represent the operations performed on the values.

A "primitive type" is a type meaningful at the VM level. Examples are Fixnum, String-Char, Short-Float. During VMR conversion we use the primitive type of an expression to determine both where we can store the result of the expression and which type-specific implementations of an operation can be applied to the value. [Ptype is a set of SCs == representation choices and representation specific operations]

The VM specific definitions provide functions that do stuff like find the primitive type corresponding to a type and test for primitive type subtypep. Usually primitive types will be disjoint except for T, which represents all types.

The primitive type T is special-cased. Not only does it overlap with all the other types, but it implies a descriptor ("boxed" or "pointer") representation. For efficiency reasons, we sometimes want to use alternate representations for some objects such as numbers. The majority of operations cannot exploit alternate representations, and would only be complicated if they had to be able to convert alternate representations into descriptors. A template can require an operand to be a descriptor by constraining the operand to be of type T.

A TN can only represent a single value, so we bare the implementation of MVs at this point. When we know the number of multiple values being handled, we use multiple TNs to hold them. When the number of values is actually unknown, we use a convention that is compatible with full function call.

Everything that is done is done by a VOP in VMR. Calls to simple primitive functions such as + and CAR are translated to VOP equivalents by a table-driven mechanism. This translation is specified by the particular VM definition; VMR conversion makes no assumptions about which operations are primitive or what operand types are worth special-casing. The default calling mechanisms and other miscellaneous builtin features are implemented using standard VOPs that must be implemented by each VM.

Type information can be forgotten after VMR conversion, since all type-specific operation selections have been made.

Simple type checking is explicitly done using CHECK-xxx VOPs. They act like innocuous effectless/unaffected VOPs which return the checked thing as a result. This allows loop-invariant optimization and common subexpression elimination to remove redundant checks. All type checking is done at the time the continuation is used.

Note that we need only check asserted types, since if type inference works, the derived types will also be satisfied. We can check whichever is more convenient, since both should be true.

Constants are turned into special Constant TNs, which are wired down in a SC that is determined by their type. The VM definition provides a function that returns a constant TN to represent a Constant Leaf.

Each component has a constant pool. There is a register dedicated to holding the constant pool for the current component. The back end allocates non-immediate constants in the constant pool when it discovers them during translation from ICR.

[\#\#\# Check that we are describing what is actually implemented. But this really isn't very good in the presence of interesting unboxed representations...]

Since LTN only deals with values from the viewpoint of the receiver, we must be prepared during the translation pass to do stuff to the continuation at the time it is used. – If a VOP yields more values than are desired, then we must create TNs to hold the discarded results. An important special-case is continuations whose value is discarded. These continuations won't be annotated at all. In the case of a Ref, we can simply skip evaluation of the reference when the continuation hasn't been annotated. Although this will eliminate bogus references that for some reason weren't optimized away, the real purpose is to handle deferred references. – If a VOP yields fewer values than desired, then we must default the extra values to NIL. – If a continuation has its type-check flag set, then we must check the type of the value before moving it into the result location. In general, this requires computing the result in a temporary, and having the type-check operation deliver it in the actual result location. – If the template's result type is T, then we must generate a boxed temporary to compute the result in when the continuation's type isn't T.

We may also need to do stuff to the arguments when we generate code for a template. If an argument continuation isn't annotated, then it must be a deferred reference. We use the leaf's TN instead. We may have to do any of the above use-time actions also. Alternatively, we could avoid hair by not deferring references that must be type-checked or may need to be boxed.

\sectionStack analysis

Think of this as a lifetime problem: a values generator is a write and a values receiver is a read. We want to annotate each VMR-Block with the unknown-values continuations that are live at that point. If we do a control transfer to a place where fewer continuations are live, then we must deallocate the newly dead continuations.

We want to convince ourselves that values deallocation based on lifetime analysis actually works. In particular, we need to be sure that it doesn't violate the required stack discipline. It is clear that it is impossible to deallocate the values before they become dead, since later code may decide to use them. So the only thing we need to ensure is that the "right" time isn't later than the time that the continuation becomes dead.

The only reason why we couldn't deallocate continuation A as soon as it becomes dead would be that there is another continuation B on top of it that isn't dead (since we can only deallocate the topmost continuation).

The key to understanding why this can't happen is that each continuation has only one read (receiver). If B is on top of A, then it must be the case that A is live at the receiver for B. This means that it is impossible for B to be live without A being live.

The reason that we don't solve this problem using a normal iterative flow analysis is that we also need to know the ordering of the continuations on the stack so that we can do deallocation. When it comes time to discard values, we want to know which discarded continuation is on the bottom so that we can reset SP to its start.

[I suppose we could also decrement SP by the aggregate size of the discarded continuations.] Another advantage of knowing the order in which we expect continuations to be on the stack is that it allows us to do some consistency checking. Also doing a localized graph walk around the values-receiver is likely to be much

more efficient than doing an iterative flow analysis problem over all the code in the component (not that big a consideration.)

\#| Actually, what we do is a backward graph walk from each unknown-values receiver. As we go, we mark each walked block with the ordered list of continuations we believe are on the stack. Starting with an empty stack, we: – When we encounter another unknown-values receiver, we push that continuation on our simulated stack. – When we encounter a receiver (which had better be for the topmost continuation), we pop that continuation. – When we pop all continuations, we terminate our walk.

[\#\#\# not quite right... It seems we may run into "dead values" during the graph walk too. It seems that we have to check if the pushed continuation is on stack top, and if not, add it to the ending stack so that the post-pass will discard it.]

[\#\#\# Also, we can't terminate our walk just because we hit a block previously walked. We have to compare the End-Stack with the values received along the current path: if we have more values on our current walk than on the walk that last touched the block, then we need to re-walk the subgraph reachable from that block, using our larger set of continuations. It seems that our actual termination condition is reaching a block whose End-Stack is already EQ to our current stack.]

If at the start, the block containing the values receiver has already been walked, we skip the walk for that continuation, since it has already been handled by an enclosing values receiver. Once a walk has started, we ignore any signs of a previous walk, clobbering the old result with our own, since we enclose that continuation, and the previous walk doesn't take into consideration the fact that our values block underlies its own.

When we are done, we have annotated each block with the stack current both at the beginning and at the end of that block. Blocks that aren't walked don't have anything on the stack either place (although they may hack MVs internally).

We then scan all the blocks in the component, looking for blocks that have predecessors with a different ending stack than that block's starting stack. (The starting stack had better be a tail of the predecessor's ending stack.) We insert a block intervening between all of these predecessors that sets SP to the end of the values for the continuation that should be on stack top. Of course, this pass needn't be done if there aren't any global unknown MVs.

Also, if we find any block that wasn't reached during the walk, but that USEs an outside unknown-values continuation, then we know that the DEST can't be reached from this point, so the values are unused. We either insert code to pop the values, or somehow mark the code to prevent the values from ever being pushed. (We could cause the popping to be done by the normal pass if we iterated over the pushes beforehand, assigning a correct END-STACK.)

[\#\#\# But I think that we have to be a bit clever within blocks, given the possibility of blocks being joined. We could collect some unknown MVs in a block, then do a control transfer out of the receiver, and this control transfer could be squeezed out by merging blocks. How about:

\beginverbatim (tagbody (return (multiple-value-prog1 (foo) (when bar (go UN-WIND))))

UNWIND (return (multiple-value-prog1 (baz) bletch))) \endverbatim

But the problem doesn't happen here (can't happen in general?) since a node buried within a block can't use a continuation outside of the block. In fact, no block can have more then one PUSH continuation, and this must always be the last continuation. So it is trivially (structurally) true that all pops come before any push.

[\#\#\# But not really: the DEST of an embedded continuation may be outside the block. There can be multiple pushes, and we must find them by iterating over the uses of MV receivers in LTN. But it would be hard to get the order right this way. We could easily get the order right if we added the generators as we saw the uses, except that we can't guarantee that the continuations will be annotated at that point. (Actually, I think we only need the order for consistency checks, but that is probably worthwhile). I guess the thing to do is when we process the receiver, add the generator blocks to the Values-Generators, then do a post-pass that re-scans the blocks adding the pushes.]

I believe that above concern with a dead use getting mashed inside a block can't happen, since the use inside the block must be the only use, and if the use isn't reachable from the push, then the use is totally unreachable, and should have been deleted, which would prevent it from ever being annotated.]] |\#

We find the partial ordering of the values globs for unknown values continuations in each environment. We don't have to scan the code looking for unknown values continuations since LTN annotates each block with the continuations that were popped and not pushed or pushed and not popped. This is all we need to do the inter-block analysis.

After we have found out what stuff is on the stack at each block boundary, we look for blocks with predecessors that have junk on the stack. For each such block, we introduce a new block containing code to restore the stack pointer. Since unknown-values continuations are represented as \verb+<start, count>+, we can easily pop a continuation using the Start TN.

Note that there is only doubt about how much stuff is on the control stack, since only it is used for unknown values. Any special stacks such as number stacks will always have a fixed allocation.

\sectionNon-local exit

If the starting and ending continuations are not in the same environment, then the control transfer is a non-local exit. In this case just call Unwind with the appropriate stack pointer, and let the code at the re-entry point worry about fixing things up.

It seems like maybe a good way to organize VMR conversion of NLX would be to have environment analysis insert funny functions in new interposed cleanup blocks. The thing is that we need some way for VMR conversion to: 1] Get its hands on the returned values. 2] Do weird control shit. 3] Deliver the values to the original continuation destination. I.e. we need some way to interpose arbitrary code in the path of value delivery.

What we do is replace the NLX uses of the continuation with another continuation that is received by a MV-Call to \%NLX-VALUES in a cleanup block that is

interposed between the NLX uses and the old continuation's block. The MV-Call uses the original continuation to deliver its values to.

[Actually, it's not really important that this be an MV-Call, since it has to be special-cased by LTN anyway. Or maybe we would want it to be an MV call. If we did normal LTN analysis of an MV call, it would force the returned values into the unknown values convention, which is probably pretty convenient for use in NLX.

Then the entry code would have to use some special VOPs to receive the unknown values. But we probably need special VOPs for NLX entry anyway, and the code can share with the call VOPs. Also we probably need the technology anyway, since THROW will use truly unknown values.]

On entry to a dynamic extent that has non-local-exists into it (always at an ENTRY node), we take a complete snapshot of the dynamic state:

\beginitemize \item the top pointers for all stacks \item current Catch and Unwind-Protect \item current special binding (binding stack pointer in shallow binding) \enditemize

We insert code at the re-entry point which restores the saved dynamic state. All TNs live at an NLX EP are forced onto the stack, so we don't have to restore them, and we don't have to worry about getting them saved.

Compiler Back End

5 Copy propagation

File: `copyprop.lisp`

This phase is optional, but should be done whenever speed or space is more important than compile speed. We use global flow analysis to find the reaching definitions for each TN. This information is used here to eliminate unnecessary TNs, and is also used later on by loop invariant optimization.

In some cases, VMR conversion will unnecessarily copy the value of a TN into another TN, since it may not be able to tell that the initial TN has the same value at the time the second TN is referenced. This can happen when ICR optimize is unable to eliminate a trivial variable binding, or when the user does a `setq`, or may also result from creation of expression evaluation temporaries during VMR conversion. Whatever the cause, we would like to avoid the unnecessary creation and assignment of these TNs.

What we do is replace TN references whose only reaching definition is a `Move VOP` with a reference to the TN moved from, and then delete the `Move VOP` if the copy TN has no remaining references. There are several restrictions on copy propagation:

bullet The TNs must be "ordinary" TNs, not restricted or otherwise unusual. Extending the life of restricted (or wired) TNs can make register allocation impossible. Some other TN kinds have hidden references.

bullet We don't want to defeat source-level debugging by replacing named variables with anonymous temporaries.

bullet We can't delete moves that representation selected might want to change into a representation conversion, since we need the primitive types of both TNs to select a conversion.

Some cleverness reduces the cost of flow analysis. As for lifetime analysis, we only need to do flow analysis on global packed TNs. We can't do the real local TN assignment pass before this, since we allocate TNs afterward, so we do a pre-pass that marks the TNs that are local for our purposes. We don't care if block splitting eventually causes some of them to be considered global.

Note also that we are really only interested in knowing if there is a unique reaching definition, which we can mash into our flow analysis rules by doing an intersection. Then a definition only appears in the set when it is unique. We then propagate only definitions of TNs with only one write, which allows the TN to stand for the definition.

6 Representation Selection

File: `represent.lisp`

Some types of object (such as `single-float`) have multiple possible representations. Multiple representations are useful mainly when there is a particularly efficient non-descriptor representation. In this case, there is the normal descriptor representation, and an alternate non-descriptor representation.

This possibility brings up two major issues:

- The compiler must decide which representation will be most efficient for any given value, and
- Representation conversion code must be inserted where the representation of a value is changed.

First, the representations for TNs are selected by examining all the TN references and attempting to minimize reference costs. Then representation conversion code is introduced.

This phase is in effect a pre-pass to register allocation. The main reason for its existence is that representation conversions may be farily complex (e.g. involving memory allocation), and thus must be discovered before register allocation.

VMR conversion leaves stubs for representation specific move operations. Representation selection recognizes `move` by name. Argument and return value passing for call VOPs is controlled by the `:move-arguments` option to `define-vop`.

Representation selection is also responsible for determining what functions use the number stack. If any representation is chosen which could involve packing into the \tt non-descriptor-stack SB, then we allocate the NFP register throughout the component. As an optimization, permit the decision of whether a number stack frame needs to be allocated to be made on a per-function basis. If a function doesn't use the number stack, and isn't in the same tail-set as any function that uses the number stack, then it doesn't need a number stack frame, even if other functions in the component do.

\chapterLifetime analysis

File: \tt life

This phase is a preliminary to Pack. It involves three passes: – A pre-pass that computes the DEF and USE sets for live TN analysis, while also assigning local TN numbers, splitting blocks if necessary. \#\#\# But not really... – A flow analysis pass that does backward flow analysis on the component to find the live TNs at each block boundary. – A post-pass that finds the conflict set for each TN.

\#| Exploit the fact that a single VOP can only exhaust LTN numbers when there are large more operands. Since more operand reference cannot be interleaved with temporary reference, the references all effectively occur at the same time. This means that we can assign all the more args and all the more results the same LTN number and the same lifetime info. |\#

\sectionFlow analysis

It seems we could use the global-conflicts structures during compute the inter-block lifetime information. The pre-pass creates all the global-conflicts for blocks

that global TNs are referenced in. The flow analysis pass just adds always-live global-conflicts for the other blocks the TNs are live in. In addition to possibly being more efficient than SSets, this would directly result in the desired global-conflicts information, rather than having to create it from another representation.

The DFO sorted per-TN global-conflicts thread suggests some kind of algorithm based on the manipulation of the sets of blocks each TN is live in (which is what we really want), rather than the set of TNs live in each block.

If we sorted the per-TN global-conflicts in reverse DFO (which is just as good for determining conflicts between TNs), then it seems we could scan though the conflicts simultaneously with our flow-analysis scan through the blocks.

The flow analysis step is the following: If a TN is always-live or read-before-written in a successor block, then we make it always-live in the current block unless there are already global-conflicts recorded for that TN in this block.

The iteration terminates when we don't add any new global-conflicts during a pass.

We may also want to promote TNs only read within a block to always-live when the TN is live in a successor. This should be easy enough as long as the global-conflicts structure contains this kind of info.

The critical operation here is determining whether a given global TN has global conflicts in a given block. Note that since we scan the blocks in DFO, and the global-conflicts are sorted in DFO, if we give each global TN a pointer to the global-conflicts for the last block we checked the TN was in, then we can guarantee that the global-conflicts we are looking for are always at or after that pointer. If we need to insert a new structure, then the pointer will help us rapidly find the place to do the insertion.]

\sectionConflict detection

[\#\#\# Environment, :more TNs.]

This phase makes use of the results of lifetime analysis to find the set of TNs that have lifetimes overlapping with those of each TN. We also annotate call VOPs with information about the live TNs so that code generation knows which registers need to be saved.

The basic action is a backward scan of each block, looking at each TN-Ref and maintaining a set of the currently live TNs. When we see a read, we check if the TN is in the live set. If not, we: – Add the TN to the conflict set for every currently live TN, – Union the set of currently live TNs with the conflict set for the TN, and – Add the TN to the set of live TNs.

When we see a write for a live TN, we just remove it from the live set. If we see a write to a dead TN, then we update the conflicts sets as for a read, but don't add the TN to the live set. We have to do this so that the bogus write doesn't clobber anything.

[We don't consider always-live TNs at all in this process, since the conflict of always-live TNs with other TNs in the block is implicit in the global-conflicts structures.

Before we do the scan on a block, we go through the global-conflicts structures of TNs that change liveness in the block, assigning the recorded LTN number to the TN's LTN number for the duration of processing of that block.]

Efficiently computing and representing this information calls for some cleverness. It would be prohibitively expensive to represent the full conflict set for every TN with sparse sets, as is done at the block-level. Although it wouldn't cause non-linear behavior, it would require a complex linked structure containing tens of elements to be created for every TN. Fortunately we can improve on this if we take into account the fact that most TNs are "local" TNs: TNs which have all their uses in one block.

First, many global TNs will be either live or dead for the entire duration of a given block. We can represent the conflict between global TNs live throughout the block and TNs local to the block by storing the set of always-live global TNs in the block. This reduces the number of global TNs that must be represented in the conflicts for local TNs.

Second, we can represent conflicts within a block using bit-vectors. Each TN that changes liveness within a block is assigned a local TN number. Local conflicts are represented using a fixed-size bit-vector of 64 elements or so which has a 1 for the local TN number of every TN live at that time. The block has a simple-vector which maps from local TN numbers to TNs. Fixed-size vectors reduce the hassle of doing allocations and allow operations to be open-coded in a maximally tense fashion.

We can represent the conflicts for a local TN by a single bit-vector indexed by the local TN numbers for that block, but in the global TN case, we need to be able to represent conflicts with arbitrary TNs. We could use a list-like sparse set representation, but then we would have to either special-case global TNs by using the sparse representation within the block, or convert the local conflicts bit-vector to the sparse representation at the block end. Instead, we give each global TN a list of the local conflicts bit-vectors for each block that the TN is live in. If the TN is always-live in a block, then we record that fact instead. This gives us a major reduction in the amount of work we have to do in lifetime analysis at the cost of some increase in the time to iterate over the set during Pack.

Since we build the lists of local conflict vectors a block at a time, the blocks in the lists for each TN will be sorted by the block number. The structure also contains the local TN number for the TN in that block. These features allow pack to efficiently determine whether two arbitrary TNs conflict. You just scan the lists in order, skipping blocks that are in only one list by using the block numbers. When we find a block that both TNs are live in, we just check the local TN number of one TN in the local conflicts vector of the other.

In order to do these optimizations, we must do a pre-pass that finds the always-live TNs and breaks blocks up into small enough pieces so that we don't run out of local TN numbers. If we can make a block arbitrarily small, then we can guarantee that an arbitrarily small number of TNs change liveness within the block. We must be prepared to make the arguments to unbounded arg count VOPs (such as function call) always-live even when they really aren't. This is enabled by a panic mode in the block splitter: if we discover that the block only contains one VOP and there are

still too many TNs that aren't always-live, then we promote the arguments (which we'd better be able to do...).

This is done during the pre-scan in lifetime analysis. We can do this because all TNs that change liveness within a block can be found by examining that block: the flow analysis only adds always-live TNs.

When we are doing the conflict detection pass, we set the LTN number of global TNs. We can easily detect global TNs that have not been locally mapped because this slot is initially null for global TNs and we null it out after processing each block. We assign all Always-Live TNs to the same local number so that we don't need to treat references to them specially when making the scan.

We also annotate call VOPs that do register saving with the TNs that are live during the call, and thus would need to be saved if they are packed in registers.

We adjust the costs for TNs that need to be saved so that TNs costing more to save and restore than to reference get packed on the stack. We would also like more often saved TNs to get higher costs so that they are packed in more savable locations.

\chapterPacking

File: \tt pack

\#|

Add lifetime/pack support for pre-packed save TNs.

Fix GTN/VMR conversion to use pre-packed save TNs for old-cont and return-PC. (Will prevent preference from passing location to save location from ever being honored?)

We will need to make packing of passing locations smarter before we will be able to target the passing location on the stack in a tail call (when that is where the callee wants it.) Currently, we will almost always pack the passing location in a register without considering whether that is really a good idea. Maybe we should consider schemes that explicitly understand the parallel assignment semantics, and try to do the assignment with a minimum number of temporaries. We only need assignment temps for TNs that appear both as an actual argument value and as a formal parameter of the called function. This only happens in self-recursive functions.

Could be a problem with lifetime analysis, though. The write by a move-arg VOP would look like a write in the current env, when it really isn't. If this is a problem, then we might want to make the result TN be an info arg rather than a real operand. But this would only be a problem in recursive calls, anyway. [This would prevent targeting, but targeting across passing locations rarely seems to work anyway.] [\#\#\# But the :ENVIRONMENT TN mechanism would get confused. Maybe put env explicitly in TN, and have it only always-live in that env, and normal in other envs (or blocks it is written in.) This would allow targeting into environment TNs.

I guess we would also want the env/PC save TNs normal in the return block so that we can target them. We could do this by considering env TNs normal in read blocks with no successors.

ENV TNs would be treated totally normally in non-env blocks, so we don't have to worry about lifetime analysis getting confused by variable initializations.

Do some kind of TN costing to determine when it is more trouble than it is worth to allocate TNs in registers.

Change pack ordering to be less pessimal. Pack TNs as they are seen in the LTN map in DFO, which at least in non-block compilations has an effect something like packing main trace TNs first, since control analysis tries to put the good code first. This could also reduce spilling, since it makes it less likely we will clog all registers with global TNs.

If we pack a TN with a specified save location on the stack, pack in the specified location.

Allow old-cont and return-pc to be kept in registers by adding a new "keep around" kind of TN. These are kind of like environment live, but are only always-live in blocks that they weren't referenced in. Lifetime analysis does a post-pass adding always-live conflicts for each "keep around" TN to those blocks with no conflict for that TN. The distinction between always-live and keep-around allows us to successfully target old-cont and return-pc to passing locations. MAKE-KEEP-AROUND-TN (ptype), PRE-PACK-SAVE-TN (tn scn offset). Environment needs a KEEP-AROUND-TNS slot so that conflict analysis can find them (no special casing is needed after then, they can be made with :NORMAL kind). VMR-component needs PRE-PACKED-SAVE-TNS so that conflict analysis or somebody can copy conflict info from the saved TN.

Note that having block granularity in the conflict information doesn't mean that a localized packing scheme would have to do all moves at block boundaries (which would clash with the desire to have saving done as part of this mechanism.) All that it means is that if we want to do a move within the block, we would need to allocate both locations throughout that block (or something).

Load TN pack:

A location is out for load TN packing if:

The location has TN live in it after the VOP for a result, or before the VOP for an argument, or

The location is used earlier in the TN-ref list (after) the saved results ref or later in the TN-Ref list (before) the loaded argument's ref.

To pack load TNs, we advance the live-tns to the interesting VOP, then repeatedly scan the vop-refs to find vop-local conflicts for each needed load TN. We insert move VOPs and change over the TN-Ref-TNs as we go so the TN-Refs will reflect conflicts with already packed load-TNs.

If we fail to pack a load-TN in the desired SC, then we scan the Live-TNs for the SB, looking for a TN that can be packed in an unbounded SB. This TN must then be repacked in the unbounded SB. It is important the load-TNs are never packed in unbounded SBs, since that would invalidate the conflicts info, preventing us from repacking TNs in unbounded SBs. We can't repack in a finite SB, since there might have been load TNs packed in that SB which aren't represented in the original conflict structures.

Is it permissible to "restrict" an operand to an unbounded SC? Not impossible to satisfy as long as a finite SC is also allowed. But in practice, no restriction would probably be as good.

We assume all locations can be used when an sc is based on an unbounded sb.
]

TN-Refs are convenient structures to build the target graph out of. If we allocated space in every TN-Ref, then there would certainly be enough to represent arbitrary target graphs. Would it be enough to allocate a single Target slot? If there is a target path through a given VOP, then the Target of the write ref would be the read, and vice-versa. To find all the TNs that target us, we look at the TN for the target of all our write refs.

We separately chain together the read refs and the write refs for a TN, allowing easy determination of things such as whether a TN has only a single definition or has no reads. It would also allow easier traversal of the target graph.

Represent per-location conflicts as vectors indexed by block number of per-block conflict info. To test whether a TN conflicts on a location, we would then have to iterate over the TNs global-conflicts, using the block number and LTN number to check for a conflict in that block. But since most TNs are local, this test actually isn't much more expensive than indexing into a bit-vector by GTN numbers.

The big win of this scheme is that it is much cheaper to add conflicts into the conflict set for a location, since we never need to actually compute the conflict set in a list-like representation (which requires iterating over the LTN conflicts vectors and unioning in the always-live TNs). Instead, we just iterate over the global-conflicts for the TN, using BIT-IOR to combine the conflict set with the bit-vector for that block in that location, or marking that block/location combination as being always-live if the conflict is always-live.

Generating the conflict set is inherently more costly, since although we believe the conflict set size to be roughly constant, it can easily contain tens of elements. We would have to generate these moderately large lists for all TNs, including local TNs. In contrast, the proposed scheme does work proportional to the number of blocks the TN is live in, which is small on average (1 for local TNs). This win exists independently from the win of not having to iterate over LTN conflict vectors.

[\#\#\# Note that since we never do bitwise iteration over the LTN conflict vectors, part of the motivation for keeping these a small fixed size has been removed. But it would still be useful to keep the size fixed so that we can easily recycle the bit-vectors, and so that we could potentially have maximally tense special primitives for doing clear and bit-ior on these vectors.]

This scheme is somewhat more space-intensive than having a per-location bit-vector. Each vector entry would be something like 150 bits rather than one bit, but this is mitigated by the number of blocks being 5-10x smaller than the number of TNs. This seems like an acceptable overhead, a small fraction of the total VMR representation.

The space overhead could also be reduced by using something equivalent to a two-dimensional bit array, indexed first by LTN numbers, and then block numbers (instead of using a simple-vector of separate bit-vectors.) This would eliminate space wastage due to bit-vector overheads, which might be 50% or more, and would also make efficient zeroing of the vectors more straightforward. We would then want efficient operations for OR'ing LTN conflict vectors with rows in the array.

This representation also opens a whole new range of allocation algorithms: ones that store allocate TNs in different locations within different portions of the program. This is because we can now represent a location being used to hold a certain TN within an arbitrary subset of the blocks the TN is referenced in.

Pack goals:

Pack should:

Subject to resource constraints: – Minimize use costs – "Register allocation" Allocate as many values as possible in scarce "good" locations, attempting to minimize the aggregate use cost for the entire program. – "Save optimization" Don't allocate values in registers when the save/restore costs exceed the expected gain for keeping the value in a register. (Similar to "opening costs" in RAOC.) [Really just a case of representation selection.]

– Minimize preference costs Eliminate as many moves as possible.

"Register allocation" is basically an attempt to eliminate moves between registers and memory. "Save optimization" counterbalances "register allocation" to prevent it from becoming a pessimization, since saves can introduce register/memory moves.

Preference optimization reduces the number of moves within an SC. Doing a good job of honoring preferences is important to the success of the compiler, since we have assumed in many places that moves will usually be optimized away.

The scarcity-oriented aspect of "register allocation" is handled by a greedy algorithm in pack. We try to pack the "most important" TNs first, under the theory that earlier packing is more likely to succeed due to fewer constraints.

The drawback of greedy algorithms is their inability to look ahead. Packing a TN may mess up later "register allocation" by precluding packing of TNs that are individually "less important," but more important in aggregate. Packing a TN may also prevent preferences from being honored.

Initial packing:

Pack all TNs restricted to a finite SC first, before packing any other TNs.

One might suppose that Pack would have to treat TNs in different environments differently, but this is not the case. Pack simply assigns TNs to locations so that no two conflicting TNs are in the same location. In the process of implementing call semantics in conflict analysis, we cause TNs in different environments not to conflict. In the case of passing TNs, cross environment conflicts do exist, but this reflects reality, since the passing TNs are live in both the caller and the callee. Environment semantics has already been implemented at this point.

This means that Pack can pack all TNs simultaneously, using one data structure to represent the conflicts for each location. So we have only one conflict set per SB location, rather than separating this information by environment.

Load TN packing:

We create load TNs as needed in a post-pass to the initial packing. After TNs are packed, it may be that some references to a TN will require it to be in a SC other than the one it was packed in. We create load-TNs and pack them on the fly during this post-pass.

What we do is have an optional SC restriction associated with TN-refs. If we pack the TN in an SC which is different from the required SC for the reference, then we create a TN for each such reference, and pack it into the required SC.

In many cases we will be able to pack the load TN with no hassle, but in general we may need to spill a TN that has already been packed. We choose a TN that isn't in use by the offending VOP, and then spill that TN onto the stack for the duration of that VOP. If the VOP is a conditional, then we must insert a new block interposed before the branch target so that the TN value is restored regardless of which branch is taken.

Instead of remembering lifetime information from conflict analysis, we rederive it. We scan each block backward while keeping track of which locations have live TNs in them. When we find a reference that needs a load TN packed, we try to pack it in an unused location. If we can't, we unpack the currently live TN with the lowest cost and force it into an unbounded SC.

The per-location and per-TN conflict information used by pack doesn't need to be updated when we pack a load TN, since we are done using those data structures.

We also don't need to create any TN-Refs for load TNs. [??? How do we keep track of load-tn lifetimes? It isn't really that hard, I guess. We just remember which load TNs we created at each VOP, killing them when we pass the loading (or saving) step. This suggests we could flush the Refs thread if we were willing to sacrifice some flexibility in explicit temporary lifetimes. Flushing the Refs would make creating the VMR representation easier.]

The lifetime analysis done during load-TN packing doubles as a consistency check. If we see a read of a TN packed in a location which has a different TN currently live, then there is a packing bug. If any of the TNs recorded as being live at the block beginning are packed in a scarce SB, but aren't current in that location, then we also have a problem.

The conflict structure for load TNs is fairly simple, the load TNs for arguments and results all conflict with each other, and don't conflict with much else. We just try packing in targeted locations before trying at random.

\chapterCode generation

This is fairly straightforward. We translate VOPs into instruction sequences on a per-block basis.

After code generation, the VMR representation is gone. Everything is represented by the assembler data structures.

\chapterAssembly

In effect, we do much of the work of assembly when the compiler is compiled.

The assembler makes one pass fixing up branch offsets, then squeezes out the space left by branch shortening and dumps out the code along with the load-time fixup information. The assembler also deals with dumping unboxed non-immediate constants and symbols. Boxed constants are created by explicit constructor code in the top-level form, while immediate constants are generated using inline code.

[\#\#\# The basic output of the assembler is: A code vector A representation of the fixups along with indices into the code vector for the fixup locations A PC map translating PCs into source paths

This information can then be used to build an output file or an in-core function object.]

The assembler is table-driven and supports arbitrary instruction formats. As far as the assembler is concerned, an instruction is a bit sequence that is broken down into subsequences. Some of the subsequences are constant in value, while others can be determined at assemble or load time.

\beginverbatim Assemble Node Form* Allow instructions to be emitted during the evaluation of the Forms by defining Inst as a local macro. This macro caches various global information in local variables. Node tells the assembler what node ultimately caused this code to be generated. This is used to create the pc=>source map for the debugger.

Assemble-Elsewhere Node Form* Similar to Assemble, but the current assembler location is changed to somewhere else. This is useful for generating error code and similar things. Assemble-Elsewhere may not be nested.

Inst Name Arg* Emit the instruction Name with the specified arguments.

Gen-Label Emit-Label (Label) Gen-Label returns a Label object, which describes a place in the code. Emit-Label marks the current position as being the location of Label. \endverbatim

\chapterDumping

So far as input to the dumper/loader, how about having a list of Entry-Info structures in the VMR-Component? These structures contain all information needed to dump the associated function objects, and are only implicitly associated with the functional/XEP data structures. Load-time constants that reference these function objects should specify the Entry-Info, rather than the functional (or something). We would then need to maintain some sort of association so VMR conversion can find the appropriate Entry-Info. Alternatively, we could initially reference the functional, and then later clobber the reference to the Entry-Info.

We have some kind of post-pass that runs after assembly, going through the functions and constants, annotating the VMR-Component for the benefit of the dumper: Resolve :Label load-time constants. Make the debug info. Make the entry-info structures.

Fasl dumper and in-core loader are implementation (but not instruction set) dependent, so we want to give them a clear interface.

\beginverbatim open-fasl-file name => fasl-file Returns a "fasl-file" object representing all state needed by the dumper. We objectify the state, since the fasdumper should be reentrant. (but could fail to be at first.)

close-fasl-file fasl-file abort-p Close the specified fasl-file.

fasl-dump-component component code-vector length fixups fasl-file Dump the code, constants, etc. for component. Code-Vector is a vector holding the assembled code. Length is the number of elements of Vector that are actually in use. Fixups is a list of conses (offset . fixup) describing the locations and things that need to be fixed up at load time. If the component is a top-level component, then the top-level lambda will be called after the component is loaded.

load-component component code-vector length fixups Like Fasl-Dump-Component, but directly installs the code in core, running any top-level code

immediately. (???) but we need some way to glue together the componenents, since we don't have a fasl table. \endverbatim

Dumping:

Dump code for each component after compiling that component, but defer dumping of other stuff. We do the fixups on the code vectors, and accumulate them in the table.

We have to grovel the constants for each component after compiling that component so that we can fix up load-time constants. Load-time constants are values needed by the code that are computed after code generation/assembly time. Since the code is fixed at this point, load-time constants are always represented as non-immediate constants in the constant pool. A load-time constant is distinguished by being a cons (Kind . What), instead of a Constant leaf. Kind is a keyword indicating how the constant is computed, and What is some context.

Some interesting load-time constants: \beginverbatim (:label . <label>) Is replaced with the byte offset of the label within the code-vector.

(:code-vector . <component>) Is replaced by the component's code-vector.

(:entry . <function>) (:closure-entry . <function>) Is replaced by the function-entry structure for the specified function. :Entry is how the top-level component gets a handle on the function definitions so that it can set them up. \endverbatim We also need to remember the starting offset for each entry, although these don't in general appear as explicit constants.

We then dump out all the :Entry and :Closure-Entry objects, leaving any constant-pool pointers uninitialized. After dumping each :Entry, we dump some stuff to let genesis know that this is a function definition. Then we dump all the constant pools, fixing up any constant-pool pointers in the already-dumped function entry structures.

The debug-info *is* a constant: the first constant in every constant pool. But the creation of this constant must be deferred until after the component is compiled, so we leave a (:debug-info) placeholder. [Or maybe this is implicitly added in by the dumper, being supplied in a VMR-component slot.]

Work out details of the interface between the back-end and the assembler/dumper.

Support for multiple assemblers concurrently loaded? (for byte code)

We need various mechanisms for getting information out of the assembler.

We can get entry PCs and similar things into function objects by making a Constant leaf, specifying that it goes in the closure, and then setting the value after assembly.

We have an operation Label-Value which can be used to get the value of a label after assembly and before the assembler data structures are deallocated.

The function map can be constructed without any special help from the assembler. Codegen just has to note the current label when the function changes from one block to the next, and then use the final value of these labels to make the function map.

Probably we want to do the source map this way too. Although this will make zillions of spurious labels, we would have to effectively do that anyway.

With both the function map and the source map, getting the locations right for uses of Elsewhere will be a bit tricky. Users of Elsewhere will need to know about how these maps are being built, since they must record the labels and corresponding information for the elsewhere range. It would be nice to have some cooperation from Elsewhere so that this isn't necessary, otherwise some VOP writer will break the rules, resulting in code that is nowhere.

The Debug-Info and related structures are dumped by consing up the structure and making it be the value of a constant.

Getting the code vector and fixups dumped may be a bit more interesting. I guess we want a Dump-Code-Vector function which dumps the code and fixups accumulated by the current assembly, returning a magic object that will become the code vector when it is dumped as a constant.]

The SBCL Lisp Virtual Machine

7 Introduction to the VM

% (defun gvp (f) % (with-open-file (s f :direction :output :if-exists :supersede) % (maphash \#'(lambda (k v) % (declare (ignore v)) % (format s "~A~%" k)) % (c::backend-template-names c::*backend*))))

\sectionScope and Purpose

This document describes the Virtual Machine that serves as the basis for the portable implementation of \ccl. The *Virtual Machine* (hereafter referred to as the VM) provides a layer of abstraction that hides low-level details of hardware and implementation strategy, while still revealing enough of the implementation so that most of the system can be written at the VM level or above.

Misc comments

\#\#\# Shouldn't specify VOPs. Instead, should specify which `sbcl` functions are primitive and which subprimitives exist. Isn't really anyone's business which VOPs actually exist. Each primitive function or subprimitive is implemented either as a VOP or as expansion into Lisp code, at the particular implementation's discretion.

From this point of view, the document is expressing the contract that the Lisp level code outside of the compiler must satisfy. All functions must ultimately be defined in terms of primitive functions and sub-primitives.

The responsibility of the compiler is to implement these primitive operations, and also to implement special forms, variables and function calling.

VOPs emitted by the hard-wired translators for non-function nodes are a somewhat different story. Each implementation will presumably implement all these VOPs in order to avoid having to rewrite IR2 translation. We also need to spend quite a bit of time discussing the semantics of these operations, since they don't just correspond to some `sbcl` function with type constraints.

Hard-wired stuff:

function call variable access: global function constant closure local closure creation non-local exit special binding/unbinding TN hacking: move VOPs TN address (???) Conditionals: Basic conditionals: EQ, ... Interface to generation of other conditional VOPs.

Some VOPs don't need to be implemented at all: VOPs to delimit the lifetimes of big stack TNs such as catch blocks Others? Move VOPs might be defined in terms of an implementation supplied move routine, since we probably also need this info outside of VOP generators so that implicit moves can be generated.

Type testing/checking (somehow)

What this document talks about:

Interface between compiler front-end and back end. (VOPs) Primitive \clisp operations directly supported by the VM. Support for complex language features such as function call.

Sub-primitives that allow system code to do things not possible in `sbcl`.

Descriptions of how the current SBCL system uses VM facilities, especially non-standard ones accessed through sub-primitives.

Notes about known portability problems.

Guidelines for writing portable SBCL system code. To some degree these guidelines are implied by statements that certain things are true of SBCL system code.

Descriptions of data structures that are not directly used by the VM, such as debug information and Core files.

Descriptions of data structures that are directly used by the VM, such as symbols and arrays.

Who should read it:

- People who want to port SBCL.
- People who want to understand the compiler.
- People who want to understand how SBCL works.
- People who need to write portable SBCL system code.
- People such as debugger writers who need to access SBCL \t()'s internal data structures.

What it won't do:

- Tell you things that are obviously implementation dependent, such as type systems or memory management disciplines. See the the various implementation VM documents.
- Tell you only what you need to know. Programmers shouldn't exploit properties of the VM documented here unless there is no way to do the same thing in portable SBCL.
- Tell you how the compiler works. In order to understand some of the subtleties of VOP descriptions, you will have to understand the IR2 representation and how it fits into the rest of the compiler.
- Tell you anything about SBCL semantics. When some part of the VM has a direct relationship to SBCL semantics, the relationship will be directly stated using SBCL terminology, since a restatement of the semantics is likely to be inaccurate or misleading. Exceptions will be made only when some implication of the SBCL semantics is non-obvious.
- Tell you everything about how `sbcl` works. This document only offers information that is likely to be needed by programmers doing a port or writing system code; portable, self-contained parts of the system are totally ignored. This document deliberately avoids replicating information that is easily available in the system sources, since such replicated information is always incorrect somewhere. In some cases, a forwarding pointer to the appropriate source will be given.

Things the VM won't do:

The VM specification does not totally solve the problem of porting SBCL, since it is inevitable that it will not map cleanly to all possible combinations of hardware and operating systems. The VM should not be regarded as being cast in concrete, since changes in many characteristics would only affect a tiny fraction of the system sources.

One current major problem with porting is that large pieces of functionality are entirely within the VM, and would need to be reimplemented for each port. A

major goal for future work on the system is moving code out of the VM, both by supporting a "fast-call" convention that allows reasonable use of Lisp in the out of line implementation of VOPs, and by having a "bugout" mechanism that allows the VM to call Lisp functions to implement the hard cases in some VOPs.

The VM is designed to support conventional, untagged, general register architectures. Suitably lobotomized, it could be mapped to less flexible hardware such as "Lisp machines", but the compiler would have serious difficulties supporting stack architectures.

The VM does not support concurrent lightweight processes. Locking primitives and deep-binding of specials would be needed.

The VM does not deal with operating systems interface issues at all. A minimal port would require implementing at least file and terminal I/O streams. `sbcl` implements system interfaces using aliens and other facilities built on top of them.

Major components:

- Specific virtual operations implemented by the VM (VOPs). VOPs are primarily the concern of the compiler, since it translates Lisp code into VOPs and then translates VOPs into the implementation.
- Sub-primitives that are used by Lisp code needing to perform operations below the Lisp level. The compiler implements some sub-primitives directly using VOPs, while others are translated into Lisp code. Sub-primitives provide a layer of insulation between the Lisp system code and the VM, since the Lisp code may assume the existence of operations that are not implemented directly by the VM. Only sub-primitives with fairly portable semantics are documented here. Others are in implementation-specific VM documentation.

#Not all sub-primitives are VOPs, and most VOPs are not sub-primitives.*

7.0.1 VOP base name rules

The names of VOPs that implement functions are based on the function name. Other VOPs may use any base that doesn't conflict with a function name. There are some rules used to obtain the base name for related operations.

To get the name of a setting operation, replace the string "`ref`" in the name with "`set`". If "`ref`" doesn't appear in the name, add the prefix "`set-`" to the base name. For example, `svref` becomes `svset`, and `symbol-value` becomes `set-symbol-value`.

To get the name of a conditional VOP from the name of a predicate, add the prefix "`if-`" to the predicate name. For example, `eq` becomes `if-eq`. `eq` by itself would be a VOP that returned true or false value.

Some operations check for some error condition, magically signalling the error through an implicit control transfer. These operations are prefixed with "`check-`", as in `check-fixnum` and `check-bound`.

7.0.2 VOP name prefixes and suffixes

Prefixes and suffixes are added to the base to get the names of variant versions of the VOP. The fully general VOP name looks like this:

{ `small-` | `fast-` } *name* { `-c` *info* } { `/`*type* { `=>` *result-type* } }

The `small-` and `fast-` prefixes indicates that the VOP does minimal safety checking and is optimized for space or speed, respectively. The absence of a prefix indicates the safest (or only) version. Usually if the `small-` VOP exists, it will be a synonym for either the fast version or the safe version, depending on which is smaller.

The `-c` suffix indicates that the some info that is passed as a normal argument to the base version of the VOP is passed as `codegen-info` in this version. A typical use would be for VOPs where it is important to use a different version when one of the arguments is a compile time constant. *info* is some (possibly null) string that indicates which `-c` variant is involved.

The `/`*type* suffix asserts that all operands that could be of *type* are. For example, `+/fixnum` adds two fixnums returning a fixnum, while `length/simple-vector` finds the length of a simple vector, but the result isn't a simple vector.

The `=>`*result-type* suffix supplies a result type assertion on the operation.

A not totally silly example of all these modifiers simultaneously is `fast-+-c/fixnum=>integer`. This operation would this operation adds two fixnums, one of which is a constant passed as codegen info, resulting in an integer. The implementation is optimized for speed at the expense of space and safety.

8 Data Types and Storage Resources

8.1 Lisp Objects

A Lisp object is fixed-size data structure that is organized in a way mandated by the VM implementation. The fixed format allows the VM to determine the type of the object. **Note** <Virtual type? VM type? Implementation type? ...provides the VM enough information about the type of the object for the VM to implement the VM-level semantics... ...supports the "dynamic types"...>

Lisp objects are stored in locations known as cells. They have the major types: immediate and non-immediate. Non-immediate objects may have the following subtypes:

- symbol (nil may be weird)
- cons
- ratio
- complex
- some float types
- g-vector
- i-vector
- string
- bit-vector
- environment (always has subtype)
- array header
- bignum
- structure
- pc (code vector)
- stack closure (control stack pointer)

Non-immediate objects are allocated in "type spaces". The type space of an object is characterized by a small integer known as the type code. Any two objects of one of the above boxed types will always have the same type code. **Note:** But not really... Some types might be allocated in different type spaces at different times. (?)

The type code doesn't totally describe the object. In general, subtype information may be involved.

Immediate types:

- character
- fixnum
- unbound trap
- short float

8.2 Type VOPs

We consider control transfer to be the fundamental result of comparison, rather than anything such as a condition code. Although most compilers with whizzy register allocation seem to explicitly allocate and manipulate the condition codes, it seems that any benefit is small in our case. This is partly because our VOPs are at a somewhat higher level, making it difficult to tell which VOPs do and don't trash the the CC. Explicitly incorporating condition codes in our VM also introduces another architecture dependency.

At the IR2 level, we have a class of `if-xxx` VOPs which transfer control to one of two places on the basis of some test on the operands. When generating code for a predicate, we peek at the destination `if` node to find where to transfer control to.

The exact representation of type tests in IR2 will be fairly implementation dependent, since it will depend on the specific type system for the given implementation. For example, if an implementation can test some types with a simple tag check, but other types require reading a field from the object in addition, then the two different kinds of checks should be distinct at the VOP level, since this will allow the VOP cost and storage information to be more accurate. Generation of type tests should be factored out of code which would otherwise be more portable. Probably the IR2 translator for `typep` and the type check generation code are the only places that should know about how type tests are represented in IR2.

`if-type` *object* [VOP]
`if-type-range` *object* [VOP]

`if-type` tests whether object has the type code that is passed in the codegen info. `if-type-range` tests for a range of type codes.

`{small, fast} if-vector-type` *object* [VOP]

Test that *object* is either of the specified type code, or is a 1d array header with data having the specified type code.

`if-vector-subtype` *object* [VOP]

Test the subtype field of a vector-like object. It is assumed that the object has already been determined to be vector-like.

`if-fixnump` *object* [VOP]
`if-short-float-p` *object* [VOP]
`if-characterp` *object* [VOP]

The rationale behind having these as separate VOPs is that they are likely to be immediate types, and thus may have bizzare type schemes.

`if-consp` *object* [VOP]
`if-listp` *object* [VOP]

We have distinct operations for these predicates since one or the other isn't a simple tag test, but we don't know which one.

`if-rationalp` *object* [VOP]
`if-floatp` *object* [VOP]
`if-integerp` *object* [VOP]

`if-numberp` *`object`* [VOP]
`if-vectorp` *`object`* [VOP]
`if-functionp` *`object`* [VOP]

The rationale behind having these operations is that they may take a lot of code, so it is reasonable to put them out of line.

8.3 Type Sub-primitives

`change-type` *`object`* => *result* [VOP]
`get-type` *`object`* [VOP]

Change the type of an object according to codegen info. The meaning of this is highly type-system dependent, but it doesn't matter, since the compiler will never emit this VOP directly. The only way that it can show up is through %Primitive.

Storage resources:

Boxed and unboxed locations:

bullet Non-immediate objects may not be stored in unboxed locations.

bullet Things not lisp objects may not be stored in boxed locations.

Also,

bullet Control stack is boxed.

bullet Optional number stack is unboxed.

bullet Heap environment is boxed.

bullet Fixed number of registers, some boxed and some unboxed.

PCs may be stored on the control stack or in boxed registers, subject to the constraint that a corresponding environment is also stored. Locations containing PCs don't need to be zeroed when they are no longer used; nothing bad will happen if an old PC is unaccompanied by an environment.

`[Trap] Illegal object trap`

This value is used in symbols to signify an undefined value or definition.

9 Characters

Character is an immediate type. Characters are manipulated primarily by converting into an integer and accessing these fields:

`%character-code-byte`
: **NOTE:** <EEK: appears obsolete> The character code. This is effectively required to start at bit 0, since CL equates `char-int` to `char-code` when there is no bits or font. All current `sbcl` systems use ASCII **NOTE:**<EEK unicode?> for the character codes, and define `\#\newline` to be a linefeed, but system code should not count on this.

`%character-control-byte`
: **NOTE:** <EEK: appears obsolete>
The character bits. Character bits are used by Hemlock to describe modifiers in keyboard events, but there is no assumption of any general portable significance of character bits. **NOTE:**<EEK surely this has been removed in SBCL>

`%character-font-byte`
: **NOTE:** <EEK: appears obsolete>
The character font. This is not used by `sbcl`, and is not particularly useful.

Characters should be converted to and from integers by using the SBCL

`char-int` [sb-impl]
`int-char` [sb-impl]
: The compiler translates these functions into these VOPs:

```
char-int (char) => int
int-char int => char
```

In the common case where *char* is known to be a `string-char`, these operations are equivalent to `char-code` and `code-char`. In addition to providing a portable interface to character conversion, the VOP representation of this type conversion allows the compiler to avoid unnecessary boxing and unboxing of character objects.
Existing code explicitly converts fixnums to characters by using the `make-immediate-type` sub-primitive with `%character-type`. Currently conversion of characters to fixnums is rather confused. Originally, characters were a subtype of the Misc type code, and the result of the `make-fixnum` sub-primitive had to be masked with `%character-int-mask`; some code still does this, while other code may not.

`if-char=` (*x y*) [obsolete]
`if-char<` (*x y*) [obsolete]
`if-char>` (*x y*) [obsolete]
: **NOTE:** <EEK: appears obsolete>
Character comparisons could be implemented by doing numeric comparisons on the result of `char-int`, or by using `eq` in the case of `char=`, but this can result in unnecessary type conversions. Instead, the compiler uses these conditional VOPs:

10 Symbols

Symbols are currently fairly boring in `sbcl`, containing only the obvious slots:

`symbol-value-slot` [sb-vm]
The current dynamic value of this symbol. If the symbol is currently unbound, then the value of this slot is the unbound marker.

`symbol-function-slot` [sb-vb]
NOTE: <EEK: appears obsolete> The global function function definition of this symbol. If the symbol is not fbound, then this slot holds the unbound marker.

`%symbol-plist-slot` [sb-vb]
`%symbol-name-slot` [sb-vb]
`%symbol-package-slot` [sb-vb]
NOTE: <EEK: Some appear to be obsolete>

10.1 Sub-primitives

`alloc-symbol` *`name`*
sub-primitive allocates a new symbol object. *name* is the simple-string that is to be the name of the symbol.

```
alloc-symbol name => symbol
```

`%set-symbol-package` [sb-impl]
sub-primitive is used by system code that must set the symbol package.

```
(%set-symbol-package symbol new-value)
```

10.2 Accessor VOPs

These VOPs read the global symbol value and definition cells. `constant-ref` may only be used on symbols that have been defined to be constants. Since a constant cannot change in value and cannot be dynamically bound, the compiler may be able to compile uses of `constant-ref` more efficiently. Unsafe versions of these VOPs may not check for the slot being unbound, which the corresponding SBCL functions are required to do.

`small, fast` *symbol-value* `symbol => value`
`small, fast` *constant-ref* `symbol => value`
`small, fast` *symbol-function* `symbol => value`
These VOPs set the global symbol value and definition cells. `makunbound` and `fmakunbound` are implemented by setting the value to the unbound marker.

`small, fast` *set-symbol-value* (*symbol new-value*)
`small, fast` *set-symbol-function* (*symbol new-value*)
The SBCL accessors for other symbol slots are translated into uses of the `slot-ref` and `slot-set` VOPs.

10.3 Special Binding

`bind` (*symbol value*)
`unbind` (*count*)

These VOPs implement dynamic binding of special variables using shallow binding. `bind` binds *symbol* to the specified *value*, while `unbind` undoes the most recent *count* special bindings on the binding stack.

10.4 Property Lists

`get` *`symbol indicator default`* => *`value`*
`put` *`symbol indicator value`*

The `get` VOP implements the corresponding SBCL function, while `put` implements its setf-inverse.

\chapterLists

cons

\beginexample list<n> (elt0 ... elt<n-1>) => list list (elt0 ... elt<n-1> more-elts) => list For some small N, we have fixed-arg versions of List. For larger lists, we pass in additional elements in a stack TN (possibly required to be on stack top). List* is similar. \endexample

These VOPs implement the corresponding SBCL functions: \beginexample small, fast car (list) => value small, fast cdr (list) => value \endexample

These VOPs set the car or cdr of a cons: \beginexample small, fast set-car (cons new-value) small, fast set-cdr (cons new-value) \endexample

These VOPs implement the SBCL \tt assoc and \tt member functions with test functions of \tt eql and \tt eq: \beginexample assoc (item alist) => cons-or-nil assq (item alist) => cons-or-nil member (item list) => cons-or-nil memq (item list) => cons-or-nil \endexample

\tt getf implements the corresponding SBCL function, while \tt putf is used to implement its setf-inverse. \tt putf returns the new value for the list so that it may stored back into the place. \beginexample getf (list indicator default) => value putf (list indicator new-value) => list \endexample

\chapterNumbers

\indexFixnum format Fixnum\\An N-bit two's complement integer.

\indexShort float format Short-Float\\An immediate float format.

\indexBignum format \labelBignums Bignum\\Bignums are infinite-precision integers, represented somehow.

\indexFlonum format \indexFloating point formats Floats\\Floats are stored as consecutive words of bits.

\indexRatio format Ratio\\Ratios are stored as two consecutive words of Lisp objects, which should both be integers.

\indexComplex number format Complex\\Complex numbers are stored as two consecutive words of Lisp objects, which should both be numbers.

\sectionNumber VOPs

integer-length small, fast integer-length/fixnum

float=>xxx-float

realpart lmagpart numerator denominator decode-float small, fast decode-float/xxx-float scale-float small, fast scale-float/xxx-float

if-= (x y) small, fast if-=/fixnum small, fast if-=/xxx-float Do numeric comparison of X and Y. The codegen-info contains the continuations to transfer to in the true and false cases. Same for $<$, $>$.

+ (x y) => z small, fast +/fixnum small, fast +/fixnum=>integer small, fast +/xxx-float Same for -, *. Fixnum multiplication by a constant power of 2 (or near power of 2) can be done by a transform.

/ (x y) => z small, fast //xxx-float

negate small, fast negate/fixnum small, fast negate/fixnum=>integer small, fast negate/xxx-float Ditto for Abs.

truncate (x y) => q r small, fast truncate/fixnum

logand (x y) => z small, fast logand/fixnum Ditto for logior, logxor.

lognot (n) => z small, fast lognot/fixnum

ash (n x) => z small, fast ash/fixnum small, fast ash-c/fixnum

ldb dpb mask-field deposit-field These will only be used as a last resort. There should be transforms that turn fixnum operations with constant byte-specifiers into standard logical operations.

\sectionNumber Sub-primitives

alloc-bignum make-complex make-ratio lsh logldb logdpb

\chapterArrays

\cl arrays can be represented in a few different ways in \rtccl – different representations have different performance advantages. Simple general vectors, simple vectors of integers, and simple strings are basic \rtccl data types, and access to these structures is quicker than access to non-simple (or "complex") arrays. However, all multi-dimensional arrays in \rtccl are complex arrays, so references to these are always through a header structure.

Once a vector has been allocated, it is possible to reduce its length by using the Shrink-Vector sub-primitive, but never to increase its length, even back to the original size, since the space freed by the reduction may have been reclaimed.

\subsectionArrays \labelArrays \indexArrays

An array header is identical in form to a G-Vector. At present, the following subtype codes are defined: \beginitemize, spread 0, spacing 1 0 Normal. 1 Array is displaced to another array (which may be simple). \enditemize The entries in the header-vector are interpreted as follows:

\indexArray header format \begindescription 0 Data Vector \\This is a pointer to the I-Vector, G-Vector, or string that contains the actual data of the array. In a multi-dimensional array, the supplied indices are converted into a single 1-D index which is used to access the data vector in the usual way. If the array is displaced, then this is the array displaced to, which may be an array header. In general, array access must loop until it finds an actual data vector.

1 Number of Elements \\This is a fixnum indicating the number of elements for which there is space in the data vector.

2 Fill Pointer \\This is a fixnum indicating how many elements of the data vector are actually considered to be in use. Normally this is initialized to the same value as the Number of Elements field, but in some array applications it will be given a smaller value. Any access beyond the fill pointer is illegal.

3 Displacement \\This fixnum value is added to the final code-vector index after the index arithmetic is done but before the access occurs. Used for mapping a portion of one array into another. For most arrays, this is 0.

4 Range of First Index \\This is the number of index values along the first dimension, or one greater than the largest legal value of this index (since the arrays are always zero-based). A fixnum in the range 0 to 2\+24-1. If any of the indices has a range of 0, the array is legal but will contain no data and accesses to it will always be out of range. In a 0-dimension array, this entry will not be present.

5 - N Ranges of Subsequent Dimensions \enddescription

The number of dimensions of an array can be determined by looking at the length of the array header. The rank will be this number minus 6. The maximum array rank is 65535 - 6, or 65529.

The ranges of all indices are checked on every access, during the conversion to a single data-vector index. In this conversion, each index is added to the accumulating total, then the total is multiplied by the range of the following dimension, the next index is added in, and so on. In other words, if the data vector is scanned linearly, the last array index is the one that varies most rapidly, then the index before it, and so on.

\sectionArray VOPs

alloc-bit-vector alloc-i-vector alloc-string alloc-g-vector Initialized and uninitialized versions?

length (sequence) => size small, fast length/vector small, fast length/simple-vector small, fast length/simple-string small, fast length/simple-bit-vector

aref1 (vector index) => value small, fast aref1/simple-vector small, fast aref1/simple-string small, fast aref1/simple-bit-vector small, fast aref1/simple-array-XXX-float

aset1 (vector index new-value) small, fast aset1/simple-vector small, fast aset1/simple-string small, fast aset1/simple-bit-vector small, fast aset1/simple-array-XXX-float

small, fast aref1/simple-array-unsigned-byte (vector index) => value small, fast aset1/simple-array-unsigned-byte (vector index new-value) Byte size is codegen info.

aref$<$N$>$ (array index0 ... index<n-1>) => value aset$<$N$>$ (array index0 ... index<n-1> new-value) For some small value of N. Of course, higher dimensional arrays can also be specialized in seven different ways.... Multi-dimensional simple array reference with known dimensions can be open-coded using a transform (useful for benchmarks.)

\sectionArray Sub-primitives

alloc-array vector-subtype set-vector-subtype vector-access-code set-vector-access-code shrink-vector

typed-vref typed-vset

header-length (header) => size header-ref (header index) => value header-set (header index new-value)

bit-bash byte-blt reverse-find-character reverse-find-character-with-attribute reverse-string-compare sxhash-simple-string sxhash-simple-substring

\chapterStructures

small, fast structure-ref (s) => value small, fast structure-set (s new-value) Read and write structure slots. Defstruct slot description is in codegen info.

alloc-structure

\chapterRuntime Environment \indexRuntime Environment \labelRuntime

\sectionRegister Allocation \indexRegister allocation

The main idea is to globally allocate only those registers with global significance.

We permanently dedicate the CONT register to point to the current control stack environment. This is the "frame pointer" in standard terminology. It isn't possible to get pack to allocate this register on an as-needed basis due to the classic phase-ordering problem. We need to know if TNs are allocated on the stack before we can determine tell how badly we need a frame pointer register. This is of little significance with the control stack environment, since we almost always need one, and if there are any stack TNs, we must allocate the frame pointer in a register, since there is nowhere else to put it. The problem is more severe with a number stack environment pointer. We can't dedicate a register to it, since we usually don't have any TNs on the number stack. The only easy solution is to always allocate the number stack environment pointer on the control stack. This really isn't too bad, when you compare the cost of doing an extra memory reference to get at the number stack to the cost of number-consing.

We also dedicate the ENV register to the current constant pool. It would be possible to explicitly allocate the constant pointer as needed if we explicitly represented non-immediate constant access by a VOP, but this would be extra work, and there are major advantages to representing all constants using TNs. Another potential efficiency advantage is since the same constant pool is shared by all the code in a component, we need only initialize ENV on entry to the component. When we make local calls, we don't have to do anything to make the constants available to the callee.

Since the constant pool will also contain the code vector and the debug info, having it in a known place may make life easier for GC and the debugger. We may not be able to count on it too much, though, since ENV holds other things will calls are in progress, and might be pretty random if we jumped into hyperspace.

Runtime environment:

CONT: the current control stack context. PC is assumed to be accessible to the debugger when an error happens. Current-Catch: pointer to the current catch frame. Format of frame is assumed. Current-Unwind-Protect: current unwind protect frame. Similar to catch.

If shallow-bind, binding stack and binding stack pointer. If deep-bind, current special binding. Format of binding frame assumed.

Everything depends on the current environment, which is CONT.

\beginverbatim PC OLD-CONT ENV A<n> CONT CS \endverbatim

\sectionOther Dynamic State

There are some dynamic state variables that are stored in known memory locations, rather than having a dedicated register: \begindescription binding stack pointer\\The current pointer to the top of the binding stack.

current catch\\The pointer to the current catch block.

current unwind-protect\\The pointer to the current unwind-protect block. \enddescription

\sectionControl-Stack Format \labelControl-Stack-Format \indexControl-stack format

The control stack contains only Lisp objects. Every object pointed to by an entry on this stack is kept alive.

The \rtccl control stack does not have a rigid frame structure. The compiler is allowed a large amount of freedom in the use of the stack so that it choose the best calling sequences. Mostly the compiler is the only system that cares how the stack is laid out, so this isn't a big problem. See chapter \refdebug-info for a description of the structures which allow the debugger to parse the stack.

\sectionValues Passing Conventions

The first \it nregs arguments are passed in registers, where nregs is an implementation dependent constant. Any additional arguments are the block of storage between CONT and CS on the control stack. The first nregs locations in this block of storage are unused so that register more-args can be stored on the stack without having to BLT the stack values up.

Returning unknown values are passed in a similar way, but the stack values block is between OLD-CONT and CS. There isn't any underneath the values: on return OLD-CONT is always what CS was when the function was called. The function returned to must copy the values into the desired location in its frame and deallocate excess stuff on the top of the stack.

More args are represented by a pointer to the block of values and a count. The function that originally created the more arg must allocate and deallocate this stuff somehow. In the case of a local call to a more arg entry, we can just allocate it as a TN. The external entry point for a more arg entry is more magical.

The caller allocates the environment for the called function, stores the arguments into it, and jumps to the function. The caller makes the called environment current, passing in the return OLD-CONT and PC as explicit arguments.

When returning values, the returner directly stores the return values into the frame being returned to. This works even though the caller doesn't know what function it is returning to, since the same return locations are allocated in all frames.

In a tail-recursive call, we can destructively modify the current frame and jump right to the callee, rather than allocating a new frame. We can do this because

TNBind globally allocates frame locations; all frames are the same size and have the same TNs in the same place.

\sectionBinding-Stack Format \indexBinding stack format \comment<In a symbol chapter?>

The special binding stack is used to hold previous values of special variables that have been bound. It grows and shrinks with the depth of the binding environment, as reflected in the control stack. This stack contains symbol-value pairs, with only boxed Lisp objects present.

Each entry of the binding-stack consists of two boxed (32-bit) words. Pushed first is a pointer to the symbol being bound. Pushed second is the symbol's old value (any boxed item) that is to be restored when the binding stack is popped.

\chapterFunctions

Function calling is a way of life.

every function is a closure. pointer to current closure is passed in ENV unless it isn't (in local call may be elsewhere).

The description of the representation of functions and the function calling conventions is a large part of the VM description, since: Function calling is one of the most complicated facilities provided by the VM.

Everything that happens, happens in a function, so all parts of the system tend to get dragged in.

Aspects of function call: Control Environment CONT, ENV Argument/value passing Argument/value count dispatching

\sectionFunction Object Format \labelFn-Format

The old notion of a "function object" is now broken down into four different parts: \begindescription Function entry\\A function entry is a structure that holds the information that we need to call a function. This is the user visible function object.

Environment\\The environment is stuff that a function needs when it runs. This includes constants computed at load time and variables closed over at run time. Environment information may be allocated in the function entry structure after the required linkage information.

Entry information\\This is information about a specific function entry that is occasionally referenced at run time, but need not be immediately accessible. Entry information will be either allocated in the function entry or in the environment that it points to.

Debug information\\This is information about a function that isn't normally needed at run time. Debug information can be found by poking around in environment objects. \enddescription See chapter \refcontrol-conventions for a description of how function objects are used.

\sectionEnvironment Object Sub-primitives

alloc-code ? alloc-closure?

\subsectionDebug Information Location

If present, debug information is stored immediately following any fixed information in the environment object. It may be necessary to chain up multiple levels of

environments to find the debug information. The debug information can be recognized because it is represented by a defstruct structure. See chapter \refdebug-info for a description of the debug information.

\sectionFunction Calls \indexfunction call

`sbcl` supports three major calling conventions. The convention used depends on the amount of information available at compile time: \begindescription Local\\Local call is used when the call and the called function are compiled at the same time. Using the term "convention" to describe this call mechanism is somewhat of a misnomer, since the compiler can do whatever it wants.

Named\\Named call is used when the call is to a global function whose name is known at compile time.

Anonymous\\Anonymous call is used when the function called is unknown until run time. \enddescription

\#| IR2 function call:

Environment manipulation code is always emitted at the location of the Bind or Return node for a Lambda.

Implicit args to functions in IR2: \begindescription \item[old-cont] cont to restore on return \item[return-pc] pc to return to \item[env] pointer to current closure (if heap) \item[closure$<$n$>$] closed values for current closure (if stack) \enddescription

Other info needed for IR2 conversion of functions: \beginitemize \item base pointers for all heap closures consed by this function also have passing locs for each explicit arg return strategy (known or unknown) and return locs \enditemize

All arguments including implicit ones must have both a passing TN and a permanent TN. Passing locs for let calls can be the actual TN that holds the variable in the case of local variables. Set closure variables must still have a separate passing TN.

If we know the values counts for the argument continuations, then we compile local mv-calls by moving the TNs for the values continuations into the argument passing locations. Other mv-calls must be compiled using various hairy stack-hacking VOPs and unknown argument count call VOPs.

For now, we will create the callee's frame just before the call, instead of creating it before the evaluation of the first argument. If we created the environment early, then we would be able to move the argument values directly into the frame, instead of having to store them somewhere else for a while. The problem with early creation is that lifetime analysis gets confused because there is more than one instance of the same TN present simultaneously in the case where there are nested calls to the same function.

It turns out that there isn't a problem with a simple self-call, because the TN in the called frame is really the "same" TN as the one in the current frame, due to the restricted way in which we use the passing TNs.

We emit code for external entry points during IR2 conversion. The external entry point is the place where we start running in a full call from a function-entry. It does arg count checking and dispatching, moves the arguments into the passing

locations for the for the lambda being called, and calls the lambda, moving the results into the standard locations if there aren't there already. |\#

In IR2, the environment manipulation semantics of function call are decoupled from the control semantics. When allocating closure variables for a Let, it is possible to do environment manipulation with only the normal sequential control flow. In the case of a Let call with the same environment, we neither manipulate the environment nor transfer control; we merely initialize the variables with Move VOPs.

If a local function returns a known number of values which is less than the number expected by the caller, then additional code must be inserted at the return site which sets the unused values to NIL.

The full function call mechanism must effectively be a subset of the local call mechanism, since the two mechanisms must mesh at entry points and full function calls. A full call turns into some kind of full call VOP. There are different VOPs for calling named functions and closures. We also have tail-recursive full call VOPs. Arguments are set up using Move VOPs, just as for local call. The only difference is that the passing locations and conventions are restricted to the standard ones.

The gory details of arg count checking and dispatching are buried in the Function-Entry VOP, which takes a functional and a list of continuations, one pointing to each external entry.

\subsectionLocal Call \indexlocal call

Named and anonymous call are called full calls, to distinguish them from local call. When making full calls, the compiler must make many worst-case assumptions that aren't necessary in a local call. The advantage of local call is that the compiler can choose to use only those parts of the full call sequence that are actually necessary.

In local call, we always know the function being called, so we never have to do argument count checking, and can always use an immediate branch for the control transfer. If the function doesn't return to more than one place, then can just use a simple branch, or even drop through.

The argument passing TNs may be allocated anywhere. The caller allocates the stack frame for the called function, moving any non-register arguments into the passing locations in the callee's frame.

If we are calling a local function that doesn't always return the same number of values, then we must use the same values returning mechanism that is used in full call, but we don't have to use the standard registers.

A tail-recursive local call doesn't require any call VOP. We just use Move VOPs to put the arguments into the passing locations and then jump to the the start of the code for the function. We don't have to do any stack hackery since we use the same stack frame format for all the functions compiled at the same time. In many cases tail-recursive local calls can be entirely optimized away, since they involve only some moves and a branch. We preference the argument values to the passing locations of the called function, making it likely that no move will be necessary. Often the control transfer can be done by simply dropping through.

We have to do some funny stuff with local calls in order to get the lifetimes for the passing locations right, since lifetime analysis skips directly from the return point

to the call point, ignoring the uses of the passing locations in the called function. Similarly, we pretend that a block ending in a return has no successors.

call-local (arg*) “fun” => value multiple-call-local (arg*) “fun” =$>$ start end val0 ... val$<$n$>$ Call-Local is used for calls to local functions that are forced to use the unknown-values passing convention. Value is the first return value register; we don’t really do anything to it, but we specify it as a result to represent the assignment done by the calling function.

Multiple-Call-Local is similar, but specifies all the values used by the unknown-values convention. Default-Values may be used to receive a specific number of values.

known-call-local (arg*) “fun” =$>$ value* This VOP is used for local calls to functions where we can determine at compile time that the number of values returned is always the same. In this case, we don’t need to indicate the number of values, and can pass them in separate TNs. The Values are the actual return locations. We don’t really do anything to the return values; we just specify them as results to represent the assignment done by the called function.

known-return (return-pc value*) “fun” This VOP is used for returning from local calls using the known return values convention. The specified return Values are moved into the passing locations in the caller’s frame.

If we know that the function we are calling is non-recursive, then we can compile it much like a tail-recursive call. We must have a call VOP to compute the return PC, but we don’t need to allocate a frame or save registers. We just set up the arguments in the frame and do the call.

We require simple functions to use the known-values convention. It would be possible to support unknown values, but it would potentially require BLT’ing return values out of the frame and on to the top of the stack. Supporting unknown values would also require a bunch more VOPs, since we need different call and return VOPs for simple call.

Known values return causes no problems, since the callee knows how many values are wanted. We store the values directly into the current frame, since it is also the caller’s frame.

known-call-simple () “fun” =$>$ return-pc known-return-simple (return-pc) “fun” Similar to the non-simple VOPs, but don’t allocate or deallocate frames, and assume that argument and value passing is done with explicit Move VOPs.

\subsectionFull Call \indexfull call

Both named and anonymous call are optimized for calls where the number of arguments is known at compile time. Unknown argument calls are a pathological case of anonymous call; this case will be ignored in the main discussion. The difference between named and anonymous calls is in the argument count dispatching mechanism.

Named call allows an arbitrary number of entry points, with start PCs at arbitrary locations in the code vector. The link-table mechanism described below allows named calls to jump directly to the actual entry point without any run-time argument count or type checking checking.

Anonymous call has a fixed number of entry points, with start PCs at fixed locations in the code vector. This allows calls to be made without knowing what function is being called, but has more run-time overhead. The object called must be checked to be a valid function-entry object. The entry PC must be computed from the function entry, and argument count checking must be done if there are more than three required or optional arguments.

Argument passing in full call is conceptually similar to local call, but the caller can't allocate the entire frame for the callee, since it doesn't know how much stack is needed. Instead we allocate the frame in two parts. The caller only allocates the beginning of the frame, which contains the stack arguments in fixed locations. We leave the first $<$n$>$ locations unused so that the called function can move register more args onto the stack without having to BLT down any stack arguments.

The place in the code where a full call jumps in is called an external entry point. The external entry point allocates the rest of the stack frame and then does a local call to the actual entry-point function, fetching the arguments from the standard passing locations. Usually we can do a tail-recursive local call.

There are two main cases where the call from the external entry point cannot be tail-recursive: – It is desirable to use the known-values convention for calling the entry-point function if the entry-point is used in other local calls (perhaps because of recursion). In this case, the called function stores the return values back into the frame allocated by the external entry point and then returns back to it. The external entry point must then return these values using the standard unknown-values convention. – In a more-arg entry point we don't know how many stack arguments there are at the beginning of the frame, so we can't really use the frame allocated by the external entry point at all. Instead we do a local call to the more-arg entry point, passing in a pointer to the first extra value. When the function returns, we deallocate the crap on the stack and then return the values. It is still o.k. to use the known-values return convention from the more-arg entry since the extra arg values are no longer needed by the time the returning function stores the return values back into the external entry point frame.

In full call we must always use the unknown-values convention for return. The first $<$n$>$ values are passed in the standard argument registers. The Old-Cont register holds the Start of the values block and SP points to the End.

small, fast call (function arg0 ... arg$<$n$>$) "nargs" =$>$ value small, fast call-named (arg0 ... arg$<$n$>$) "nargs" "name" =$>$ value Call-Closure calls Function with the specified register arguments, returning the first value as the result. "nargs" is the total number of arguments passed. Only the register arguments actually passed should be specified as operands.

Call-Named is similar, but calls a global function specified at compile time by "name".

small, fast tail-call (function pc arg0 ... arg$<$n$>$) "nargs" small, fast tail-call-named (pc arg0 ... arg$<$n$>$) "nargs" "name" Similar to the standard call VOPs, but passes PC as the return PC, rather than returning to the call site. These VOPs have no results since they don't return.

small, fast multiple-call (function arg0 ... arg$<$n$>$) “nargs” =$>$ start end val0 ... val$<$n$>$ small, fast multiple-call-named (arg0 ... arg$<$n$>$) “nargs” “name” =$>$ start end val0 ... val$<$n$>$ These VOPs are similar to the standard call VOPs, but allow any number of values to be received by returning all the value passing registers as results. A specific number of values may be received by using Default-Values.

call-unknown (function count arg0 ... arg$<$n$>$) =$>$ start end val0 ... val$<$n$>$ tail-call-unknown (function pc count arg0 ... arg$<$n$>$) Call a function with an unknown number of arguments. Used for apply and hairy multiple-value-call.

Function-Entry () “function” =$>$ env return-pc old-cont arg* This marks the place where we jump into a component for an external entry point. It represents whatever magic is necessary to do argument count checking and dispatching. The external entry points for each argument count will be successors of the entry-vector block (might be in the same block if only one?)

Function-Entry also represents argument passing by specifying the actual external passing locations as results, thus marking the beginning of their lifetimes. All passing locations actually used by any entry point are specified as Args, including stack arguments. \#\#\# Do we really need this? If we do, then we probably also need similar entry markers for local functions. The lifetimes don’t really need to be explicitly bounded, since an entry point is effectively “the end of the world.”

\section(Returning from a Function Call) \label(Return) \index(Return)

return (return-pc value) multiple-return (return-pc start end val0 ... val$<$n$>$) Return Value from the current function, jumping back to the location specified by Return-PC. Perhaps allow to return any fixed, known number of values.

Multiple-Return is similar, but allows an arbitrary number of values to be returned. End - Start is the total number of values returned. Start points to the beginning of the block of return values, but the first $<$n$>$ values val0 ... val$<$n$>$ are actually returned in registers.

default-values (start end val0 ... val$<$n$>$) =$>$ val0 ... val$<$j$>$ This VOP is used when we want to receive exactly J values. If fewer than J values were supplied, then missing values are defaulted to NIL. As a side-effect, this VOP pops off any returned stack values.

\sectionSaving and Restoring Registers

We use a caller-saves convention. The caller explicitly emits saving and restoring code. Tail-recursive calls don’t need any register saving since we never come back.

\chapterNon-local exits

\subsectionUnwind Blocks \indexCatch \indexCatch frames

There is one aspect of the control stack format that is fixed, and which concerns us at this level. This is the format of the “frames” which mark the destination of non-local exits, such as for BLOCK and CATCH. These frames are collectively known as unwind blocks. The basic unwind block is used for lexical exists such as BLOCK, and for UNWIND-PROTECT. Its format is the following: \beginverbatim 0 Pointer to current unwind-protect. 1 Control stack context to restore on entry. 2 PC to enter at. \endverbatim

The unwind block for CATCH is identical except for additional cells containing the catch tag and previous catch. \beginverbatim 0 Pointer to current unwind-protect. 1 Control stack context to restore on entry. 2 PC to enter at. 3 Catch tag. 4 Previous catch. \endverbatim

The conventions used to manipulate unwind blocks are described in chapter \refControl-Conventions.

\sectionNon-Local Exits \labelCatch \indexCatch \indexThrow \indexUnwinding \indexUnwind-Protect \indexNon-Local Exits

In the normal flow of control, each function that is called executes until it reaches a return point; under these conditions no special effort is needed to restore the environment as long as each function undoes any change that it makes to the dynamic state before it returns. When we make a non-local transfer, we skip a potentially arbitrary collection of these cleanup actions. Since we cannot in general know what changes have been made to the dynamic environment below us on the stack, we must restore a snapshot of the dynamic environment at the re-entry point.

We represent the closed continuation by the pointer to the unwind-block for the reentry point. At the exit point, we just pass this stack pointer to the Unwind VOP, which deals with processing any unwind-protects. When Unwind is done, it grabs the re-entry PC out of the location at the stack pointer and jumps in.

Catch and Unwind-Protect work in pretty much the same way. We make a stack TN to hold the catch frame or whatever, allocate TNs in them to represent the slots, and then initialize them. The frame can be explicitly linked in by TN manipulations, since the active catch and whatnot are represented by TNs. Since allocation of the frame is decoupled from linking and unlinking, some of this stuff could be moved out of loops. We will need a VOP for loading the PC for an arbitrary continuation so that we can set up the reentry PC. This can be done using the Call VOP. Using a call instruction is probably a good way to get a PC on most architectures anyway.

These TNs are allocated by Pack like any others; we use special alloc and dealloc VOPs to delimit the aggregate lifetimes.

In the non-local case, the the Block, Catch and Unwind-Protect special forms are implemented using unwind blocks. The unwind blocks are built by move operations emitted inline by the compiler. The compiler adds and removes catches and unwind protects by explicit moves to the locations that hold the current catch and unwind protect blocks. The entry PC is loaded using the Call VOP.

The Unwind miscop is the basis non-local exits. It takes the address of an unwind block and processes unwind-protects until the current unwind-protect is the one recorded in the unwind block, then jumps in at the entry in the unwind block. The entry for the unwind block is responsible for restoring any state other than the current unwind-protect.

Unwind is used directly to implement non-local Return-From. The address of the unwind block is stored in a closure variable.

Catch just does a scan up the chain of Catch blocks, starting at the current catch. When it finds the right one, it calls unwind on it.

Unwind-protects are represented by unwind blocks linked into the current unwind-protect chain. The cleanup code is entered just like any other any other

unwind entry. As before, the entry is responsible for establishing the correct dynamic environment for the cleanup code. The target unwind block is passed in some non-argument register. When the cleanup code is done, it just calls Unwind with the block passed in. The cleanup code must be careful not to trash the argument registers or CS, since there may be multiple values lurking out there.

With Catch/Throw, we always use the variable values return value passing convention, since we don't know how many values the catch wants. With Block/Return-From, we can do whatever we want, since the returner and receiver know each other.

If a Block or Catch receives stack values, it must call a VOP that BLT's the values down the stack, squeezing out any intermediate crud.

unwind (context) throw (tag) Unwind does a non-local exit, unwinding to the place indicated by Context. Context is a pointer to a block of storage allocated on the control stack, containing the entry PC, current environment and current unwind-protect. We scan up the stack, processing unwind-protects until we reach the entry point. The values being returned are passed in the standard locations. Throw is similar, but does a dynamic lookup for the Tag to determine what context to unwind to.

User Interface to the Compiler

11 User Interface of the Compiler

11.1 Error Message Utilities

11.2 Source Paths

Retargeting the Compile

\partCompiler Retargeting

12 Retargeting the Compiler

[\#\#\#

In general, it is a danger sign if a generator references a TN that isn't an operand or temporary, since lifetime analysis hasn't been done for that use. We are doing weird stuff for the old-cont and return-pc passing locations, hoping that the conflicts at the called function have the desired effect. Other stuff? When a function returns unknown values, we don't reference the values locations when a single-value return is done. But nothing is live at a return point anyway.

Have a way for template conversion to special-case constant arguments? How about: If an arg restriction is (:satisfies [$<$predicate function$>$]), and the corresponding argument is constant, with the constant value satisfying the predicate, then (if any other restrictions are satisfied), the template will be emitted with the literal value passed as an info argument. If the predicate is omitted, then any constant will do.

We could sugar this up a bit by allowing (:member $<$object$>$*) for (:satisfies (lambda (x) (member x '($<$object$>$*))))

We could allow this to be translated into a Lisp type by adding a new Constant type specifier. This could only appear as an argument to a function type. To satisfy (Constant $<$type$>$), the argument must be a compile-time constant of the specified type. Just Constant means any constant (i.e. (Constant *)). This would be useful for the type constraints on ICR transforms.

Constant TNs: we count on being able to indirect to the leaf, and don't try to wedge the information into the offset. We set the FSC to an appropriate immediate SC.

Allow "more operands" to VOPs in define-vop. You can't do much with the more operands: define-vop just fills in the cost information according to the loading costs for a SC you specify. You can't restrict more operands, and you can't make local preferences. In the generator, the named variable is bound to the TN-ref for the first extra operand. This should be good enough to handle all the variable arg VOPs (primarily function call and return). Usually more operands are used just to get TN lifetimes to work out; the generator actually ignores them.

Variable-arg VOPs can't be used with the VOP macro. You must use VOP*. VOP* doesn't do anything with these extra operand except stick them on the ends of the operand lists passed into the template. VOP* is often useful within the convert functions for non-VOP templates, since it can emit a VOP using an already prepared TN-Ref list.

It is pretty basic to the whole primitive-type idea that there is only one primitive-type for a given lisp type. This is really the same as saying primitive types are disjoint. A primitive type serves two somewhat unrelated purposes: – It is an abstraction of a Lisp type used to select type specific operations. Originally kind of an efficiency hack, but it lets a template's type signature be used both for selection and operand representation determination. – It represents a set of possible representations for a value (SCs). The primitive type is used to determine the legal SCs for a TN, and is also used to determine which type-coercion/move VOP to use.

]

There are basically three levels of target dependence:

- Code in the "front end" (before VMR conversion) deals only with Lisp semantics, and is totally target independent.
- Code after VMR conversion and before code generation depends on the VM, but should work with little modification across a wide range of "conventional" architectures.
- Code generation depends on the machine's instruction set and other implementation details, so it will have to be redone for each implementation. Most of the work here is in defining the translation into assembly code of all the supported VOPs.

13 Storage bases and classes

Storage base

name [sb-allocated-size]

The size of the *name* SB in the currently compiled component. This is useful mainly for finding the size for allocating stack frames.

New interface: instead of `current-frame-size`, have `current-sb-size` `\verb+<name>+` which returns the current element size of the named SB.

New interface: instead of `current-frame-size`, have `current-sb-size` `\verb+<name>+` which returns the current element size of the named SB.

(* 4 (sb-allocated-size 'stack))

;; in codgen.lisp (defun sb-allocated-size (name) "The size of the Name'd SB in the currently compiled component. Useful mainly for finding the size for allocating stack frames." (finite-sb-current-size (sb-or-lose name)))

How can we have primitive types that overlap, i.e. `(unsigned-byte 32)`, `(signed-byte 32)`, `fixnum`?

Primitive types are used for two things:

- Representation selection: which SCs can be used to represent this value? For this purpose, it isn't necessary that primitive types be disjoint, since any primitive type can choose an arbitrary set of representations. For moves between the overlapping representations, the move/load operations can just be noops when the locations are the same (vanilla MOVE), since any bad moves should be caught out by type checking.
- VOP selection: Is this operand legal for this VOP? When ptypes overlap in interesting ways, there is a problem with allowing just a simple ptype restriction, since we might want to allow multiple ptypes. This could be handled by allowing "union primitive types", or by allowing multiple primitive types to be specified (only in the operand restriction.) The latter would be along the lines of other more flexible VOP operand restriction mechanisms, (constant, etc.)

Ensure that `load/save-operand` never need to do representation conversion.

The PRIMITIVE-TYPE more/coerce info would be moved into the SC. This could perhaps go along with flushing the TN-COSTS. We would annotate the TN with best SC, which implies the representation (boxed or unboxed). We would still need to represent the legal SCs for restricted TNs somehow, and also would have to come up with some other way for pack to keep track of which SCs we have already tried.

An SC would have a list of "alternate" SCs and a boolean SAVE-P value that indicates it needs to be saved across calls in some non-SAVE-P SC. A TN is initially given its "best" SC. The SC is annotated with VOPs that are used for moving between the SC and its alternate SCs (load/save operand, save/restore register). It is also annotated with the "move" VOPs used for moving between this SC and all

other SCs it is possible to move between. We flush the idea that there is only c-to-t and c-from-t.

But how does this mesh with the idea of putting operand load/save back into the generator? Maybe we should instead specify a load/save function? The load/save functions would also differ from the move VOPs in that they would only be called when the TN is in fact in that particular alternate SC, whereas the move VOPs will be associated with the primary SC, and will be emitted before it is known whether the TN will be packed in the primary SC or an alternate.

I guess a packed SC could also have immediate SCs as alternate SCs, and constant loading functions could be associated with SCs using this mechanism.

So given a TN packed in SC X and an SC restriction for Y and Z, how do we know which load function to call? There would be ambiguity if X was an alternate for both Y and Z and they specified different load functions. This seems unlikely to arise in practice, though, so we could just detect the ambiguity and give an error at define-vop time. If they are doing something totally weird, they can always inhibit loading and roll their own.

Note that loading costs can be specified at the same time (same syntax) as association of loading functions with SCs. It seems that maybe we will be rolling DEFINE-SAVE-SCS and DEFINE-MOVE-COSTS into DEFINE-STORAGE-CLASS.

Fortunately, these changes will affect most VOP definitions very little.

A *Storage Base* (SB) represents a physical storage resource such as a register set or stack frame. Storage bases for non-global resources such as the stack are relativized by the environment that the TN is allocated in. Packing conflict information is kept in the storage base, but non-packed storage resources such as closure environments also have storage bases.

Some storage bases:

- General purpose registers
- Floating point registers
- Boxed (control) stack environment
- Unboxed (number) stack environment
- Closure environment

A storage class is a potentially arbitrary set of the elements in a storage base. Although conceptually there may be a hierarchy of storage classes such as "all registers", "boxed registers", "boxed scratch registers", this doesn't exist at the implementation level. Such things can be done by specifying storage classes whose locations overlap. A TN shouldn't have lots of overlapping SC's as legal SC's, since time would be wasted repeatedly attempting to pack in the same locations.

There will be some SC's whose locations overlap a great deal, since we get Pack to do our representation analysis by having lots of SC's. An SC is basically a way of looking at a storage resource. Although we could keep a fixnum and an unboxed representation of the same number in the same register, they correspond to different SC's since they are different representation choices.

TNs are annotated with the primitive type of the object that they hold:

- `t`: random boxed object with only one representation.

- `fixnum`, `integer`, `xxx-float`: object is always of the specified numeric type.
- `string-char`: object is always a string-char.

When a TN is packed, it is annotated with the SC it was packed into. The code generator for a VOP must be able to uniquely determine the representation of its operands from the SC. (debugger also...)
Some SCs:

- `reg`: any register (immediate objects)
- `save-reg`: a boxed register near r15 (registers easily saved in a call)
- `boxed-reg`: any boxed register (any boxed object)
- `unboxed-reg`: any unboxed register (any unboxed object)
- `float-reg`, `double-float-reg`: float in fp register.
- `stack`: boxed object on the stack (on cstack)
- `word`: any 32bit unboxed object on nstack.
- `double`: any 64bit unboxed object on nstack.

We have a number of non-packed storage classes which serve to represent access costs associated with values that are not allocated using conflicts information. Non-packed TNs appear to already be packed in the appropriate storage base so that Pack doesn't get confused. Costs for relevant non-packed SC's appear in the TN-Ref cost information, but need not ever be summed into the TN cost vectors, since TNs cannot be packed into them.

There are SCs for non-immediate constants and for each significant kind of immediate operand in the architecture. On the RT, 4, 8 and 20 bit integer SCs are probably worth having.

\beginverbatim Non-packed SCs: Constant Immediate constant SCs: Signed-Byte-<N>, Unsigned-Byte-<N>, for various architecture dependent values of <N> String-Char XXX-Float Magic values: T, NIL, 0. \endverbatim

\chapterType system parameterization

The main aspect of the VM that is likely to vary for good reason is the type system:

– Different systems will have different ways of representing dynamic type information. The primary effect this has on the compiler is causing VMR conversion of type tests and checks to be implementation dependent. Rewriting this code for each implementation shouldn't be a big problem, since the portable semantics of types has already been dealt with.

– Different systems will have different specialized number and array types, and different VOPs specialized for these types. It is easy to add this kind of knowledge without affecting the rest of the compiler. All you have to do is define the VOPs and translations.

– Different systems will offer different specialized storage resources such as floating-point registers, and will have additional kinds of primitive-types. The storage class mechanism handles a large part of this, but there may be some problem in getting VMR conversion to realize the possibly large hidden costs in implicit

moves to and from these specialized storage resources. Probably the answer is to have some sort of general mechanism for determining the primitive-type for a TN given the Lisp type, and then to have some sort of mechanism for automatically using specialized Move VOPs when the source or destination has some particular primitive-type.

\#| How to deal with list/null(symbol)/cons in primitive-type structure? Since cons and symbol aren't used for type-specific template selection, it isn't really all that critical. Probably Primitive-Type should return the List primitive type for all of Cons, List and Null (indicating when it is exact). This would allow type-dispatch for simple sequence functions (such as length) to be done using the standard template-selection mechanism. [Not a wired assumption] |\#

\chapterVOP Definition

Before the operand TN-refs are passed to the emit function, the following stuff is done: – The refs in the operand and result lists are linked together in order using the Across slot. This list is properly NIL terminated. – The TN slot in each ref is set, and the ref is linked into that TN's refs using the Next slot. – The Write-P slot is set depending on whether the ref is an argument or result. – The other slots have the default values.

The template emit function fills in the Vop, Costs, Cost-Function, SC-Restriction and Preference slots, and links together the Next-Ref chain as appropriate.

\sectionLifetime model

\#| Note in doc that the same TN may not be used as both a more operand and as any other operand to the same VOP, to simplify more operand LTN number coalescing. |\#

It seems we need a fairly elaborate model for intra-VOP conflicts in order to allocate temporaries without introducing spurious conflicts. Consider the important case of a VOP such as a miscop that must have operands in certain registers. We allocate a wired temporary, create a local preference for the corresponding operand, and move to (or from) the temporary. If all temporaries conflict with all arguments, the result will be correct, but arguments could never be packed in the actual passing register. If temporaries didn't conflict with any arguments, then the temporary for an earlier argument might get packed in the same location as the operand for a later argument; loading would then destroy an argument before it was read.

A temporary's intra-VOP lifetime is represented by the times at which its life starts and ends. There are various instants during the evaluation that start and end VOP lifetimes. Two TNs conflict if the live intervals overlap. Lifetimes are open intervals: if one TN's lifetime begins at a point where another's ends, then the TNs don't conflict.

The times within a VOP are the following:

:Load This is the beginning of the argument's lives, as far as intra-vop conflicts are concerned. If load-TNs are allocated, then this is the beginning of their lives.

(:Argument $<$n$>$) The point at which the N'th argument is read for the last time (by this VOP). If the argument is dead after this VOP, then the argument becomes dead at this time, and may be reused as a temporary or result load-TN.

(:Eval $<$n$>$) The N'th evaluation step. There may be any number of evaluation steps, but it is unlikely that more than two are needed.

(:Result $<$n$>$) The point at which the N'th result is first written into. This is the point at which that result becomes live.

:Save Similar to :Load, but marks the end of time. This is the point at which result load-TNs are stored back to the actual location.

In any of the list-style time specifications, the keyword by itself stands for the first such time, i.e. \beginverbatim :argument <==> (:argument 0) \endverbatim

Note that argument/result read/write times don't actually have to be in the order specified, but they must *appear* to happen in that order as far as conflict analysis is concerned. For example, the arguments can be read in any order as long as no TN is written that has a life beginning at or after (:Argument $<$n$>$), where N is the number of an argument whose reading was postponed.

[\#\#\# (???)

We probably also want some syntactic sugar in Define-VOP for automatically moving operands to/from explicitly allocated temporaries so that this kind of thing is somewhat easy. There isn't really any reason to consider the temporary to be a load-TN, but we want to compute costs as though it was and want to use the same operand loading routines.

We also might consider allowing the lifetime of an argument/result to be extended forward/backward. This would in many cases eliminate the need for temporaries when operands are read/written out of order.]

\sectionVOP Cost model

Note that in this model, if an operand has no restrictions, it has no cost. This makes sense, since the purpose of the cost is to indicate the relative value of packing in different SCs. If the operand isn't required to be in a good SC (i.e. a register), then we might as well leave it in memory. The SC restriction mechanism can be used even when doing a move into the SC is too complex to be generated automatically (perhaps requiring temporary registers), since Define-VOP allows operand loading to be done explicitly.

\sectionEfficiency notes

In addition to being used to tell whether a particular unsafe template might get emitted, we can also use it to give better efficiency notes: – We can say what is wrong with the call types, rather than just saying we failed to open-code. – We can tell whether any of the "better" templates could possibly apply, i.e. is the inapplicability of a template because of inadequate type information or because the type is just plain wrong. We don't want to flame people when a template that couldn't possibly match doesn't match, e.g. complaining that we can't use fixnum+ when the arguments are known to be floats.

This is how we give better efficiency notes:

The Template-Note is a short noun-like string without capitalization or punctuation that describes what the template "does", i.e. we say "Unable to do ~A, doing ~A instead."

The Cost is moved from the Vop-Info to the Template structure, and is used to determine the "goodness" of possibly applicable templates. [Could flush

Template/Vop-Info distinction] The cost is used to choose the best applicable template to emit, and also to determine what better templates we might have been able to use.

A template is possibly applicable if there is an intersection between all of the arg/result types and the corresponding arg/result restrictions, i.e. the template is not clearly impossible: more declarations might allow it to be emitted.

\chapterAssembler Retargeting

\chapterWriting Assembly Code

VOP writers expect: \beginLentry \item[MOVE] You write when you port the assembler.) \item[EMIT-LABEL] Assembler interface like INST. Takes a label you made and says "stick it here." \item[GEN-LABEL] Returns a new label suitable for use with EMIT-LABEL exactly once and for referencing as often as necessary. \item[INST] Recognizes and dispatches to instructions you defined for assembler. \item[ALIGN] This takes the number of zero bits you want in the low end of the address of the next instruction. \item[ASSEMBLE] \item[ASSEMBLE-ELSEWHERE] Get ready for assembling stuff. Takes a VOP and arbitrary PROGN-style body. Wrap these around instruction emission code announcing the first pass of our assembler. \item[CURRENT-NFP-TN] This returns a TN for the NFP if the caller uses the number stack, or nil. \item[SB-ALLOCATED-SIZE] This returns the size of some storage base used by the currently compiling component. \item[...] \endLentry ;;; ;;; VOP idioms ;;;

\beginLentry \item[STORE-STACK-TN] \item[LOAD-STACK-TN] These move a value from a register to the control stack, or from the control stack to a register. They take care of checking the TN types, modifying offsets according to the address units per word, etc. \endLentry

\chapterRequired VOPS

Note: the move VOP cannot have any wired temps. (Move-Argument also?) This is so we can move stuff into wired TNs without stepping on our toes.

We create set closure variables using the Value-Cell VOP, which takes a value and returns a value cell containing the value. We can basically use this instead of a Move VOP when initializing the variable. Value-Cell-Set and Value-Cell-Ref are used to access the value cell. We can have a special effect for value cells so that value cells references can be discovered to be common subexpressions or loop invariants.

Represent unknown-values continuations as (start, count). Unknown values continuations are always outside of the current frame (on stack top). Within a function, we always set up and receive values in the standard passing locations. If we receive stack values, then we must BLT them down to the start of our frame, filling in any unsupplied values. If we generate unknown values (i.e. PUSH-VALUES), then we set the values up in the standard locations, then BLT them to stack top. When doing a tail-return of MVs, we just set them up in the standard locations and decrement SP: no BLT is necessary.

Unknown argument call (MV-CALL) takes its arguments on stack top (is given a base pointer). If not a tail call, then we just set the arg pointer to the base pointer

and call. If a tail call, we must BLT the arguments down to the beginning of the current frame.

Implement more args by BLT'ing the more args *on top* of the current frame. This solves two problems: \beginitemize \item Any register more arguments can be made uniformly accessibly by copying them into memory. [We can't store the registers in place, since the beginning of the frame gets double use for storing the old-cont, return-pc and env.] \item It solves the deallocation problem: the arguments will be deallocated when the frame is returned from or a tail full call is done out of it. So keyword args will be properly tail-recursive without any special mechanism for squeezing out the more arg once the parsing is done. Note that a tail local call won't blast the more arg, since in local call the callee just takes the frame it is given (in this case containing the more arg). \enditemize

More args in local call??? Perhaps we should not attempt local call conversion in this case. We already special-case keyword args in local call. It seems that the main importance of more args is primarily related to full call: it is used for defining various kinds of frobs that need to take arbitrary arguments: \beginitemize \item Keyword arguments \item Interpreter stubs \item "Pass through" applications such as dispatch functions \enditemize Given the marginal importance of more args in local call, it seems unworth going to any implementation difficulty. In fact, it seems that it would cause complications both at the VMR level and also in the VM definition. This being the case, we should flush it.

\sectionFunction Call

\subsectionRegisters and frame format

These registers are used in function call and return:

A0..A\it n In full call, the first three arguments. In unknown values return, the first three return values.

CFP The current frame pointer. In full call, this initially points to a partial frame large enough to hold the passed stack arguments (zero-length if none).

CSP The current control stack top pointer.

OCFP In full call, the passing location for the frame to return to.

In unknown-values return of other than one value, the pointer to returned stack values. In such a return, OCFP is always initialized to point to the frame returned from, even when no stack values are returned. This allows OCFP to be used to restore CSP.

LRA In full call, the passing location for the return PC.

NARGS In full call, the number of arguments passed. In unknown-values return of other than one value, the number of values returned.

\subsectionFull call

What is our usage of CFP, OCFP and CSP?

It is an invariant that CSP always points after any useful information so that at any time an interrupt can come and allocate stuff in the stack.

TR call is also a constraint: we can't deallocate the caller's frame before the call, since it holds the stack arguments for the call.

What we do is have the caller set up CFP, and have the callee set CSP to CFP plus the frame size. The caller leaves CSP alone: the callee is the one who does any necessary stack deallocation.

In a TR call, we don't do anything: CFP is left as CFP, and CSP points to the end of the frame, keeping the stack arguments from being trashed.

In a normal call, CFP is set to CSP, causing the callee's frame to be allocated after the current frame.

\subsectionUnknown values return

The unknown values return convention is always used in full call, and is used in local call when the compiler either can't prove that a fixed number of values are returned, or decides not to use the fixed values convention to allow tail-recursive XEP calls.

The unknown-values return convention has variants: single value and variable values. We make this distinction to optimize the important case of a returner who knows exactly one value is being returned. Note that it is possible to return a single value using the variable-values convention, but it is less efficient.

We indicate single-value return by returning at the return-pc+4; variable value return is indicated by returning at the return PC.

Single-value return makes only the following guarantees: A0 holds the value returned. CSP has been reset: there is no garbage on the stack.

In variable value return, more information is passed back: A0..A2 hold the first three return values. If fewer than three values are returned, then the unused registers are initialized to NIL.

OCFP points to the frame returned from. Note that because of our tail-recursive implementation of call, the frame receiving the values is always immediately under the frame returning the values. This means that we can use OCFP to index the values when we access them, and to restore CSP when we want to discard them.

NARGS holds the number of values returned.

CSP is always (+ OCFP (* NARGS 4)), i.e. there is room on the stack allocated for all returned values, even if they are all actually passed in registers.

\subsectionExternal Entry Points

Things that need to be done on XEP entry: 1] Allocate frame 2] Move more arg above the frame, saving context 3] Set up env, saving closure pointer if closure 4] Move arguments from closure to local home Move old-cont and return-pc to the save locations 5] Argument count checking and dispatching

XEP VOPs: \beginverbatim Allocate-Frame Copy-More-Arg <nargs-tn> 'fixed in a3 => <context>, <count> Setup-Environment Setup-Closure-Environment => <closure> Verify-Argument-Count <nargs-tn> 'count for fixed-arg lambdas Argument-Count-Error <nargs-tn> Drop-thru on hairy arg dispatching Use fast-if-=/fixnum and fast-if-</fixnum for dispatching. \endverbatim

Closure vops: \beginverbatim make-closure <fun entry> <slot count> => <closure> closure-init <closure> <values> 'slot \endverbatim

Things that need to be done on all function entry: \beginitemize \item Move arguments to the variable home (consing value cells as necessary) \item Move en-

vironment values to the local home \item Move old-cont and return-pc to the save locations \enditemize

\sectionCalls

Calling VOP's are a cross product of the following sets (with some members missing): Return values multiple (all values) fixed (calling with unknown values conventions, wanting a certain number.) known (only in local call where caller/callee agree on number of values.) tail (doesn't return but does tail call) What function local named (going through symbol, like full but stash fun name for error sys) full (have a function) Args fixed (number of args are known at compile-time) variable (MULTIPLE-VALUE-CALL and APPLY)

Note on all jumps for calls and returns that we want to put some instruction in the jump's delay slot(s).

Register usage at the time of the call:

LEXENV This holds the lexical environment to use during the call if it's a closure, and it is undefined otherwise.

CNAME This holds the symbol for a named call and garbage otherwise.

OCFP This holds the frame pointer, which the system restores upon return. The callee saves this if necessary; this is passed as a pseudo-argument.

A0 ... An These holds the first n+1 arguments.

NARGS This holds the number of arguments, as a fixnum.

LRA This holds the lisp-return-address object which indicates where to return. For a tail call, this retains its current value. The callee saves this if necessary; this is passed as a pseudo-argument.

CODE This holds the function object being called.

CSP The caller ignores this. The callee sets it as necessary based on CFP.

CFP This holds the callee's frame pointer. Caller sets this to the new frame pointer, which it remembered when it started computing arguments; this is CSP if there were no stack arguments. For a tail call CFP retains its current value.

NSP The system uses this within a single function. A function using NSP must allocate and deallocate before returning or making a tail call.

Register usage at the time of the return for single value return, which goes with the unknown-values convention the caller used.

A0 This holds the value.

CODE This holds the lisp-return-address at which the system continues executing.

CSP This holds the CFP. That is, the stack is guaranteed to be clean, and there is no code at the return site to adjust the CSP.

CFP This holds the OCFP.

Additional register usage for multiple value return:

NARGS This holds the number of values returned.

A0 ... An These holds the first n+1 values, or NIL if there are less than n+1 values.

CSP Returner stores CSP to hold its CFP + NARGS * \verb+<address units per word>+

OCFP Returner stores this as its CFP, so the returnee has a handle on either the start of the returned values on the stack.

ALLOCATE FULL CALL FRAME.

If the number of call arguments (passed to the VOP as an info argument) indicates that there are stack arguments, then it makes some callee frame for arguments: \beginverbatim VOP-result <- CSP CSP <- CSP + value of VOP info arg times address units per word. \endverbatim

In a call sequence, move some arguments to the right places.

There's a variety of MOVE-ARGUMENT VOP's.

FULL CALL VOP'S (variations determined by whether it's named, it's a tail call, there is a variable arg count, etc.) \beginverbatim if variable number of arguments NARGS <- (CSP - value of VOP argument) shift right by address units per word. A0...An <- values off of VOP argument (just fill them all) else NARGS <- value of VOP info argument (always a constant)

if tail call OCFP <- value from VOP argument LRA <- value from VOP argument CFP stays the same since we reuse the frame NSP <- NFP else OCFP <- CFP LRA <- compute LRA by adding an assemble-time determined constant to CODE. CFP <- new frame pointer (remembered when starting to compute args) This is CSP if no stack args. when (current-nfp-tn VOP-self-pointer) stack-temp <- NFP

if named CNAME <- function symbol name the-fun <- function object out of symbol

LEXENV <- the-fun (from previous line or VOP argument) CODE <- function-entry (the first word after the-fun) LIP <- calc first instruction addr (CODE + constant-offset) jump and run off temp

<emit Lisp return address data-block> <default and move return values OR receive return values> when (current-nfp-tn VOP-self-pointer) NFP <- stack-temp \endverbatim Callee:

\beginverbatim XEP-ALLOCATE-FRAME emit function header (maybe initializes offset back to component start, but other pointers are set up at load-time. Pads to dual-word boundary.) CSP <- CFP + compile-time determined constant (frame size) if the function uses the number stack NFP <- NSP NSP <- NSP + compile-time determined constant (number stack frame size) \endverbatim

\beginverbatim SETUP-ENVIRONMENT (either use this or the next one)

CODE <- CODE - assembler-time determined offset from function-entry back to the code data-block address. \endverbatim

\beginverbatim SETUP-CLOSURE-ENVIRONMENT (either use this or the previous one) After this the CLOSURE-REF VOP can reference closure variables.

VOP-result <- LEXENV CODE <- CODE - assembler-time determined offset from function-entry back to the code data-block address. \endverbatim

Return VOP's RETURN and RETURN-MULTIPLE are for the unknown-values return convention. For some previous caller this is either it wants n values (and it doesn't know how many are coming), or it wants all the values returned (and it doesn't know how many are coming).

RETURN (known fixed number of values, used with the unknown-values convention in the caller.) When compiler invokes VOP, all values are already where they should be; just get back to caller.

\beginverbatim when (current-nfp-tn VOP-self-pointer) ;; The number stack grows down in memory. NSP <- NFP + number stack frame size for calls within the currently compiling component times address units per word CODE <- value of VOP argument with LRA if VOP info arg is 1 (number of values we know we're returning) CSP <- CFP LIP <- calc target addr (CODE + skip over LRA header word + skip over address units per branch) (The branch is in the caller to skip down to the MV code.) else NARGS <- value of VOP info arg nil out unused arg regs OCFP <- CFP (This indicates the start of return values on the stack, but you leave space for those in registers for convenience.) CSP <- CFP + NARGS * address-units-per-word LIP <- calc target addr (CODE + skip over LRA header word) CFP <- value of VOP argument with OCFP jump and run off LIP \endverbatim

RETURN-MULTIPLE (unknown number of values, used with the unknown-values convention in the caller.) When compiler invokes VOP, it gets TN's representing a pointer to the values on the stack and how many values were computed.

\beginverbatim when (current-nfp-tn VOP-self-pointer) ;; The number stack grows down in memory. NSP <- NFP + number stack frame size for calls within the currently compiling component times address units per word NARGS <- value of VOP argument copy the args to the beginning of the current (returner's) frame. Actually some go into the argument registers. When putting the rest at the beginning of the frame, leave room for those in the argument registers. CSP <- CFP + NARGS * address-units-per-word nil out unused arg regs OCFP <- CFP (This indicates the start of return values on the stack, but you leave space for those in registers for convenience.) CFP <- value of VOP argument with OCFP CODE <- value of VOP argument with LRA LIP <- calc target addr (CODE + skip over LRA header word) jump and run off LIP \endverbatim

Returnee The call VOP's call DEFAULT-UNKNOWN-VALUES or RECEIVE-UNKNOWN-VALUES after spitting out transfer control to get stuff from the returner.

DEFAULT-UNKNOWN-VALUES (We know what we want and we got something.) If returnee wants one value, it never does anything to deal with a shortage of return values. However, if start at PC, then it has to adjust the stack pointer to dump extra values (move OCFP into CSP). If it starts at PC+N, then it just goes along with the "want one value, got it" case. If the returnee wants multiple values, and there's a shortage of return values, there are two cases to handle. One, if the returnee wants fewer values than there are return registers, and we start at PC+N, then it fills in return registers \verb|A1..A<desired values necessary>|; if we start at PC, then the returnee is fine since the returning conventions have filled in the unused return registers with nil, but the returnee must adjust the stack pointer to dump possible stack return values (move OCFP to CSP). Two, if the returnee wants more values than the number of return registers, and it starts at PC+N (got one value), then it sets up returnee state as if an unknown number of values came back: \beginverbatim A0 has the one value A1..An get nil NARGS gets 1 OCFP

gets CSP, so general code described below can move OCFP into CSP If we start at PC, then branch down to the general "got k values, wanted n" code which takes care of the following issues: If k < n, fill in stack return values of nil for shortage of return values and move OCFP into CSP If k >= n, move OCFP into CSP This also restores CODE from LRA by subtracting an assemble-time constant. \endverbatim

RECEIVE-UKNOWN-VALUES (I want whatever I get.) We want these at the end of our frame. When the returnee starts at PC, it moves the return value registers to OCFP..OCFP[An] ignoring where the end of the stack is and whether all the return value registers had values. The returner left room on the stack before the stack return values for the register return values. When the returnee starts at PC+N, bump CSP by 1 and copy A0 there. This also restores CODE from LRA by subtracting an assemble-time constant.

Local call

There are three flavors: 1] KNOWN-CALL-LOCAL Uses known call convention where caller and callee agree where all the values are, and there's a fixed number of return values. 2] CALL-LOCAL Uses the unknown-values convention, but we expect a particular number of values in return. 3] MULTIPLE-CALL-LOCAL Uses the unknown-values convention, but we want all values returned.

ALLOCATE-FRAME

If the number of call arguments (passed to the VOP as an info argument) indicates that there are stack arguments, then it makes some callee frame for arguments: \beginverbatim VOP-result1 <- CSP CSP <- CSP + control stack frame size for calls within the currently compiling component times address units per word. when (callee-nfp-tn <VOP info arg holding callee>) ;; The number stack grows down. ;; May have to round to dual-word boundary if machines C calling ;; conventions demand this. NSP <- NSP - number stack frame size for calls within the currently compiling component times address units per word VOP-result2 <- NSP \endverbatim KNOWN-CALL-LOCAL, CALL-LOCAL, MULTIPLE-CALL-LOCAL KNOWN-CALL-LOCAL has no need to affect CODE since CODE is the same for the caller/returnee and the returner. This uses KNOWN-RETURN. With CALL-LOCAL and MULTIPLE-CALL-LOCAL, the caller/returnee must fixup CODE since the callee may do a tail full call. This happens in the code emitted by DEFAULT-UNKNOWN-VALUES and RECEIVE-UNKNOWN-VALUES. We use these return conventions since we don't know what kind of values the returner will give us. This could happen due to a tail full call to an unknown function, or because the callee had different return points that returned various numbers of values.

\beginverbatim when (current-nfp-tn VOP-self-pointer) ;Get VOP self-pointer with ;DEFINE-VOP switch :vop-var. stack-temp <- NFP CFP <- value of VOP arg when (callee-nfp-tn <VOP info arg holding callee>) <where-callee-wants-NFP-tn> <- value of VOP arg <where-callee-wants-LRA-tn> <- compute LRA by adding an assemble-time determined constant to CODE. jump and run off VOP info arg holding start instruction for callee

<emit Lisp return address data-block> <case call convention known: do nothing call: default and move return values multiple: receive return values > when (current-nfp-tn VOP-self-pointer) NFP <- stack-temp \endverbatim

KNOWN-RETURN \beginverbatim CSP <- CFP when (current-nfp-tn VOP-self-pointer) ;; number stack grows down in memory. NSP <- NFP + number stack frame size for calls within the currently compiling component times address units per word LIP <- calc target addr (value of VOP arg + skip over LRA header word) CFP <- value of VOP arg jump and run off LIP

\endverbatim

\chapterStandard Primitives

\chapterCustomizing VMR Conversion

Another way in which different implementations differ is in the relative cost of operations. On machines without an integer multiply instruction, it may be desirable to convert multiplication by a constant into shifts and adds, while this is surely a bad idea on machines with hardware support for multiplication. Part of the tuning process for an implementation will be adding implementation dependent transforms and disabling undesirable standard transforms.

When practical, ICR transforms should be used instead of VMR generators, since transforms are more portable and less error-prone. Note that the Lisp code need not be implementation independent: it may contain all sorts of sub-primitives and similar stuff. Generally a function should be implemented using a transform instead of a VMR translator unless it cannot be implemented as a transform due to being totally evil or it is just as easy to implement as a translator because it is so simple.

\sectionConstant Operands

If the code emitted for a VOP when an argument is constant is very different than the non-constant case, then it may be desirable to special-case the operation in VMR conversion by emitting different VOPs. An example would be if SVREF is only open-coded when the index is a constant, and turns into a miscop call otherwise. We wouldn't want constant references to spuriously allocate all the miscop linkage registers on the off chance that the offset might not be constant. See the :constant feature of VOP primitive type restrictions.

\sectionSupporting Multiple Hardware Configurations

A winning way to change emitted code depending on the hardware configuration, i.e. what FPA is present is to do this using primitive types. Note that the Primitive-Type function is VM supplied, and can look at any appropriate hardware configuration switches. Short-Float can become 6881-Short-Float, AFPA-Short-Float, etc. There would be separate SBs and SCs for the registers of each kind of FP hardware, with each hardware-specific primitive type using the appropriate float register SC. Then the hardware specific templates would provide AFPA-Short-Float as the argument type restriction.

Primitive type changes:

The primitive-type structure is given a new \%Type slot, which is the CType structure that is equivalent to this type. There is also a Guard slot, which, if true is a function that control whether this primitive type is allowed (due to hardware configuration, etc.)

We add new :Type and :Guard keywords to Def-Primitive-Type. Type is the type specifier that is equivalent (default to the primitive-type name), and Guard

is an expression evaluated in the null environment that controls whether this type applies (default to none, i.e. constant T).

The Primitive-Type-Type function returns the Lisp CType corresponding to a primitive type. This is the \%Type unless there is a guard that returns false, in which case it is the empty type (i.e. NIL).

[But this doesn't do what we want it to do, since we will compute the function type for a template at load-time, so they will correspond to whatever configuration was in effect then. Maybe we don't want to dick with guards here (if at all). I guess we can defer this issue until we actually support different FP configurations. But it would seem pretty losing to separately flame about all the different FP configurations that could be used to open-code + whenever we are forced to closed-code +.

If we separately report each better possibly applicable template that we couldn't use, then it would be reasonable to report any conditional template allowed by the configuration.

But it would probably also be good to give some sort of hint that perhaps it would be a good time to make sure you understand how to tell the compiler to compile for a particular configuration. Perhaps if there is a template that applies *but for the guard*, then we could give a note. This way, if someone thinks they are being efficient by throwing in lots of declarations, we can let them know that they may have to do more.

I guess the guard should be associated with the template rather than the primitive type. This would allow LTN and friends to easily tell whether a template applies in this configuration. It is also probably more natural for some sorts of things: with some hardware variants, it may be that the SBs and representations (SCs) are really the same, but there are some different allowed operations. In this case, we could easily conditionalize VOPs without the increased complexity due to bogus SCs. If there are different storage resources, then we would conditionalize Primitive-Type as well.

\sectionSpecial-case VMR convert methods

(defun continuation-tn (cont \&optional (check-p t)) ...) Return the TN which holds Continuation's first result value. In general this may emit code to load the value into a TN. If Check-P is true, then when policy indicates, code should be emitted to check that the value satisfies the continuation asserted type.

(defun result-tn (cont) ...) Return the TN that Continuation's first value is delivered in. In general, may emit code to default any additional values to NIL.

(defun result-tns (cont n) ...) Similar to Result-TN, except that it returns a list of N result TNs, one for each of the first N values.

Nearly all open-coded functions should be handled using standard template selection. Some (all?) exceptions: \beginitemize \item List, List* and Vector take arbitrary numbers of arguments. Could implement Vector as a source transform. Could even do List in a transform if we explicitly represent the stack args using \%More-Args or something. \item \%Typep varies a lot depending on the type specifier. We don't want to transform it, since we want \%Typep as a canonical form so that we can do type optimizations. \item Apply is weird. \item Funny functions

emitted by the compiler: \%Listify-Rest-Args, Arg, \%More-Args, \%Special-Bind, \%Catch, \%Unknown-Values (?), \%Unwind-Protect, \%Unwind, \%\%Primitive. \enditemize % -*- Dictionary: design; Package: C -*- \chapterDebugger Two classes of errors are handled by the Lisp debugger. These are synchronous errors caused by something erring in program code and asynchronous errors caused by some external context of execution (clock interrupts, control-c interrupts). Asynchronous errors can often be postponed if they are delivered at an inconvenient time.

Synchronous errors are frequently handled by directly invoking the debugger. However, there are several places where the strategy of jumping into the debugger is not used. In those situations the compiler emits a stylized breakpoint; a breakpoint instruction (usually an INT3) followed by several bytes of argument data. This will cause a trip through the operating system and ultimately the invocation of the C-level SIGTRAP handler which, in turn, interprets the argument bytes following the breakpoint and dispatches to the correct handler. There is a switch statement in "sigtrap_handler" which gives the whole story on what types of errors rely on this mechanism. The most commonly invoked handler is probably "interrupt_internal_error" as it fields such common exceptions as the use of unbound symbols. To familiarize with the context these traps are created in, one can disassemble just about any function and look at the bottom of the disassembly for blocks of error handling code. There will often be "BREAK 10" opcodes followed by several "BYTE" opcodes with the meaning of the arguments in neatly decoded form off in the right-hand column.

The other types of synchronous errors are those errors delivered by the operating system such as FPU traps and SIGSEGVs. The invocation of those signals should be funneled through a C-level trampoline which makes a callback into Lisp passing all of the signal handler arguments. That code is pretty straight forward and the "interrupt_handle_now" function is pretty much where all of the runtime logic is localized.

Handling asynchronous errors and deferred asynchronous errors is a bit more involved\ldots

\sectionTracing and Breakpoints \labelsec:trace-and-breakpoints

Here are a few notes on how tracing of compiled code works.

When a function is traced, a breakpoint instruction is placed at the start of the function, replacing the instruction that was there. (This is a \verb+:function-start+ breakpoint.) (This appears to be one instruction after the no-arg parsing entry point.) The breakpoint instruction is, of course, architecture-specific, but it must signal a \verb+trap_Breakpoint+ trap.

When the code is run, the breakpoint instruction is executed causing a trap. The trap handler runs \verb+HANDLE-BREAKPOINT+ to process it. After doing the appropriate processing, we now need to continue. Of course, since the real instruction has been replaced, we to run the original instruction. This is done by now inserting a \emphnew breakpoint after the original breakpoint. This breakpoint must be of the type \verb+trap_AfterBreakpoint+. The original instruction is restored and execution continues from there. Then the \verb+trap_AfterBreakpoint+

instruction gets executed. The handler for this puts back the original breakpoint, thereby preserving the breakpoint. Then we replace the AfterBreakpoint with the original instruction and continue from there.

That's all pretty straightforward in concept.

When tracing, additional information is needed. Breakpoints have the ability to run arbitrary lisp code to process the breakpoint. Tracing uses this feature.

When this breakpoint is reached, \verb+HANDLE-BREAKPOINT+ runs the breakpoint hook function. This function figures out where this function would return to and creates a new return area and replaces the original return address with this new address. Thus, when the function returns, it returns to this new location instead of the original.

This new return address is a specially created bogus LRA object. It is a code-component whose body consists of a code template copied from an assembly routine into the body. The assembly routine is the code in \verb+function_end_breakpoint_guts+. This bogus LRA object stores the real LRA for the function, and also an indication if the known-return convention is used for this function.

The bogus LRA object contains a function-end breakpoint (\verb+trap_FunctionEndBreakpoint+). When it's executed the trap handler handles this breakpoint. It figures out where this trap come from and calls \verb+HANDLE-BREAKPOINT+ to handle it. \verb+HANDLE-BREAKPOINT+ returns and the trap handler arranges it so that this bogus LRA returns to the real LRA.

Thus, we can do something when a Lisp function returns, like printing out the return value for the function for tracing.

There are lots of internal details left out here, but gives a short overview of how this works. For more info, look at \verb+code/debug-int.lisp+ and \verb+lisp/breakpoint.c+, and, of course, the various \verb+<foo>-arch.c+ files.

\chapterDebugger Information \indexdebugger information \labeldebug-info

Although the compiler's great freedom in choice of function call conventions and variable representations has major efficiency advantages, it also has unfortunate consequences for the debugger. The debug information that we need is even more elaborate than for conventional "compiled" languages, since we cannot even do a simple backtrace without some debug information. However, once having gone this far, it is not that difficult to go the extra distance, and provide full source level debugging of compiled code.

Full debug information has a substantial space penalty, so we allow different levels of debug information to be specified. In the extreme case, we can totally omit debug information.

\sectionThe Debug-Info Structure \indexdebug-info structure

The Debug-Info structure directly represents information about the source code, and points to other structures that describe the layout of run-time data structures.

Make some sort of minimal debug-info format that would support at least the common cases of level 1 (since that is what we would release), and perhaps level 0. Actually, it seems it wouldn't be hard to crunch nearly all of the debug-function

structure and debug-info function map into a single byte-vector. We could have an uncrunch function that restored the current format. This would be used by the debugger, and also could be used by purify to delete parts of the debug-info even when the compiler dumps it in crunched form. [Note that this isn't terribly important if purify is smart about debug-info...]

Compiled source map representation:

[\#\#\# store in debug-function PC at which env is properly initialized, i.e. args (and return-pc, etc.) in internal locations. This is where a :function-start breakpoint would break.]

[\#\#\# Note that that we can easily cache the form-number => source-path or form-number => form translation using a vector indexed by form numbers that we build during a walk.]

Instead of using source paths in the debug-info, use "form numbers". The form number of a form is the number of forms that we walk to reach that form when doing a pre-order walk of the source form. [Might want to use a post-order walk, as that would more closely approximate evaluation order.]

We probably want to continue using source-paths in the compiler, since they are quick to compute and to get you to a particular form. [\#\#\# But actually, I guess we don't have to precompute the source paths and annotate nodes with them: instead we could annotate the nodes with the actual original source form. Then if we wanted to find the location of that form, we could walk the root source form, looking that original form. But we might still need to enter all the forms in a hashtable so that we can tell during IR1 conversion that a given form appeared in the original source.]

Note that form numbers have an interesting property: it is quite efficient to determine whether an arbitrary form is a subform of some other form, since the form number of B will be \verb+>+ than A's number and \verb+<+ A's next sibling's number iff B is a subform of A.

This should be quite useful for doing the \verb|source=>pc| mapping in the debugger, since that problem reduces to finding the subset of the known locations that are for subforms of the specified form.

Assume a byte vector with a standard variable-length integer format, something like this:

\beginverbatim 0..253 => the integer 254 => read next two bytes for integer 255 => read next four bytes for integer \endverbatim

Then a compiled debug block is just a sequence of variable-length integers in a particular order, something like this:

\beginverbatim number of successors ...offsets of each successor in the function's blocks vector... first PC [offset of first top-level form (in forms) (only if not component default)] form number of first source form first live mask (length in bytes determined by number of VARIABLES) ...more <PC, top-level form offset, form-number, live-set> tuples... \endverbatim

We determine the number of locations recorded in a block by finding the start of the next compiled debug block in the blocks vector.

[\#\#\# Actually, only need 2 bits for number of successors 0,1,2. We might want to use other bits in the first byte to indicate the kind of location.] [\#\#\# We could support local packing by having a general concept of "alternate locations" instead of just regular and save locations. The location would have a bit indicating that there are alternate locations, in which case we read the number of alternate locations and then that many more SC-OFFSETs. In the debug-block, we would have a second bit mask with bits set for TNs that are in an alternate location. We then read a number for each such TN, with the value being interpreted as an index into the Location's alternate locations.]

It looks like using structures for the compiled-location-info is too bulky. Instead we need some packed binary representation.

First, let's represent an SC/offset pair with an "SC-Offset", which is an integer with the SC in the low 5 bits and the offset in the remaining bits: \beginverbatim —————————————————————————- | Offset (as many bits as necessary) | SC (5 bits) | ————————————————————————- \endverbatim Probably the result should be constrained to fit in a fixnum, since it will be more efficient and gives more than enough possible offsets.

We can then represent a compiled location like this: \beginverbatim single byte of boolean flags: uninterned name packaged name environment-live has distinct save location has ID (name not unique in this fun) name length in bytes (as var-length integer) ...name bytes... [if packaged, var-length integer that is package name length] ...package name bytes...] [If has ID, ID as var-length integer] SC-Offset of primary location (as var-length integer) [If has save SC, SC-Offset of save location (as var-length integer)] \endverbatim

But for a whizzy breakpoint facility, we would need a good \verb+source=>code+ map. Dumping a complete \verb+code=>source map+ might be as good a way as any to represent this, due to the one-to-many relationship between source and code locations.

We might be able to get away with just storing the source locations for the beginnings of blocks and maintaining a mapping from code ranges to blocks. This would be fine both for the profiler and for the "where am I running now" indication. Users might also be convinced that it was most interesting to break at block starts, but I don't really know how easily people could develop an understanding of basic blocks.

It could also be a bit tricky to map an arbitrary user-designated source location to some "closest" source location actually in the debug info. This problem probably exists to some degree even with a full source map, since some forms will never appear as the source of any node. It seems you might have to negotiate with the user. He would mouse something, and then you would highlight some source form that has a common prefix (i.e. is a prefix of the user path, or vice-versa.) If they aren't happy with the result, they could try something else. In some cases, the designated path might be a prefix of several paths. This ambiguity might be resolved by picking the shortest path or letting the user choose.

At the primitive level, I guess what this means is that the structure of source locations (i.e. source paths) must be known, and the \verb+source=>code+ oper-

ation should return a list of \verb+<source,code>+ pairs, rather than just a list of code locations. This allows the debugger to resolve the ambiguity however it wants.

I guess the formal definition of which source paths we would return is: \beginquote All source paths in the debug info that have a maximal common prefix with the specified path. i.e. if several paths have the complete specified path as a prefix, we return them all. Otherwise, all paths with an equally large common prefix are returned: if the path with the most in common matches only the first three elements, then we return all paths that match in the first three elements. As a degenerate case (which probably shouldn't happen), if there is no path with anything in common, then we return *all* of the paths. \endquote

In the DEBUG-SOURCE structure we may ultimately want a vector of the start positions of each source form, since that would make it easier for the debugger to locate the source. It could just open the file, FILE-POSITION to the form, do a READ, then loop down the source path. Of course, it could read each form starting from the beginning, but that might be too slow.

Do XEPs really need Debug-Functions? The only time that we will commonly end up in the debugger on an XEP is when an argument type check fails. But I suppose it would be nice to be able to print the arguments passed...

Note that assembler-level code motion such as pipeline reorganization can cause problems with our PC maps. The assembler needs to know that debug info markers are different from real labels anyway, so I suppose it could inhibit motion across debug markers conditional on policy. It seems unworthwhile to remember the node for each individual instruction.

For tracing block-compiled calls: \beginverbatim Info about return value passing locations? Info about where all the returns are? \endverbatim

We definitely need the return-value passing locations for debug-return. The question is what the interface should be. We don't really want to have a visible debug-function-return-locations operation, since there are various value passing conventions, and we want to paper over the differences.

Probably should be a compiler option to initialize stack frame to a special uninitialized object (some random immediate type). This would aid debugging, and would also help GC problems. For the latter reason especially, this should be locally-turn-onable (off of policy? the new debug-info quality?).

What about the interface between the evaluator and the debugger? (i.e. what happens on an error, etc.) Compiler error handling should be integrated with run-time error handling. Ideally the error messages should look the same. Practically, in some cases the run-time errors will have less information. But the error should look the same to the debugger (or at least similar).

\subsectionDebugger Interface

How does the debugger interface to the "evaluator" (where the evaluator means all of native code, byte-code and interpreted IR1)? It seems that it would be much more straightforward to have a consistent user interface to debugging all code representations if there was a uniform debugger interface to the underlying stuff, and vice-versa.

Of course, some operations might not be supported by some representations, etc. For example, fine-control stepping might not be available in native code. In other cases, we might reduce an operation to the lowest common denominator, for example fetching lexical variables by string and admitting the possibility of ambiguous matches. [Actually, it would probably be a good idea to store the package if we are going to allow variables to be closed over.]

Some objects we would need: \beginverbatim Location: The constant information about the place where a value is stored, everything but which particular frame it is in. Operations: location name, type, etc. location-value frame location (setf'able) monitor-location location function Function is called whenever location is set with the location, frame and old value. If active values aren't supported, then we dummy the effect using breakpoints, in which case the change won't be noticed until the end of the block (and intermediate changes will be lost.) debug info: All the debug information for a component. Frame: frame-changed-locations frame => location* Return a list of the locations in frame that were changed since the last time this function was called. Or something. This is for displaying interesting state changes at breakpoints. save-frame-state frame => frame-state restore-frame-state frame frame-state These operations allow the debugger to back up evaluation, modulo side-effects and non-local control transfers. This copies and restores all variables, temporaries, etc, local to the frame, and also the current PC and dynamic environment (current catch, etc.)

At the time of the save, the frame must be for the running function (not waiting for a call to return.) When we restore, the frame becomes current again, effectively exiting from any frames on top. (Of course, frame must not already be exited.)

Thread: Representation of which stack to use, etc. Block: What successors the block has, what calls there are in the block. (Don't need to know where calls are as long as we know called function, since can breakpoint at the function.) Whether code in this block is wildly out of order due to being the result of loop-invariant optimization, etc. Operations: block-successors block => code-location* block-forms block => (source-location code-location)* Return the corresponding source locations and code locations for all forms (and form fragments) in the block. \endverbatim

\subsectionVariable maps

There are about five things that the debugger might want to know about a variable: \beginitemize

\item[Name] Although a lexical variable's name is "really" a symbol (package and all), in practice it doesn't seem worthwhile to require all the symbols for local variable names to be retained. There is much less VM and GC overhead for a constant string than for a symbol. (Also it is useful to be able to access gensyms in the debugger, even though they are theoretically ineffable).

\item[ID] Which variable with the specified name is this? It is possible to have multiple variables with the same name in a given function. The ID is something that makes Name unique, probably a small integer. When variables aren't unique, we could make this be part of the name, e.g. "FOO\#1", "FOO\#2". But there are advantages to keeping this separate, since in many cases lifetime information can be used to disambiguate, making qualification unnecessary.

\item[SC] When unboxed representations are in use, we must have type information to properly read and write a location. We only need to know the SC for this, which would be amenable to a space-saving numeric encoding.

\item[Location] Simple: the offset in SC. [Actually, we need the save location too.]

\item[Lifetime] In what parts of the program does this variable hold a meaningful value? It seems prohibitive to record precise lifetime information, both in space and compiler effort, so we will have to settle for some sort of approximation.

The finest granularity at which it is easy to determine liveness is the block: we can regard the variable lifetime as the set of blocks that the variable is live in. Of course, the variable may be dead (and thus contain meaningless garbage) during arbitrarily large portions of the block.

Note that this subsumes the notion of which function a variable belongs to. A given block is only in one function, so the function is implicit. \enditemize

The variable map should represent this information space-efficiently and with adequate computational efficiency.

The SC and ID can be represented as small integers. Although the ID can in principle be arbitrarily large, it should be $<$100 in practice. The location can be represented by just the offset (a moderately small integer), since the SB is implicit in the SC.

The lifetime info can be represented either as a bit-vector indexed by block numbers, or by a list of block numbers. Which is more compact depends both on the size of the component and on the number of blocks the variable is live in. In the limit of large component size, the sparse representation will be more compact, but it isn't clear where this crossover occurs. Of course, it would be possible to use both representations, choosing the more compact one on a per-variable basis. Another interesting special case is when the variable is live in only one block: this may be common enough to be worth picking off, although it is probably rarer for named variables than for TNs in general.

If we dump the type, then a normal list-style type descriptor is fine: the space overhead is small, since the shareability is high.

We could probably save some space by cleverly representing the var-info as parallel vectors of different types, but this would be more painful in use. It seems better to just use a structure, encoding the unboxed fields in a fixnum. This way, we can pass around the structure in the debugger, perhaps even exporting it from the low-level debugger interface.

[\#\#\# We need the save location too. This probably means that we need two slots of bits, since we need the save offset and save SC. Actually, we could let the save SC be implied by the normal SC, since at least currently, we always choose the same save SC for a given SC. But even so, we probably can't fit all that stuff in one fixnum without squeezing a lot, so we might as well split and record both SCs.

In a localized packing scheme, we would have to dump a different var-info whenever either the main location or the save location changes. As a practical matter, the save location is less likely to change than the main location, and should never change without the main location changing.

One can conceive of localized packing schemes that do saving as a special case of localized packing. If we did this, then the concept of a save location might be eliminated, but this would require major changes in the IR2 representation for call and/or lifetime info. Probably we will want saving to continue to be somewhat magical.]

How about:

\beginverbatim (defstruct var-info ;; ;; This variable's name. (symbol-name of the symbol) (name nil :type simple-string) ;; ;; The SC, ID and offset, encoded as bit-fields. (bits nil :type fixnum) ;; ;; The set of blocks this variable is live in. If a bit-vector, then it has ;; a 1 when indexed by the number of a block that it is live in. If an ;; I-vector, then it lists the live block numbers. If a fixnum, then that is ;; the number of the sole live block. (lifetime nil :type (or vector fixnum)) ;; ;; The variable's type, represented as list-style type descriptor. type) \endverbatim

Then the debug-info holds a simple-vector of all the var-info structures for that component. We might as well make it sorted alphabetically by name, so that we can binary-search to find the variable corresponding to a particular name.

We need to be able to translate PCs to block numbers. This can be done by an I-Vector in the component that contains the start location of each block. The block number is the index at which we find the correct PC range. This requires that we use an emit-order block numbering distinct from the IR2-Block-Number, but that isn't any big deal. This seems space-expensive, but it isn't too bad, since it would only be a fraction of the code size if the average block length is a few words or more.

An advantage of our per-block lifetime representation is that it directly supports keeping a variable in different locations when in different blocks, i.e. multi-location packing. We use a different var-info for each different packing, since the SC and offset are potentially different. The Name and ID are the same, representing the fact that it is the same variable. It is here that the ID is most significant, since the debugger could otherwise make same-name variables unique all by itself.

\subsectionStack parsing

[\#\#\# Probably not worth trying to make the stack parseable from the bottom up. There are too many complications when we start having variable sized stuff on the stack. It seems more profitable to work on making top-down parsing robust. Since we are now planning to wire the bottom-up linkage info, scanning from the bottom to find the top frame shouldn't be too inefficient, even when there was a runaway recursion. If we somehow jump into hyperspace, then the debugger may get confused, but we can debug this sort of low-level system lossage using ADB.]

There are currently three relevant context pointers: \beginitemize \item The PC. The current PC is wired (implicit in the machine). A saved PC (RETURN-PC) may be anywhere in the current frame. \item The current stack context (CONT). The current CONT is wired. A saved CONT (OLD-CONT) may be anywhere in the current frame. \item The current code object (ENV). The current ENV is wired. When saved, this is extra-difficult to locate, since it is saved by the caller, and is thus at an unknown offset in OLD-CONT, rather than anywhere in the current frame. \enditemize

We must have all of these to parse the stack.

With the proposed Debug-Function, we parse the stack (starting at the top) like this: \beginenumerate \item Use ENV to locate the current Debug-Info \item Use the Debug-Info and PC to determine the current Debug-Function. \item Use the Debug-Function to find the OLD-CONT and RETURN-PC. \item Find the old ENV by searching up the stack for a saved code object containing the RETURN-PC. \item Assign old ENV to ENV, OLD-CONT to CONT, RETURN-PC to PC and goto 1. \endenumerate

If we changed the function representation so that the code and environment were a single object, then the location of the old ENV would be simplified. But we still need to represent ENV as separate from PC, since interrupts and errors can happen when the current PC isn't positioned at a valid return PC.

It seems like it might be a good idea to save OLD-CONT, RETURN-PC and ENV at the beginning of the frame (before any stack arguments). Then we wouldn't have to search to locate ENV, and we also have a hope of parsing the stack even if it is damaged. As long as we can locate the start of some frame, we can trace the stack above that frame. We can recognize a probable frame start by scanning the stack for a code object (presumably a saved ENV).

Probably we want some fairly general mechanism for specifying that a TN should be considered to be live for the duration of a specified environment. It would be somewhat easier to specify that the TN is live for all time, but this would become very space-inefficient in large block compilations.

This mechanism could be quite useful for other debugger-related things. For example, when debuggability is important, we could make the TNs holding arguments live for the entire environment. This would guarantee that a backtrace would always get the right value (modulo setqs).

Note that in this context, "environment" means the Environment structure (one per non-let function). At least according to current plans, even when we do inter-routine register allocation, the different functions will have different environments: we just "equate" the environments. So the number of live per-environment TNs is bounded by the size of a "function", and doesn't blow up in block compilation.

The implementation is simple: per-environment TNs are flagged by the :Environment kind. :Environment TNs are treated the same as :Normal TNs by everyone except for lifetime/conflict analysis. An environment's TNs are also stashed in a list in the IR2-Environment structure. During the conflict analysis post-pass, we look at each block's environment, and make all the environment's TNs always-live in that block.

We can implement the "fixed save location" concept needed for lazy frame creation by allocating the save TNs as wired TNs at IR2 conversion time. We would use the new "environment lifetime" concept to specify the lifetimes of the save locations. There isn't any run-time overhead if we never get around to using the save TNs. [Pack would also have to notice TNs with pre-allocated save TNs, packing the original TN in the stack location if its FSC is the stack.]

We want a standard (recognizable) format for an "escape" frame. We must make an escape frame whenever we start running another function without the

current function getting a chance to save its registers. This may be due either to a truly asynchronous event such as a software interrupt, or due to an "escape" from a miscop. An escape frame marks a brief conversion to a callee-saves convention.

Whenever a miscop saves registers, it should make an escape frame. This ensures that the "current" register contents can always be located by the debugger. In this case, it may be desirable to be able to indicate that only partial saving has been done. For example, we don't want to have to save all the FP registers just so that we can use a couple extra general registers.

When the debugger see an escape frame, it knows that register values are located in the escape frame's "register save" area, rather than in the normal save locations.

It would be nice if there was a better solution to this internal error concept. One problem is that it seems there is a substantial space penalty for emitting all that error code, especially now that we don't share error code between errors because we want to preserve the source context in the PC. But this probably isn't really all that bad when considered as a fraction of the code. For example, the check part of a type check is 12 bytes, whereas the error part is usually only 6. In this case, we could never reduce the space overhead for type checks by more than 1/3, thus the total code size reduction would be small. This will be made even less important when we do type check optimizations to reduce the number of type checks.

Probably we should stick to the same general internal error mechanism, but make it interact with the debugger better by allocating linkage registers and allowing proceedable errors. We could support shared error calls and non-proceedable errors when space is more important than debuggability, but this is probably more complexity than is worthwhile.

We jump or trap to a routine that saves the context (allocating at most the return PC register). We then encode the error and context in the code immediately following the jump/trap. (On the MIPS, the error code can be encoded in the trap itself.) The error arguments would be encoded as SC-offsets relative to the saved context. This could solve both the arg-trashing problem and save space, since we could encode the SC-offsets more tersely than the corresponding move instructions.

\chapterGlossary% -*- Dictionary: int:design -*-

% Note: in an entry, any word that is also defined should be \it % should entries have page references as well?

\begindescription \item[assert (a type)] In Python, all type checking is done via a general type assertion mechanism. Explicit declarations and implicit assertions (e.g. the arg to + is a number) are recorded in the front-end (implicit continuation) representation. Type assertions (and thus type-checking) are "unbundled" from the operations that are affected by the assertion. This has two major advantages: \beginitemize \item Code that implements operations need not concern itself with checking operand types.

\item Run-time type checks can be eliminated when the compiler can prove that the assertion will always be satisfied. \enditemize See also \it restrict.

\item[back end] The back end is the part of the compiler that operates on the \it virtual machine intermediate representation. Also included are the compiler phases involved in the conversion from the \it front end representation (or \it ICR).

\item[bind node] This is a node type the that marks the start of a \it lambda body in \it ICR. This serves as a placeholder for environment manipulation code.

\item[IR1] The first intermediate representation, also known as \it ICR, or the Implicit Continuation Represenation.

\item[IR2] The second intermediate representation, also known as \it VMR, or the Virtual Machine Representation.

\item[basic block] A basic block (or simply "block") has the pretty much the usual meaning of representing a straight-line sequence of code. However, the code sequence ultimately generated for a block might contain internal branches that were hidden inside the implementation of a particular operation. The type of a block is actually \tt cblock. The \tt block-info slot holds an \tt VMR-block containing backend information.

\item[block compilation] Block compilation is a term commonly used to describe the compile-time resolution of function names. This enables many optimizations.

\item[call graph] Each node in the call graph is a function (represented by a \it flow graph.) The arcs in the call graph represent a possible call from one function to another. See also \it tail set.

\item[cleanup] A cleanup is the part of the implicit continuation representation that retains information scoping relationships. For indefinite extent bindings (variables and functions), we can abandon scoping information after ICR conversion, recovering the lifetime information using flow analysis. But dynamic bindings (special values, catch, unwind protect, etc.) must be removed at a precise time (whenever the scope is exited.) Cleanup structures form a hierarchy that represents the static nesting of dynamic binding structures. When the compiler does a control transfer, it can use the cleanup information to determine what cleanup code needs to be emitted.

\item[closure variable] A closure variable is any lexical variable that has references outside of its \it home environment. See also \it indirect value cell.

\item[closed continuation] A closed continuation represents a \tt tagbody tag or \tt block name that is closed over. These two cases are mostly indistinguishable in \it ICR.

\item[home] Home is a term used to describe various back-pointers. A lambda variable's "home" is the lambda that the variable belongs to. A lambda's "home environment" is the environment in which that lambda's variables are allocated.

\item[indirect value cell] Any closure variable that has assignments (\tt setqs) will be allocated in an indirect value cell. This is necessary to ensure that all references to the variable will see assigned values, since the compiler normally freely copies values when creating a closure.

\item[set variable] Any variable that is assigned to is called a "set variable". Several optimizations must special-case set variables, and set closure variables must have an \it indirect value cell.

\item[code generator] The code generator for a \it VOP is a potentially arbitrary list code fragment which is responsible for emitting assembly code to implement that VOP.

\item[constant pool] The part of a compiled code object that holds pointers to non-immediate constants.

\item[constant TN] A constant TN is the \it VMR of a compile-time constant value. A constant may be immediate, or may be allocated in the \it constant pool.

\item[constant leaf] A constant \it leaf is the \it ICR of a compile-time constant value.

\item[combination] A combination \it node is the \it ICR of any fixed-argument function call (not \tt apply or \tt multiple-value-call.)

\item[top-level component] A top-level component is any component whose only entry points are top-level lambdas.

\item[top-level lambda] A top-level lambda represents the execution of the outermost form on which the compiler was invoked. In the case of \tt compile-file, this is often a truly top-level form in the source file, but the compiler can recursively descend into some forms (\tt eval-when, etc.) breaking them into separate compilations.

\item[component] A component is basically a sequence of blocks. Each component is compiled into a separate code object. With \it block compilation or \it local functions, a component will contain the code for more than one function. This is called a component because it represents a connected portion of the call graph. Normally the blocks are in depth-first order (\it DFO).

\item[component, initial] During ICR conversion, blocks are temporarily assigned to initial components. The "flow graph canonicalization" phase determines the true component structure.

\item[component, head and tail] The head and tail of a component are dummy blocks that mark the start and end of the \it DFO sequence. The component head and tail double as the root and finish node of the component's flow graph.

\item[local function (call)] A local function call is a call to a function known at compile time to be in the same \it component. Local call allows compile time resolution of the target address and calling conventions. See \it block compilation.

\item[conflict (of TNs, set)] Register allocation terminology. Two TNs conflict if they could ever be live simultaneously. The conflict set of a TN is all TNs that it conflicts with.

\item[continuation] The ICR data structure which represents both: \beginitemize \item The receiving of a value (or multiple values), and

\item A control location in the flow graph. \enditemize In the Implicit Continuation Representation, the environment is implicit in the continuation's BLOCK (hence the name.) The ICR continuation is very similar to a CPS continuation in its use, but its representation doesn't much resemble (is not interchangeable with) a lambda.

\item[cont] A slot in the \it node holding the \it continuation which receives the node's value(s). Unless the node ends a \it block, this also implicitly indicates which node should be evaluated next.

\item[cost] Approximations of the run-time costs of operations are widely used in the back end. By convention, the unit is generally machine cycles, but the values

are only used for comparison between alternatives. For example, the VOP cost is used to determine the preferred order in which to try possible implementations.

\item[CSP, CFP] See \it control stack pointer and \it control frame pointer.

\item[Control stack] The main call stack, which holds function stack frames. All words on the control stack are tagged \it descriptors. In all ports done so far, the control stack grows from low memory to high memory. The most recent call frames are considered to be "on top" of earlier call frames.

\item[Control stack pointer] The allocation pointer for the \it control stack. Generally this points to the first free word at the top of the stack.

\item[Control frame pointer] The pointer to the base of the \it control stack frame for a particular function invocation. The CFP for the running function must be in a register.

\item[Number stack] The auxiliary stack used to hold any \it non-descriptor (untagged) objects. This is generally the same as the C call stack, and thus typically grows down.

\item[Number stack pointer] The allocation pointer for the \it number stack. This is typically the C stack pointer, and is thus kept in a register.

\item[NSP, NFP] See \it number stack pointer, \it number frame pointer.

\item[Number frame pointer] The pointer to the base of the \it number stack frame for a particular function invocation. Functions that don't use the number stack won't have an NFP, but if an NFP is allocated, it is always allocated in a particular register. If there is no variable-size data on the number stack, then the NFP will generally be identical to the NSP.

\item[Lisp return address] The name of the \it descriptor encoding the "return pc" for a function call.

\item[LRA] See \it lisp return address. Also, the name of the register where the LRA is passed.

\item[Code pointer] A pointer to the header of a code object. The code pointer for the currently running function is stored in the \tt code register.

\item[Interior pointer] A pointer into the inside of some heap-allocated object. Interior pointers confuse the garbage collector, so their use is highly constrained. Typically there is a single register dedicated to holding interior pointers.

\item[dest] A slot in the \it continuation which points the the node that receives this value. Null if this value is not received by anyone.

\item[DFN, DFO] See \it Depth First Number, \it Depth First Order.

\item[Depth first number] Blocks are numbered according to their appearance in the depth-first ordering (the \tt block-number slot.) The numbering actually increases from the component tail, so earlier blocks have larger numbers.

\item[Depth first order] This is a linearization of the flow graph, obtained by a depth-first walk. Iterative flow analysis algorithms work better when blocks are processed in DFO (or reverse DFO.)

\item[Object] In low-level design discussions, an object is one of the following: \beginitemize \item a single word containing immediate data (characters, fixnums, etc) \item a single word pointing to an object (structures, conses, etc.) \enditemize

These are tagged with three low-tag bits as described in the section \refsec:tagging This is synonymous with \it descriptor. In other parts of the documentation, may be used more loosely to refer to a \it lisp object.

\item[Lisp object] A Lisp object is a high-level object discussed as a data type in the Common Lisp definition.

\item[Data-block] A data-block is a dual-word aligned block of memory that either manifests a Lisp object (vectors, code, symbols, etc.) or helps manage a Lisp object on the heap (array header, function header, etc.).

\item[Descriptor] A descriptor is a tagged, single-word object. It either contains immediate data or a pointer to data. This is synonymous with \it object. Storage locations that must contain descriptors are referred to as descriptor locations.

\item[Pointer descriptor] A descriptor that points to a \it data block in memory (i.e. not an immediate object.)

\item[Immediate descriptor] A descriptor that encodes the object value in the descriptor itself; used for characters, fixnums, etc.

\item[Word] A word is a 32-bit quantity.

\item[Non-descriptor] Any chunk of bits that isn't a valid tagged descriptor. For example, a double-float on the number stack. Storage locations that are not scanned by the garbage collector (and thus cannot contain \it pointer descriptors) are called non-descriptor locations. \it Immediate descriptors can be stored in non-descriptor locations.

\item[Entry point] An entry point is a function that may be subject to "unpredictable" control transfers. All entry points are linked to the root of the flow graph (the component head.) The only functions that aren't entry points are \it let functions. When complex lambda-list syntax is used, multiple entry points may be created for a single lisp-level function. See \it external entry point.

\item[External entry point] A function that serves as a "trampoline" to intercept function calls coming in from outside of the component. The XEP does argument syntax and type checking, and may also translate the arguments and return values for a locally specialized calling calling convention.

\item[XEP] An \it external entry point.

\item[lexical environment] A lexical environment is a structure that is used during VMR conversion to represent all lexically scoped bindings (variables, functions, declarations, etc.) Each \tt node is annotated with its lexical environment, primarily for use by the debugger and other user interfaces. This structure is also the environment object passed to \tt macroexpand.

\item[environment] The environment is part of the ICR, created during environment analysis. Environment analysis apportions code to disjoint environments, with all code in the same environment sharing the same stack frame. Each environment has a "\it real" function that allocates it, and some collection \tt let functions. Although environment analysis is the last ICR phase, in earlier phases, code is sometimes said to be "in the same/different environment(s)". This means that the code will definitely be in the same environment (because it is in the same real function), or that is might not be in the same environment, because it is not in the same function.

\item[fixup] Some sort of back-patching annotation. The main sort encountered are load-time \it assembler fixups, which are a linkage annotation mechanism.

\item[flow graph] A flow graph is a directed graph of basic blocks, where each arc represents a possible control transfer. The flow graph is the basic data structure used to represent code, and provides direct support for data flow analysis. See component and ICR.

\item[foldable] An attribute of \it known functions. A function is foldable if calls may be constant folded whenever the arguments are compile-time constant. Generally this means that it is a pure function with no side effects.

\item[FSC] \item[full call] \item[function attribute] function "real" (allocates environment) meaning function-entry more vague (any lambda?) funny function GEN (kill and...) global TN, conflicts, preference GTN (number) IR ICR VMR ICR conversion, VMR conversion (translation) inline expansion, call kill (to make dead) known function LAMBDA leaf let call lifetime analysis, live (tn, variable) load tn LOCS (passing, return locations) local call local TN, conflicts, (or just used in one block) location (selection) LTN (number) main entry mess-up (for cleanup) more arg (entry) MV non-local exit non-packed SC, TN non-set variable operand (to vop) optimizer (in icr optimize) optional-dispatch pack, packing, packed pass (in a transform) passing locations (value) conventions (known, unknown) policy (safe, fast, small, ...) predecessor block primitive-type reaching definition REF representation selection for value result continuation (for function) result type assertion (for template) (or is it restriction) restrict a TN to finite SBs a template operand to a primitive type (boxed...) a tn-ref to particular SCs

return (node, vops) safe, safety saving (of registers, costs) SB SC (restriction) semi-inline side-effect in ICR in VMR sparse set splitting (of VMR blocks) SSET SUBPRIMITIVE successor block tail recursion tail recursive tail recursive loop user tail recursion

template TN TNBIND TN-REF transform (source, ICR) type assertion inference top-down, bottom-up assertion propagation derived, asserted descriptor, specifier, intersection, union, member type check type-check (in continuation) UNBOXED (boxed) descriptor unknown values continuation unset variable unwind-block, unwinding used value (dest) value passing VAR VM VOP \item[XEP]

\enddescription

14 Introduction

Internal Design of CMU Common Lisp on the IBM RT PC

David B. McDonald Scott E. Fahlman Skef Wholey

CMU-CS-87-157

EEK: The purpose for temporarily adding this historical work is to demonstrate how to coordinate the documentation of a specific port with the hardware abstracted description of SBCL provided in the rest of the document.

CMU Common Lisp is an implementation of Common Lisp that currently runs on the IBM RT PC under Mach, a Berkeley Unix 4.3 binary compatible operating system. This document describes low level details of the implementation. In particular, it describes the data formats used for all Lisp objects, the assembler language routines (miscops) used to support compiled code, the function call and return mechanism, and other design information necessary to understand the underlying structure of the CMU Common Lisp implementation on the IBM RT PC under the Mach operating system.

Contract=Strategic87-90

This document is based heavily on the document *Revised Internal Design of Spice Lisp* by Skef Wholey, Scott Fahlman, and Joseph Ginder.

The FASL file format was designed by Guy L. Steele Jr. and Walter van Roggen, and the appendix on this subject is their document with very few modifications.

14.1 Scope and Purpose

This document describes a new implementation of CMU Common Lisp (nee Spice Lisp) as it is implemented on the [No value for "("]DinkyMachine) running Mach, a Berkeley Unix 4.3 binary compatible operating system. This design is undergoing rapid change, and for the present is not guaranteed to accurately describe any past, present, or future implementation of CMU Common Lisp. All questions and comments on this material should be directed to David B. McDonald (David.McDonald@CS.CMU.EDU).

This document specifies the hand-coded assembler routines (miscops) and virtual memory architecture of the [No value for "("]DinkyMachine) CMU Common Lisp system. This is a working document, and it will change frequently as the system is developed and maintained. If some detail of the system does not agree with what is specified here, it is to be considered a bug.

14.2 Notational Conventions

CMU Common Lisp objects are 32 bits long. The high-order bit of each word is numbered 0; the low-order bit is numbered 31. If a word is broken into smaller units, these are packed into the word from left to right. For example, if we break a word into bytes, byte 0 would occupy bits 0-7, byte 1 would occupy 8-15, byte 2 would occupy 16-23, and byte 3 would occupy 24-31.

All CMU Common Lisp documentation uses decimal as the default radix; other radices will be indicated by a subscript (as in 77[8]) or by a clear statement of what radix is in use.

15 [Data Types and Object Formats]

15.1 [Lisp Objects]

Lisp objects are 32 bits long. They come in 32 basic types, divided into three classes: immediate data types, pointer types, and forwarding pointer types. The storage formats are as follows:

```
@b[Immediate Data Types:]
 0              4 5                                                          31
-------------------------------------------------------------------------------
| Type Code (5) |        Immediate Data (27)                |
-------------------------------------------------------------------------------

@b[Pointer and Forwarding Types:]
 0              4 5                6 7                        29            31
-------------------------------------------------------------------------------
| Type Code (5) | Space Code (2) |    Pointer (23)     | Unused (2) |
-------------------------------------------------------------------------------
```

15.2 [Table of Type Codes]

Code	Type	Class	Explanation
0	+ Fixnum Immediate		Positive fixnum, miscop code, etc.
1	GC-Forward	Pointer	GC forward pointer, used during GC.
4	Bignum	Pointer	Bignum.
5	Ratio	Pointer	Two words: numerator, denominator.
6	+ Short Float	Immediate	Positive short flonum.
7	- Short Float	Immediate	Negative short flonum.
8	Single Float	Pointer	Single precision float.
9	Double Float	Pointer	Double precision float (?).
9	Long Float	Pointer	Long float.
10	Complex Pointer	Two words: real, imaginary parts.	
11	String	Pointer	Character string.
12	Bit-Vector	Pointer	Vector of bits
13	Integer-Vector	Pointer	Vector of integers
14	General-Vector	Pointer	Vector of Lisp objects.
15	Array	Pointer	Array header.
16	Function Pointer	Compiled function header.	
17	Symbol	Pointer	Symbol.
18	List	Pointer	Cons cell.
20	C. S. Pointer	Pointer	Pointer into control stack.
21	B. S. Pointer	Pointer	Pointer into binding stack.

```
26      Interruptible     Immediate        Marks a miscop as interruptible.
27      Character         Immediate        Character object.
28      Values-Marker     Immediate        Multiple values marker.
29      Catch-All         Immediate        Catch-All object.
30      Trap              Immediate        Illegal object trap.
31      - Fixnum Immediate        Negative fixnum.
```

15.3 [Table of Space Codes]

```
Code     Space            Explanation
----     -----            -----------
0        Dynamic-0        Storage normally garbage collected, space 0.
1        Dynamic-1        Storage normally garbage collected, space 1.
2        Static           Permanent objects, never moved or reclaimed.
3        Read-Only        Objects never moved, reclaimed, or altered.
```

15.4 Immediate Data Type Descriptions

`Fixnum` 28-bit two's complement integer. The sign bit is stored redundantly in the top 5 bits of the word.

`Short-Float`

The sign bit is stored as part of the type code, allowing a 28 bit signed short float format. The format of short floating point numbers is:

```
 0              3     4        5           12 13                  31
-----------------------------------------------------------------
| Type code (4) | Sign (1) | Exponent (8) |   Mantissa (19)   |
-----------------------------------------------------------------
```

The floating point number is the same format as the [No value for "("]DinkyMachine) supports for single precision numbers, except it has been shifted right by four bits for the type code. The result of any operation is therefore truncated. Long floating point numbers are also available if you need more accuracy and better error propagation properties.

`Character` A character object holding a character code, control bits, and font in the following format:

```
 0              4 6           7  8       15 16        23 24       31
-----------------------------------------------------------------
| Type code (5) | Unused (3) | Font (8) | Bits (8) | Code (8) |
-----------------------------------------------------------------
```

`Values-Marker`

Used to mark the presence of multiple values on the stack. The low 16 bits indicate how many values are being returned. Note that only 65535 values can be returned from a multiple-values producing form.

These are pushed onto the stack in order, and the Values-Marker is returned in register A0.

`Catch-All` Object used as the catch tag for unwind-protects. Special things happen when a catch frame with this as its tag is encountered during a throw. See section ⟨undefined⟩ "[", page ⟨undefined⟩Catch] for details.

`Trap` Illegal object trap. This value is used in symbols to signify an undefined value or definition.

`Interruptible-Marker`
Object used to mark a miscop as interruptible. This object is put in one of the registers and signals to the interrupt handler that the miscop can be interrupted safely. Only miscops that can take a long time (e.g., length when passed a circular list, system call miscops that may wait indefinitely) are marked this way.

15.5 [Pointer-Type Objects and Spaces]

Each of the pointer-type lisp objects points into a different space in virtual memory. There are separate spaces for Bit-Vectors, Symbols, Lists, and so on. The 5-bit type-code provides the high-order virtual address bits for the object, followed by the 2-bit space code, followed by the 25-bit pointer address. This gives a 30-bit virtual address to a 32-bit word; since the [No value for "("]DinkyMachine) is a byte-addressed machine, the two low-order bits are 0. In effect we have carved a 30-bit space into a fixed set of 23-bit subspaces, not all of which are used.

The space code divides each of the type spaces into four sub-spaces, as shown in the table above. At any given time, one of the dynamic spaces is considered newspace, while the other is oldspace. During a stop and copy garbage collection, a "flip" can be done, turning the old newspace into the new oldspace. All type-spaces are flipped at once. Allocation of new dynamic objects always occurs in newspace.

Optionally, the user (or system functions) may allocate objects in static or read-only space. Such objects are never reclaimed once they are allocated – they occupy the space in which they were initially allocated for the lifetime of the Lisp process. The advantage of static allocation is that the GC never has to move these objects, thereby saving a significant amount of work, especially if the objects are large. Objects in read-only space are static, in that they are never moved or reclaimed; in addition, they cannot be altered once they are set up. Pointers in read-only space may only point to read-only or static space, never to dynamic space. This saves even more work, since read-only space does not need to be scavenged, and pages of read-only material do not need to be written back onto the disk during paging.

Objects in a particular type-space will contain either pointers to garbage-collectible objects or words of raw non-garbage-collectible bits, but not both. Similarly, a space will contain either fixed-length objects or variable-length objects, but not both. A variable-length object always contains a 24-bit length field right-justified in the first word, with the positive fixnum type-code in the high-order five bits. The remaining three bits can be used for sub-type information. The length

field gives the size of the object in 32-bit words, including the header word. The garbage collector needs this information when the object is moved, and it is also useful for bounds checking.

The format of objects in each space are as follows:

`Symbol` Each symbol is represented as a fixed-length block of boxed Lisp cells. The number of cells per symbol is 5, in the following order:

```
0  Value cell for shallow binding.
1  Definition cell: a function or list.
2  Property list: a list of attribute-value pairs.
3  Print name: a string.
4  Package: the obarray holding this symbol.
```

`List` A fixed-length block of two boxed Lisp cells, the CAR and the CDR.

`General-Vector`
Vector of lisp objects, any length. The first word is a fixnum giving the number of words allocated for the vector (up to 24 bits). The highest legal index is this number minus 2. The second word is vector entry 0, and additional entries are allocated contiguously in virtual memory. General vectors are sometimes called G-Vectors. (See section ⟨undefined⟩ "[", page ⟨undefined⟩Vectors] for further details.)

`Integer-Vector`
Vector of integers, any length. The 24 low bits of the first word give the allocated length in 32-bit words. The low-order 28 bits of the second word gives the length of the vector in entries, whatever the length of the individual entries may be. The high-order 4 bits of the second word contain access-type information that yields, among other things, the number of bits per entry. Entry 0 is left-justified in the third word of the vector. Bits per entry will normally be powers of 2, so they will fit neatly into 32-bit words, but if necessary some empty space may be left at the low-order end of each word. Integer vectors are sometimes called I-Vectors. (See section ⟨undefined⟩ "[", page ⟨undefined⟩Vectors] for details.)

`Bit-Vector`
Vector of bits, any length. Bit-Vectors are represented in a form identical to I-Vectors, but live in a different space for efficiency reasons.

`Bignum` Bignums are infinite-precision integers, represented in a format identical to G-Vectors. Each bignum is stored as a series of 32-bit words, with the low-order word stored first. The representation is two's complement, but the sign of the number is redundantly encoded in the type field of the fixnum in the header word. If this fixnum is non-negative, then so is the bignum, if it is negative, so is the bignum.

`Floats` Floats are stored as two or more consecutive words of bits, in the following format:

```
-----------------------------------------------------------------
```

```
|  Header word, used only for GC forward pointers.        |
-------------------------------------------------------------
|  Appropriate number of 32-bit words in machine format          |
-------------------------------------------------------------
```

The number of words used to represent a floating point number is one plus the size of the floating point number being stored. The floating point numbers will be represented in whatever format the [No value for "("]DinkyMachine) expects. The extra header word is needed so that a valid floating point number is not mistaken for a gc-forward pointer during a garbage collection.

`Ratio` Ratios are stored as two consecutive words of Lisp objects, which should both be integers.

`Complex` Complex numbers are stored as two consecutive words of Lisp objects, which should both be numbers.

`Array` This is actually a header which holds the accessing and other information about the array. The actual array contents are held in a vector (either an I-Vector or G-Vector) pointed to by an entry in the header. The header is identical in format to a G-Vector. For details on what the array header contains, see section ⟨undefined⟩ "[", page ⟨undefined⟩Arrays].

`String` A vector of bytes. Identical in form to I-Vectors with the access type always 8-Bit. However, instead of accepting and returning fixnums, string accesses accept and return character objects. Only the 8-bit code field is actually stored, and the returned character object always has bit and font values of 0.

`Function` A compiled CMU Common Lisp function consists of both lisp objects and raw bits for the code. The Lisp objects are stored in the Function space in a format identical to that used for general vectors, with a 24-bit length field in the first word. This object contains assorted parameters needed by the calling machinery, a pointer to an 8-bit I-Vector containing the compiled code, a number of pointers to symbols used as special variables within the function, and a number of lisp objects used as constants by the function.

15.6 [Forwarding Pointers]

`GC-Forward`

When a data structure is transported into newspace, a GC-Forward pointer is left behind in the first word of the oldspace object. This points to the same type-space in which it is found. For example, a GC-Forward in G-Vector space points to a structure in the G-Vector newspace. GC-Forward pointers are only found in oldspace.

15.7 [System and Stack Spaces]

The virtual addresses below 08000000[16] are not occupied by Lisp objects, since Lisp objects with type code 0 are positive fixnums. Some of this space is used for other purposes by Lisp. A couple of pages (4096 byte pages) at address 00100000[16] contain tables that Lisp needs to access frequently. These include the allocation table, the active-catch-frame, information to link to C routines, etc. Memory at location 00200000[16] contains code for various miscops. Also, any C code loaded into a running Lisp process is loaded after the miscops. The format of the allocation table is described in chapter ⟨undefined⟩ "[", page ⟨undefined⟩Alloc-Chapter].

The control stack grows upward (toward higher addresses) in memory, and is a framed stack. It contains only general Lisp objects (with some random things encoded as fixnums). Every object pointed to by an entry on this stack is kept alive. The frame for a function call contains an area for the function's arguments, an area for local variables, a pointer to the caller's frame, and a pointer into the binding stack. The frame for a Catch form contains similar information. The precise stack format can be found in chapter ⟨undefined⟩ "[", page ⟨undefined⟩Runtime].

The special binding stack grows downward. This stack is used to hold previous values of special variables that have been bound. It grows and shrinks with the depth of the binding environment, as reflected in the control stack. This stack contains symbol-value pairs, with only boxed Lisp objects present.

All Lisp objects are allocated on word boundaries, since the [No value for "("]DinkyMachine) can only access words on word boundaries.

15.8 [Vectors and Arrays]

Common Lisp arrays can be represented in a few different ways in CMU Common Lisp – different representations have different performance advantages. Simple general vectors, simple vectors of integers, and simple strings are basic CMU Common Lisp data types, and access to these structures is quicker than access to non-simple (or "complex") arrays. However, all multi-dimensional arrays in CMU Common Lisp are complex arrays, so references to these are always through a header structure.

15.8.1 [General Vectors]

G-Vectors contain Lisp objects. The format is as follows:

```
-------------------------------------------------------------------
|  Fixnum code (5) | Subtype (3) |   Allocated length (24)    |
-------------------------------------------------------------------
|  Vector entry 0   (Additional entries in subsequent words)  |
-------------------------------------------------------------------
```

The first word of the vector is a header indicating its length; the remaining words hold the boxed entries of the vector, one entry per 32-bit word. The header word is of type fixnum. It contains a 3-bit subtype field, which is used to indicate several special types of general vectors. At present, the following subtype codes are defined:

0 Normal. Used for assorted things.

1 Named structure created by DEFSTRUCT, with type name in entry 0.

2 EQ Hash Table, last rehashed in dynamic-0 space.

3 EQ Hash Table, last rehashed in dynamic-1 space.

4 EQ Hash Table, must be rehashed.

Following the subtype is a 24-bit field indicating how many 32-bit words are allocated for this vector, including the header word. Legal indices into the vector range from zero to the number in the allocated length field minus 2, inclusive. Normally, the index is checked on every access to the vector. Entry 0 is stored in the second word of the vector, and subsequent entries follow contiguously in virtual memory.

Once a vector has been allocated, it is possible to reduce its length by using the Shrink-Vector miscop, but never to increase its length, even back to the original size, since the space freed by the reduction may have been reclaimed. This reduction simply stores a new smaller value in the length field of the header word.

It is not an error to create a vector of length 0, though it will always be an out-of-bounds error to access such an object. The maximum possible length for a general vector is 2[24]-2 entries, and that can't fit in the available space. The maximum length is 2[23]-2 entries, and that is only possible if no other general vectors are present in the space.

Bignums are identical in format to G-Vectors although each entry is a 32-bit integer, and thus only assembler routines should ever access an entry.

Objects of type Function and Array are identical in format to general vectors, though they have their own spaces.

15.8.2 [Integer Vectors]

I-Vectors contain unboxed items of data, and their format is more complex. The data items come in a variety of lengths, but are of constant length within a given vector. Data going to and from an I-Vector are passed as Fixnums, right justified. Internally these integers are stored in packed form, filling 32-bit words without any type-codes or other overhead. The format is as follows:

```
-----------------------------------------------------------------
| Fixnum code (5) | Subtype (3) |  Allocated length (24)       |
-----------------------------------------------------------------
| Access type (4) | Number of entries (28)                    |
-----------------------------------------------------------------
| Entry 0 left justified                                      |
-----------------------------------------------------------------
```

The first word of an I-Vector contains the Fixnum type-code in the top 5 bits, a 3-bit subtype code in the next three bits, and the total allocated length of the vector (in 32-bit words) in the low-order 24 bits. At present, the following subtype codes are defined:

0 Normal. Used for assorted things.

1 Code. This is the code-vector for a function object.

The second word of the vector is the one that is looked at every time the vector is accessed. The low-order 28 bits of this word contain the number of valid entries in the vector, regardless of how long each entry is. The lowest legal index into the vector is always 0; the highest legal index is one less than this number-of-entries field from the second word. These bounds are checked on every access. Once a vector is allocated, it can be reduced in size but not increased. The Shrink-Vector miscop changes both the allocated length field and the number-of-entries field of an integer vector.

The high-order 4 bits of the second word contain an access-type code which indicates how many bits are occupied by each item (and therefore how many items are packed into a 32-bit word). The encoding is as follows:

0	1-Bit	8	Unused
1	2-Bit	9	Unused
2	4-Bit	10	Unused
3	8-Bit	11	Unused
4	16-Bit	12	Unused
5	32-Bit	13	Unused
6	Unused	14	Unused
7	Unused	15	Unused

In I-Vectors, the data items are packed into the third and subsequent words of the vector. Item 0 is left justified in the third word, item 1 is to its right, and so on until the allocated number of items has been accommodated. All of the currently-defined access types happen to pack neatly into 32-bit words, but if this should not be the case, some unused bits would remain at the right side of each word. No attempt will be made to split items between words to use up these odd bits. When allocated, an I-Vector is initialized to all 0's.

As with G-Vectors, it is not an error to create an I-Vector of length 0, but it will always be an error to access such a vector. The maximum possible length of an I-Vector is 2[28]-1 entries or 2[23]-3 words, whichever is smaller.

Objects of type String are identical in format to I-Vectors, though they have their own space. Strings always have subtype 0 and access-type 3 (8-Bit). Strings differ from normal I-Vectors in that the accessing miscops accept and return objects of type Character rather than Fixnum.

15.8.3 [Arrays]

An array header is identical in form to a G-Vector. Like any G-Vector, its first word contains a fixnum type-code, a 3-bit subtype code, and a 24-bit total length field (this is the length of the array header, not of the vector that holds the data). At present, the subtype code is always 0. The entries in the header-vector are interpreted as follows:

0 Data Vector

This is a pointer to the I-Vector, G-Vector, or string that contains the actual data of the array. In a multi-dimensional array, the supplied indices are converted into a single 1-D index which is used to access the data vector in the usual way.

1 Number of Elements
This is a fixnum indicating the number of elements for which there is space in the data vector.

2 Fill Pointer
This is a fixnum indicating how many elements of the data vector are actually considered to be in use. Normally this is initialized to the same value as the Number of Elements field, but in some array applications it will be given a smaller value. Any access beyond the fill pointer is illegal.

3 Displacement
This fixnum value is added to the final code-vector index after the index arithmetic is done but before the access occurs. Used for mapping a portion of one array into another. For most arrays, this is 0.

4 Range of First Index
This is the number of index values along the first dimension, or one greater than the largest legal value of this index (since the arrays are always zero-based). A fixnum in the range 0 to 2[24]-1. If any of the indices has a range of 0, the array is legal but will contain no data and accesses to it will always be out of range. In a 0-dimension array, this entry will not be present.

5 - N Ranges of Subsequent Dimensions
The number of dimensions of an array can be determined by looking at the length of the array header. The rank will be this number minus 6. The maximum array rank is 65535 - 6, or 65529.

The ranges of all indices are checked on every access, during the conversion to a single data-vector index. In this conversion, each index is added to the accumulating total, then the total is multiplied by the range of the following dimension, the next index is added in, and so on. In other words, if the data vector is scanned linearly, the last array index is the one that varies most rapidly, then the index before it, and so on.

15.9 [Symbols Known to the Assembler Routines]

A large number of symbols will be pre-defined when a CMU Common Lisp system is fired up. A few of these are so fundamental to the operation of the system that their addresses have to be known to the assembler routines. These symbols are listed here. All of these symbols are in static space, so they will not move around.

`NIL` 94000000[16] The value of NIL is always NIL; it is an error to alter it. The plist of NIL is always NIL; it is an error to alter it. NIL is unique among symbols in that it is stored in Cons cell space and thus you can take its CAR and CDR, yielding NIL in either case. NIL has been placed in Cons cell space so that the more common operations on lists will yield the desired results. This slows down some symbol operations but this should be insignificant compared to the savings in

list operations. A test for NIL for the [No value for "("]DinkyMachine) is:

```
xiu R0,P,X'9400'
bz IsNIL or bnz IsNotNIL
```

`T` 8C000000[16] The value of T is always T; it is an error to alter it. A similar sequence of code as for NIL above can test for T, if necessary.

`%SP-Internal-Apply`
8C000014[16] The function stored in the definition cell of this symbol is called by an assembler routine whenever compiled code calls an interpreted function.

`%SP-Internal-Error`
8C000028[16] The function stored in the definition cell of this symbol is called whenever an error is detected during the execution of an assembler routine. See section ⟨undefined⟩ "[", page ⟨undefined⟩Errors] for details.

`%SP-Software-Interrupt-Handler`
8C00003C[16] The function stored in the definition cell of this symbol is called whenever a software interrupt occurs. See section ⟨undefined⟩ "[", page ⟨undefined⟩Interrupts] for details.

`%SP-Internal-Throw-Tag`
8C000050[16] This symbol is bound to the tag being thrown when a Catch-All frame is encountered on the stack. See section ⟨undefined⟩ "[", page ⟨undefined⟩Catch] for details.

`%Initial-function`
8c000064[16] This symbol's function cell should contain a function that is called when the initial core image is started. This function should initialize all the data structures that Lisp needs to run.

`%Link-table-header`
8c000078[16] This symbol's value cell contains a pointer to the link table information.

`Current-allocation-space`
8c00008c[16] This symbol's value cell contains an encoded form of the current space that new lisp objects are to be allocated in.

`%SP-bignum/fixnum`
8c0000a0[16] This function is invoked by the miscops when a division of a bignum by a fixnum results in a ratio.

`%SP-bignum/bignum`
8c0000b4[16] This function is invoked by the miscops when a division of a fixnum by a bignum results in a ratio.

`%SP-bignum/bignum`
8c0000c8[16] This function is invoked by the miscops when a division of a bignum by a bignum results in a ratio.

`%SP-abs-ratio`
: 8c0000dc[16] This function is invoked by the miscops when the absolute value of a ratio is taken.

`%SP-abs-complex`
: 8c0000f0[16] This function is invoked by the miscops when the absolute value of a complex is taken.

`%SP-negate-ratio`
: 8c000104[16] This function is invoked by the miscops when a ratio is to be negated.

`%SP-negate-ratio`
: 8c000118[16] This function is invoked by the miscops when a complex is to be negated.

`%SP-integer+ratio`
: 8c00012c[16] This function is invoked by the miscops when a fixnum or bignum is added to a ratio.

`%SP-ratio+ratio`
: 8c000140[16] This function is invoked by the miscops when a ratio is added to a ratio.

`%SP-complex+number`
: 8c000154[16] This function is invoked by the miscops when a complex is added to a number.

`%SP-number+complex`
: 8c000168[16] This function is invoked by the miscops when a number is added to a complex.

`%SP-complex+complex`
: 8c00017c[16] This function is invoked by the miscops when a number is added to a complex.

`%SP-1+ratio`
: 8c000190[16] This function is invoked by the miscops when 1 is added to a ratio.

`%SP-1+complex`
: 8c000190[16] This function is invoked by the miscops when 1 is added to a complex.

`%SP-ratio-integer`
: 8c0001b8[16] This function is invoked by the miscops when an integer is subtracted from a ratio.

`%SP-ratio-ratio`
: 8c0001cc[16] This function is invoked by the miscops when an ratio is subtracted from a ratio.

`%SP-complex-number`
: 8c0001e0[16] This function is invoked by the miscops when a complex is subtracted from a number.

`%SP-number-complex`
: 8c0001f4[16] This function is invoked by the miscops when a number is subtracted from a complex.

`%SP-complex-complex`
: 8c000208[16] This function is invoked by the miscops when a complex is subtracted from a complex.

`%SP-1-complex`
: 8c000230[16] This function is invoked by the miscops when 1 is subtracted from a complex.

`%SP-ratio*ratio`
: 8c000244[16] This function is invoked by the miscops to multiply two ratios.

`%SP-number*complex`
: 8c000258[16] This function is invoked by the miscops to multiply a number by a complex.

`%SP-complex*number`
: 8c00026c[16] This function is invoked by the miscops to multiply a complex by a number.

`%SP-complex*complex`
: 8c000280[16] This function is invoked by the miscops to multiply a complex by a complex.

`%SP-integer/ratio`
: 8c000294[16] This function is invoked by the miscops to divide an integer by a ratio.

`%SP-ratio/integer`
: 8c0002a8[16] This function is invoked by the miscops to divide a ratio by an integer.

`%SP-ratio/ratio`
: 8c0002bc[16] This function is invoked by the miscops to divide a ratio by a ratio.

`%SP-number/complex`
: 8c0002d0[16] This function is invoked by the miscops to divide a number by a complex.

`%SP-complex/number`
: 8c0002e4[16] This function is invoked by the miscops to divide a complex by a number.

`%SP-complex/complex`
8c0002f8[16] This function is invoked by the miscops to divide a complex by a complex.

`%SP-integer-truncate-ratio`
8c00030c[16] This function is invoked by the miscops to truncate an integer by a ratio.

`%SP-ratio-truncate-integer`
8c000320[16] This function is invoked by the miscops to truncate a ratio by an integer.

`%SP-ratio-truncate-ratio`
8c000334[16] This function is invoked by the miscops to truncate a ratio by a ratio.

`%SP-number-truncate-complex`
8c000348[16] This function is invoked by the miscops to truncate a number by a complex.

`%SP-complex-truncate-number`
8c00035c[16] This function is invoked by the miscops to truncate a complex by a number.

`%SP-complex-truncate-complex`
8c000370[16] This function is invoked by the miscops to truncate a complex by a complex.

`Maybe-GC` 8c000384[16] This function may be invoked by any miscop that does allocation. This function determines whether it is time to garbage collect or not. If it is it performs a garbage collection. Whether it invokes a garbage collection or not, it returns the single argument passed to it.

`Lisp-environment-list`
8c000398[16] The value of this symbol is set to the a list of the Unix environment strings passed into the Lisp process. This list by Lisp to obtain various environment information, such as the user's home directory, etc.

`Call-lisp-from-C`
8c0003ac[16] This function is called whenever a C function called by Lisp tries to call a Lisp function.

`Lisp-command-line-list`
8c0003c0[16] The value of this symbol is set to the list of strings passed into the Lisp process as the command line.

`*Nameserverport*`
8c0003d4[16] The value of this symbol is set to the C global variable name_server_port. This allows Lisp to access the name server.

`*Ignore-Floating-Point-Underflow*`

8c0003e8[16] If the the value of this symbol is NIL then an error is signalled when floating point underflow occurs, otherwise the operation quietly returns zero.

16 [Runtime Environment]

16.1 [Register Allocation]

To describe the assembler support routines in chapter ⟨undefined⟩ "[", page ⟨undefined⟩Instr-Chapter] and the complicated control conventions in chapter ⟨undefined⟩ "[", page ⟨undefined⟩Control-Conventions] requires that we talk about the allocation of the 16 32-bit general purpose registers provided by the [No value for "("]DinkyMachine).

`Program-Counter (PC) [R15]`
: This register contains an index into the current code vector when a Lisp function is about to be called. When a miscop is called, it contains the return address. It may be used as a super temporary between miscop and function calls.

`Active-Function-Pointer (AF) [R14]`
: This register contains a pointer to the active function object. It is used to access the symbol and constant area for the currently running function.

`Active-Frame-Pointer (FP) [R13]`
: This register contains a pointer to the current active frame on the control stack. It is used to access the arguments and local variables stored on the control stack.

`Binding-Stack-Pointer (BS) [R12]`
: This register contains the current binding stack pointer. The binding stack is a downward growing stack and follows a decrement-write/increment-read discipline.

`Local registers (L0-L4) [R7-R11]`
: These registers contain locals and saved arguments for the currently executing function. Functions may use these registers, so that stack accesses can be reduced, since a stack access is relatively expensive compared to a register access.

`Argument register (A0, A1, A2) [R1, R3, R5]`
: These registers contain arguments to a function or miscop that has just been called. On entry to a function or miscop, they contain the first three arguments. The first thing a function does is to move the contents of these registers into the local registers.

`Miscop argument register (A3) [R4]`
: This register is used to pass a fourth argument to miscops requiring four or more arguments. It is also used as a super temporary by the compiler.

`Control-Stack-Pointer (CS) [R6]`
: The stack pointer for the control stack, an object of type Control-Stack-Pointer. Points to the last used word in Control-Stack space; this upward growing stack uses a increment-write/read-decrement discipline.

`Non-Lisp temporary registers (NL0, NL1) [R0, R2]`
: These registers are used to contain non-Lisp values. They will normally be used during miscop calls, but may also be used in in-line code to contain temporary data. These are the only two registers never examined by the garbage collector, so no pointers to Lisp objects should be stored here (since they won't get updated during a garbage collection).

16.2 [Function Object Format]

Each compiled function is represented in the machine as a Function Object. This is identical in form to a G-Vector of lisp objects, and is treated as such by the garbage collector, but it exists in a special function space. (There is no particular reason for this distinction. We may decide later to store these things in G-Vector space, if we become short on spaces or have some reason to believe that this would improve paging behavior.) Usually, the function objects and code vectors will be kept in read-only space, but nothing should depend on this; some applications may create, compile, and destroy functions often enough to make dynamic allocation of function objects worthwhile.

header information needed by the function-calling mechanism: a pointer to the I-Vector that holds the actual code. Following this is the so-called "symbols and constants" area. The first few entries in this area are fixnums that give the offsets into the code vector for various numbers of supplied arguments. Following this begin the true symbols and constants used by the function. Any symbol used by the code as a special variable. Fixnum constants can be generated faster with in-line code than they can be accessed from the function-object, so they are not stored in the constants area.

The subtype of the G-Vector header indicates the type of the function:

- 0 - A normal function (expr).
- 1 - A special form (fexpr).
- 2 - A defmacro macroexpansion function.
- 3 - An anonymous expr. The name is the name of the parent function.
- 4 - A compiled top-level form.

Only the fexpr information has any real meaning to the system. The rest is there for the printer and anyone else who cares.

After the one-word G-Vector header, the entries of the function object are as follows:

```
0  Name of the innermost enclosing named function.
1  Pointer to the unboxed Code vector holding the instructions.
```

```
2  A fixnum with bit fields as follows:
   24  - 31: The minimum legal number of args (0 to 255).
   16  - 23: The maximum number of args, not counting &rest (0 to 255).
         The fixnum has a negative type code, if the function accepts a &rest
         arg and a positive one otherwise.
3  A string describing the source file from which the function was defined.
   See below for a description of the format.
4  A string containing a printed representation of the argument list, for
   documentation purposes. If the function is a defmacro macroexpansion
   function, the argument list will be the one originally given to defmacro
   rather than the actual arglist to the expansion function.
5  The symbols and constants area starts here.
   This word is entry 0 of the symbol/constant area.
   The first few entries in this area are fixnums representing the
   code-vector entry points for various numbers of optional arguments.
```

16.3 [Defined-From String Format]

The defined-from string may have any of three different formats, depending on which of the three compiling functions compiled it:

`compile-file "[filename user-time universal-time]"`
: The [filename] is the namestring of the truename of the file the function was defined from. The time is the file-write-date of the file.

`compile "Lisp on [user-time], machine [machine universal-time]"`
: The time is the time that the function was compiled. [Machine] is the machine-instance of the machine on which the compilation was done.

  compile-from-stream "[stream] on [user-time], machine [machine-instance

`universal-time]"`
: [Stream] is the printed representation of the stream compiled from. The time is the time the compilation started.

An example of the format of [user-time] is 6-May-86 1:04:44. The [universal-time] is the same time represented as a decimal integer. It should be noted that in each case, the universal time is the last thing in the string.

16.4 [Control-Stack Format]

The CMU Common Lisp control stack is a framed stack. Call frames, which hold information for function calls, are intermixed with catch frames, which hold information used for non-local exits. In addition, the control stack is used as a scratchpad for random computations.

16.4.1 [Call Frames]

At any given time, the machine contains pointers to the current top of the control stack and the start of the current active frame (in which the current function is

executing). In addition, there is a pointer to the current top of the special binding stack. CMU Common Lisp on the Perq also has a pointer to an open frame. An open frame is one which has been partially built, but which is still having arguments for it computed. When all the arguments have been computed and saved on the frame, the function is then started. This means that the call frame is completed, becomes the current active frame, and the function is executed. At this time, special variables may be bound and the old values are saved on the binding stack. Upon return, the active frame is popped away and the result is either sent as an argument to some previously opened frame or goes to some other destination. The binding stack is popped and old values are restored.

On the [No value for "("]DinkyMachine), open frames still exist, however, no register is allocated to point at the most recent one. Instead, a count of the arguments to the function is kept. In most cases, a known fixed number of arguments are passed to a function, and this is all that is needed to calculate the correct place to set the active frame pointer. In some cases, it is not as simple, and runtime calculations are necessary to set up the frame pointer. These calculations are simple except in some very strange cases.

The active frame contains pointers to the previously-active frame and to the point to which the binding stack will be popped on exit, among other things. Following this is a vector of storage locations for the function's arguments and local variables. Space is allocated for the maximum number of arguments that the function can take, regardless of how many are actually supplied.

In an open frame, stack space is allocated up to the point where the arguments are stored. Nothing is stored in the frame at this time. Thus, as arguments are computed, they can simply be pushed on the stack. Since the first three arguments are passed in registers, it is sometimes necessary to save these values when succeeding arguments are complicated. When the function is finally started, the remainder of the frame is built (including storing all the registers that must be saved). A call frame looks like this:

```
0   Saved local 0 register.
1   Saved local 1 register.
2   Saved local 2 register.
3   Saved local 3 register.
4   Saved local 4 register.
5   Pointer to previous binding stack.
6   Pointer to previous active frame.
7   Pointer to previous active function.
8   Saved PC of caller. A fixnum.
9   Args-and-locals area starts here. This is entry 0.
```

The first slot is pointed to by the Active-Frame register if this frame is currently active.

16.4.2 [Catch Frames]

Catch frames contain much of the same information that call frames do, and have a very similar format. A catch frame holds the function object for the current

function, a stack pointer to the current active frame, a pointer to the current top of the binding stack, and a pointer to the previous catch frame. When a Throw occurs, an operation similar to returning from this catch frame (as if it were a call frame) is performed, and the stacks are unwound to the proper place for continued execution in the current function. A catch frame looks like this:

```
0   Pointer to current binding stack.
1   Pointer to current active frame.
2   Pointer to current function object.
3   Destination PC for a Throw.
4   Tag caught by this catch frame.
5   Pointer to previous catch frame.
```

The conventions used to manipulate call and catch frames are described in chapter ⟨undefined⟩ "[", page ⟨undefined⟩Control-Conventions].

16.5 [Binding-Stack Format]

Each entry of the binding-stack consists of two boxed (32-bit) words. Pushed first is a pointer to the symbol being bound. Pushed second is the symbol's old value (any boxed item) that is to be restored when the binding stack is popped.

17 [Storage Management]

New objects are allocated from the lowest unused addresses within the specified space. Each allocation call specifies how many words are wanted, and a Free-Storage pointer is incremented by that amount. There is one of these Free-Storage pointers for each space, and it points to the lowest free address in the space. There is also a Clean-Space pointer associated with each space that is used during garbage collection. These pointers are stored in a table which is indexed by the type and space code. The address of the Free-Storage pointer for a given space is

```
(+ alloc-table-base (lsh type 5) (lsh space 3)).
```

The address of the Clean-Space pointer is

```
(+ alloc-table-base (lsh type 5) (lsh space 3) 4).
```

Common Lisp on the [No value for "("]DinkyMachine) uses a stop-and-copy garbage collector to reclaim storage. The Collect-Garbage miscop performs a full GC. The algorithm used is a degenerate form of Baker's incremental garbage collection scheme. When the Collect-Garbage miscop is executed, the following happens:

1. The current newspace becomes oldspace, and the current oldspace becomes newspace.
2. The newspace Free-Storage and Clean-Space pointers are initialized to point to the beginning of their spaces.
3. The objects pointed at by contents of all the registers containing Lisp objects are transported if necessary.
4. The control stack and binding stack are scavenged.
5. Each static pointer space is scavenged.
6. Each new dynamic space is scavenged. The scavenging of the dynamic spaces continues until an entire pass through all of them does not result in anything being transported. At this point, every live object is in newspace.

A Lisp-level GC function returns the oldspace pages to Mach.

17.1 [The Transporter]

The transporter moves objects from oldspace to newspace. It is given an address ⌈A⌉, which contains the object to be transported, ⌈B⌉. If ⌈B⌉ is an immediate object, a pointer into static space, a pointer into read-only space, or a pointer into newspace, the transporter does nothing.

If ⌈B⌉ is a pointer into oldspace, the object it points to must be moved. It may, however, already have been moved. Fetch the first word of ⌈B⌉, and call it ⌈C⌉. If ⌈C⌉ is a GC-forwarding pointer, we form a new pointer with the type code of ⌈B⌉ and the low 27 bits of ⌈C⌉. Write this into ⌈A⌉.

If ⌈C⌉ is not a GC-forwarding pointer, we must copy the object that ⌈B⌉ points to. Allocate a new object of the same size in newspace, and copy the contents. Replace ⌈C⌉ with a GC-forwarding pointer to the new structure, and write the address of the new structure back into ⌈A⌉.

Hash tables maintained with an EQ relation need special treatment by the transporter. Whenever a G-Vector with subtype 2 or 3 is transported to newspace, its subtype code is changed to 4. The Lisp-level hash-table functions will see that the subtype code has changed, and re-hash the entries before any access is made.

17.2 [The Scavenger] The scavenger looks through an area of

pointers for pointers into oldspace, transporting the objects they point to into newspace. The stacks and static spaces need to be scavenged once, but the new dynamic spaces need to be scavenged repeatedly, since new objects will be allocated while garbage collection is in progress. To keep track of how much a dynamic space has been scavenged, a Clean-Space pointer is maintained. The Clean-Space pointer points to the next word to be scavenged. Each call to the scavenger scavenges the area between the Clean-Space pointer and the Free-Storage pointer. The Clean-Space pointer is then set to the Free-Storage pointer. When all Clean-Space pointers are equal to their Free-Storage pointers, GC is complete.

To maintain (and create) locality of list structures, list space is treated specially. When a list cell is transported, if the cdr points to oldspace, it is immediately transported to newspace. This continues until the end of the list is encountered or a non-oldspace pointer occurs in the cdr position. This linearizes lists in the cdr direction which should improve paging performance.

17.3 [Purification]

Garbage is created when the files that make up a CMU Common Lisp system are loaded. Many functions are needed only for initialization and bootstrapping (e.g. the "one-shot" functions produced by the compiler for random forms between function definitions), and these can be thrown away once a full system is built. Most of the functions in the system, however, will be used after initialization. Rather than bend over backwards to make the compiler dump some functions in read-only space and others in dynamic space (which involves dumping their constants in the proper spaces, also), [everything] is dumped into dynamic space. A purify miscop is provided that does a garbage collection and moves accessible information in dynamic space into read-only or static space.

18 [Assembler Support Routines]

To support compiled Common Lisp code many hand coded assembler language routines (miscops) are required. These routines accept arguments in the three argument registers, the special miscop argument register, and in a very few cases on the stack. The current register assignments are:

A0 contains the first argument.

A1 contains the second argument.

A2 contains the third argument.

A3 contains the fourth argument.

The rest of the arguments are passed on the stack with the last argument at the end of the stack. All arguments on the stack must be popped off the stack by the miscop. All miscops return their values in register A0. A few miscops return two or three values, these are all placed in the argument registers. The main return value is stored in register A0, the others in A1 and A2. The compiler must generate code to use the multiple values correctly, i.e., place the return values on the stack and put a values marker in register A0 if multiple-values are wanted. Otherwise the compiler can use the value(s) it needs and ignore the rest. NB: Most of the miscops follow this scheme, however, a few do not. Any discrepancies are explained in the description of particular miscops.

Several of the instructions described in the Perq Internal Design Document do not have associated miscops, rather they have been code directly in-line. Examples of these instructions include push, pop, bind, bind-null, many of the predicates, and a few other instructions. Most of these instructions can be performed in 4 or fewer [No value for "("]DinkyMachine) instructions and the overhead of calling a miscop seemed overly expensive. Some instructions are encoded in-line or as a miscop call depending on settings of compiler optimization switches. If space is more important than speed, then some Perq instructions are compiled as calls to out of line miscops rather than generating in-line code.

18.1 [Miscop Descriptions]

There are 10 classes of miscops: allocation, stack manipulation, list manipulation, symbol manipulation, array manipulation, type predicate, arithmetic and logical, function call and return, miscellaneous, and system hacking.

18.1.1 [Allocation]

All non-immediate objects are allocated in the "current allocation space," which is dynamic space, static space, or read-only space. The current allocation space is initially dynamic space, but can be changed by using the Set-Allocation-Space miscop below. The current allocation space can be determined by using the Get-Allocation-Space miscop. One usually wants to change the allocation space around some section of code; an unwind protect should be used to insure that the allocation space is restored to some safe value.

`Get-Allocation-Space ([])`
returns 0, 2, or 3 if the current allocation space is dynamic, static, or read-only, respectively.

`Set-Allocation-Space ([X])`
sets the current allocation space to dynamic, static, or read-only if [X] is 0, 2, or 3 respectively. Returns [X].

`Alloc-Bit-Vector (Length)`
returns a new bit-vector [Length] bits long, which is allocated in the current allocation space. [Length] must be a positive fixnum.

`Alloc-I-Vector ([Length A])`
returns a new I-Vector [Length] bytes long, with the access code specified by [A]. [Length] and [A] must be positive fixnums.

`Alloc-String ([Length])`
returns a new string [Length] characters long. [Length] must be a fixnum.

`Alloc-Bignum ([Length])`
returns a new bignum [Length] 32-bit words long. [Length] must be a fixnum.

`Make-Complex ([Realpart Imagpart])`
returns a new complex number with the specified [Realpart] and [Imagpart]. [Realpart] and [Imagpart] should be the same type of non-complex number.

`Make-Ratio ([Numerator Denominator])`
returns a new ratio with the specified [Numerator] and [Denominator]. [Numerator] and [Denominator] should be integers.

`Alloc-G-Vector ([Length Initial-Element])`
returns a new G-Vector with [Length] elements initialized to [Initial-Element]. [Length] should be a fixnum.

`Static-G-Vector ([Length Initial-Element])`
returns a new G-Vector in static allocation space with [Length] elements initialized to [Initial-Element].

`Vector ([Elt[0] Elt[1] ... Elt[Length - 1] Length])`
returns a new G-Vector containing the specified [Length] elements. [Length] should be a fixnum and is passed in register A0. The rest of the arguments are passed on the stack.

`Alloc-Function ([Length])`
returns a new function with [Length] elements. [Length] should be a fixnum.

`Alloc-Array ([Length])`
returns a new array with [Length] elements. [Length] should be a fixnum.

`Alloc-Symbol ([Print-Name])`
returns a new symbol with the print-name as [Print-Name]. The value is initially Trap, the definition is Trap, the property list and the package are initially NIL. The symbol is not interned by this operation – that is done in Lisp code. [Print-Name] should be a simple-string.

`Cons ([Car Cdr])`
returns a new cons with the specified [Car] and [Cdr].

`List ([Elt[0] Elt[1] ... Elt[CE - 1] Length])`
returns a new list containing the [Length] elements. [Length] should be fixnum and is passed in register NL0. The first three arguments are passed in A0, A1, and A2. The rest of the arguments are passed on the stack.

`List* ([Elt[0] Elt[1] ... Elt[CE - 1] Length])`
returns a list* formed by the [Length-1] elements. The last element is placed in the cdr of the last element of the new list formed. [Length] should be a fixnum and is passed in register NL0. The first three arguments are passed in A0, A1, and A2. The rest of the arguments are passed on the stack.

`MV-List ([Elt<0> Elt<1> ... Elt<CE - 1> Length])`
returns a list formed from the elements, all of which are on the stack. [Length] is passed in register A0. This miscop is invoked when multiple values from a function call are formed into a list.

18.1.2 [Stack Manipulation]

`Push ([E])` pushes E on to the control stack.

`Pop ([E])` pops the top item on the control stack into [E].

`NPop ([N])` If [N] is positive, [N] items are popped off of the stack. If [N] is negative, NIL is pushed onto the stack -[N] times. [N] must be a fixnum.

`Bind-Null ([E])`
pushes [E] (which must be a symbol) and its current value onto the binding stack, and sets the value of [E] to NIL. Returns NIL.

`Bind (Value Symbol)`
pushes [Symbol] (which must be a symbol) and its current value onto the binding stack, and sets the value cell of [Symbol] to [Value]. Returns [Symbol].

`Unbind ([N])`
undoes the top [N] bindings on the binding stack.

18.1.3 [List Manipulation]

`Car, Cdr, Caar, Cadr, Cdar, Cddr ([E])`
returns the car, cdr, caar, cadr, cdar, or cddr of [E] respectively.

`Set-Cdr, Set-Cddr ([E])`
The cdr or cddr of the contents of [E] is stored in [E]. The contents of [E] should be either a list or NIL.

`Set-Lpop ([E])`
The car of the contents of [E] is returned; the cdr of the contents of [E] is stored in [E]. The contents of [E] should be a list or NIL.

`Spread ([E])`
pushes the elements of the list [E] onto the stack in left-to-right order.

`Replace-Car, Replace-Cdr ([List Value])`
sets the car or cdr of the [List] to [Value] and returns [Value].

`Endp (X)` sets the condition code eq bit to 1 if [X] is NIL, or 0 if [X] is a cons cell. Otherwise an error is signalled.

`Assoc, Assq ([List Item])`
returns the first cons in the association-list [List] whose car is EQL to [Item]. If the = part of the EQL comparison bugs out (and it can if the numbers are too complicated), a Lisp-level Assoc function is called with the current cdr of the [List]. Assq returns the first cons in the association-list [List] whose car is EQ to [Item].

`Member, Memq (`*`List Item`*`)`
returns the first cons in the list *List* whose car is EQL to *Item*. If the = part of the EQL comparison bugs out, a Lisp-level Member function is called with the current cdr of the *List*. Memq returns the first cons in *List* whose car is EQ to the *Item*.

`GetF (`*`List Indicator Default`*`)`
searches for the *Indicator* in the list *List*, cddring down as the Common Lisp form GetF would. If *Indicator* is found, its associated value is returned, otherwise *Default* is returned.

18.1.4 [Symbol Manipulation]

Most of the symbol manipulation miscops are compiled in-line rather than actual calls.

`Get-Value (`*`Symbol`*`)`
returns the value of *Symbol* (which must be a symbol). An error is signalled if *Symbol* is unbound.

`Set-Value (`*`Symbol Value`*`)`
sets the value cell of the symbol *Symbol* to *Value*. *Value* is returned.

`Get-Definition (Symbol)`
returns the definition of the symbol *Symbol*. If *Symbol* is undefined, an error is signalled.

`Set-Definition (Symbol Definition)`
sets the definition of the symbol *Symbol* to *Definition*. *Definition* is returned.

`Get-Plist (Symbol)`
returns the property list of the symbol *Symbol*.

`Set-Plist (Symbol Plist)`
sets the property list of the symbol *Symbol* to *Plist*. *Plist* is returned.

`Get-Pname (Symbol)`
returns the print name of the symbol *Symbol*.

`Get-Package (Symbol)`
returns the package cell of the symbol *Symbol*.

`Set-Package (Symbol Package)`
sets the package cell of the symbol *Symbol* to *Package*. *Package* is returned.

`Boundp (Symbol)`
sets the eq condition code bit to 1 if the symbol *Symbol* is bound; sets it to 0 otherwise.

`FBoundp (Symbol)`
sets the eq condition code bit to 1 if the symbol *Symbol* is defined; sets it to 0 otherwise.

`Get (Symbol Indicator Default)`
searches the property list of *Symbol* for *Indicator* and returns the associated value. If *Indicator* is not found, *Default* is returned.

`Put (Symbol Indicator Value)`
searches the property list of *Symbol* for *Indicator* and replaces the associated value with *Value*. If *Indicator* is not found, the *Indicator Value* pair are consed onto the front of the property list.

18.1.5 [Array Manipulation]

Common Lisp arrays have many manifestations in CMU Common Lisp. The CMU Common Lisp data types Bit-Vector, Integer-Vector, String, General-Vector, and Array are used to implement the collection of data types the Common Lisp manual calls "arrays."

In the following miscop descriptions, "simple-array" means an array implemented in CMU Common Lisp as a Bit-Vector, I-Vector, String, or G-Vector. "Complex-array" means an array implemented as a CMU Common Lisp Array object. "Complex-bit-vector" means a bit-vector implemented as a CMU Common Lisp array; similar remarks apply for "complex-string" and so forth.

`(Vector)` returns the length of the one-dimensional Common Lisp array *Vector*. G-Vector-Length, Simple-String-Length, and Simple-Bit-Vector-Length return the lengths of G-Vectors, CMU Common Lisp strings, and CMU Common Lisp Bit-Vectors respectively. *Vector* should be a vector of the appropriate type.

`Get-Vector-Subtype (Vector)`
returns the subtype field of the vector *Vector* as an integer. *Vector* should be a vector of some sort.

`Set-Vector-Subtype (Vector A)`
sets the subtype field of the vector *Vector* to *A*, which must be a fixnum.

`Get-Vector-Access-Code (Vector)`
returns the access code of the I-Vector (or Bit-Vector) *Vector* as a fixnum.

`Shrink-Vector (Vector Length)`
sets the length field and the number-of-entries field of the vector *Vector* to *Length*. If the vector contains Lisp objects, entries beyond the new end are set to Trap. Returns the shortened vector. *Length* should be a fixnum. One cannot shrink array headers or function headers.

`Typed-Vref (A Vector I)`
returns the *I*'th element of the I-Vector *Vector* by indexing into it as if its access-code were *A*. *A* and *I* should be fixnums.

`Typed-Vset (A Vector I Value)`
sets the *I*'th element of the I-Vector *Vector* to *Value* indexing into *Vector* as if its access-code were *A*. *A*, *I*, and *Value* should be fixnums. *Value* is returned.

`Header-Length (Object)`
returns the number of Lisp objects in the header of the function or array *Object*. This is used to find the number of dimensions of an array or the number of constants in a function.

`Header-Ref (Object I)`
returns the *I*'th element of the function or array header *Object*. *I* must be a fixnum.

`Header-Set (Object I Value)`
sets the *I*'th element of the function of array header *Object* to *Value*, and pushes *Value*. *I* must be a fixnum.

The names of the miscops used to reference and set elements of arrays are based somewhat on the Common Lisp function names. The SVref, SBit, and SChar miscops perform the same operation as their Common Lisp namesakes – referencing elements of simple-vectors, simple-bit-vectors, and simple-strings respectively. Aref1 references any kind of one dimensional array. The names of setting functions

are derived by replacing "ref" with "set", "char" with "charset", and "bit" with "bitset."

`Aref1, SVref, SChar, SBit (Array I)`
: returns the *I*'th element of the one-dimensional array *Array*. SVref pushes an element of a G-Vector; SChar an element of a string; Sbit an element of a Bit-Vector. *I* should be a fixnum.

`Aset1, SVset, SCharset, SBitset (Array I Value)`
: sets the *I*'th element of the one-dimensional array *Array* to *Value*. SVset sets an element of a G-Vector; SCharset an element of a string; SBitset an element of a Bit-Vector. *I* should be a fixnum and *Value* is returned.

`CAref2, CAref3 (Array I1 I2)`
: returns the element (*I1*, *I2*) of the two-dimensional array *Array*. *I1* and *I2* should be fixnums. CAref3 pushes the element (*I1*, *I2*, *I3*).

`CAset2, CAset3 (Array I1 I2 Value)`
: sets the element (*I1*, *I2*) of the two-dimensional array *Array* to *Value* and returns *Value*. *I1* and *I2* should be fixnums. CAset3 sets the element (*I1*, *I2*, *I3*).

`Bit-Bash (V1 V2 V3 Op)`
: *V1*, *V2*, and *V3* should be bit-vectors and *Op* should be a fixnum. The elements of the bit vector *V3* are filled with the result of *Op*'ing the corresponding elements of *V1* and *V2*. *Op* should be a Boole-style number (see the Boole miscop in section ⟨undefined⟩ "[", page ⟨undefined⟩Boole-Section]).

The rest of the miscops in this section implement special cases of sequence or string operations. Where an operand is referred to as a string, it may actually be an 8-bit I-Vector or system area pointer.

`Byte-BLT (Src-String Src-Start Dst-String Dst-Start Dst-End)`
: moves bytes from *Src-String* into *Dst-String* between *Dst-Start* (inclusive) and *Dst-End* (exclusive). *Dst-Start* - *Dst-End* bytes are moved. If the substrings specified overlap, "the right thing happens," i.e. all the characters are moved to the right place. This miscop corresponds to the Common Lisp function REPLACE when the sequences are simple-strings.

`Find-Character (String Start End Character)`
: searches *String* for the *Character* from *Start* to *End*. If the character is found, the corresponding index into *String* is returned, otherwise NIL is returned. This miscop corresponds to the Common Lisp function FIND when the sequence is a simple-string.

`Find-Character-With-Attribute (String Start End Table Mask)`
: The codes of the characters of *String* from *Start* to *End* are used as indices into the *Table*, which is an I-Vector of 8-bit bytes. When the

number picked up from the table bitwise ANDed with *Mask* is non-zero, the current index into the *String* is returned.

`SXHash-Simple-String` (*String Length*)
Computes the hash code of the first *Length* characters of *String* and pushes it on the stack. This corresponds to the Common Lisp function SXHASH when the object is a simple-string. The *Length* operand can be Nil, in which case the length of the string is calculated in assembler.

18.1.6 [Type Predicates]

Many of the miscops described in this sub-section can be coded in-line rather than as miscops. In particular, all the predicates on basic types are coded in-line with default optimization settings in the compiler. Currently, all of these predicates set the eq condition code bit to return an indication of whether the predicate is true or false. This is so that the [No value for "("]DinkyMachine) branch instructions can be used directly without having to test for NIL. However, this only works if the value of the predicate is needed for a branching decision. In the cases where the value is actually needed, T or NIL is generated in-line according to whether the predicate is true or false. At some point it might be worthwhile having two versions of these predicates, one which sets the eq condition code bit, and one which returns T or NIL. This is especially true if space becomes an issue.

`Bit-Vector-P` (*Object*)
sets the eq condition code bit to 1 if *Object* is a Common Lisp bit-vector or 0 if it is not.

`Simple-Bit-Vector-P` (*Object*)
sets the eq condition code bit to 1 if *Object* is a CMU Common Lisp bit-vector or 0 if it is not.

`Simple-Integer-Vector-P` (*Object*)
sets the eq condition code bit to 1 if *Object* is a CMU Common Lisp I-Vector or 0 if it is not.

`StringP` (*Object*)
sets the eq condition code bit to 1 if *Object* is a Common Lisp string or 0 if it is not.

`Simple-String-P` (*Object*)
sets the eq condition code bit to 1 if *Object* is a CMU Common Lisp string or 0 if it is not.

`BignumP` (*Object*)
sets the eq condition code bit to 1 if *Object* is a bignum or 0 if it is not.

`Long-Float-P` (*Object*)
sets the eq condition code bit to 1 if *Object* is a long-float or 0 if it is not.

`ComplexP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a complex number or 0 if it is not.

`RatioP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a ratio or 0 if it is not.

`IntegerP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a fixnum or bignum or 0 if it is not.

`RationalP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a fixnum, bignum, or ratio or 0 if it is not.

`FloatP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a short-float or long-float or 0 if it is not.

`NumberP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a number or 0 if it is not.

`General-Vector-P` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a Common Lisp general vector or 0 if it is not.

`Simple-Vector-P` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a CMU Common Lisp G-Vector or 0 if it is not.

`Compiled-Function-P` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a compiled function or 0 if it is not.

`ArrayP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a Common Lisp array or 0 if it is not.

`VectorP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a Common Lisp vector of 0 if it is not.

`Complex-Array-P` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a CMU Common Lisp array or 0 if it is not.

`SymbolP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a symbol or 0 if it is not.

`ListP` (*Object*)
: sets the eq condition code bit to 1 if *Object* is a cons or NIL or 0 if it is not.

`ConsP (Object)`
sets the eq condition code bit to 1 if *Object* is a cons or 0 if it is not.

`FixnumP (Object)`
sets the eq condition code bit to 1 if *Object* is a fixnum or 0 if it is not.

`Single-Float-P (Object)`
sets the eq condition code bit to 1 if *Object* is a single-float or 0 if it is not.

`CharacterP (Object)`
sets the eq condition code bit to 1 if *Object* is a character or 0 if it is not.

18.1.7 [Arithmetic]

`Integer-Length (Object)`
returns the integer-length (as defined in the Common Lisp manual) of the integer *Object*.

`Logcount (Object)`
returns the number of 1's if *object* is a positive integer, the number of 0's if *object* is a negative integer, and signals an error otherwise.

`Float-Short (Object)`
returns a short-float corresponding to the number *Object*.

`Float-Long (Number)`
returns a long float formed by coercing *Number* to a long float. This corresponds to the Common Lisp function Float when given a long float as its second argument.

`Realpart (Number)`
returns the realpart of the *Number*.

`Imagpart (Number)`
returns the imagpart of the *Number*.

`Numerator (Number)`
returns the numerator of the rational *Number*.

`Denominator (Number)`
returns the denominator of the rational *Number*.

`Decode-Float (Number)`
performs the Common Lisp Decode-Float function, returning 3 values.

`Scale-Float (Number X)`
performs the Common Lisp Scale-Float function, returning the result.

`= (X Y)` sets the condition codes according to whether X is equal to Y. Both X and Y must be numbers, otherwise an error is signalled. If a rational

is compared with a flonum, the rational is converted to a flonum of the same type first. If a short flonum is compared with a long flonum, the short flonum is converted to a long flonum. Flonums must be exactly equal (after conversion) for the condition codes to be set to equality. This miscop also deals with complex numbers.

`Compare (X Y)`
: sets the condition codes according to whether X is less than, equal to, or greater than Y. X and Y must be numbers. Conversions as described in = above are done as necessary. This miscop replaces the `<` and `>` instructions on the Perq, so that the branch on condition instructions can be used more effectively. The value of `<` and `>` as defined for the Perq are only generated if necessary, i.e., the result is saved. If X or Y is a complex number, an error is signalled.

`Truncate (N X)`
: performs the Common Lisp TRUNCATE operation. There are 3 cases depending on X:

- If X is fixnum 1, return two items: a fixnum or bignum representing the integer part of N (rounded toward 0), then either 0 if N was already an integer or the fractional part of N represented as a flonum or ratio with the same type as N.
- If X and N are both fixnums or bignums and X is not 1, divide N by X. Return two items: the integer quotient (a fixnum or bignum) and the integer remainder.
- If either X or N is a flonum or ratio, return a fixnum or bignum quotient (the true quotient rounded toward 0), then a flonum or ratio remainder. The type of the remainder is determined by the same type-coercion rules as for `+`. The value of the remainder is equal to N - X * *Quotient*.

On the [No value for "("]DinkyMachine), the integer part is returned in register A0, and the remainder in A1.

`+, -, *, / (N X)`
: returns N + X. -, *, and / are similar.

`Fixnum*Fixnum, Fixnum/Fixnum (N X)`
: returns N * X, where both N and X are fixnums. Fixnum/ is similar.

`1+ (E)` returns E + 1.

`1- (E)` returns E - 1.

`Negate (N)` returns -N.

`Abs (N)` returns |N|.

`GCD (N X)` returns the greatest common divisor of the integers N and X.

`Logand (N X)`
returns the bitwise and of the integers N and X. Logior and Logxor are analogous.

`Lognot (N)` returns the bitwise complement of N.

`Boole (Op X Y)`
performs the Common Lisp Boole operation Op on X, and Y. The Boole constants for CMU Common Lisp are these:

```
boole-clr        0
boole-set        1
boole-1  2
boole-2  3
boole-c1 4
boole-c2 5
boole-and        6
boole-ior        7
boole-xor        8
boole-eqv        9
boole-nand       10
boole-nor        11
boole-andc1      12
boole-andc2      13
boole-orc1       14
boole-orc2       15
```

`Ash (N X)` performs the Common Lisp ASH operation on N and X.

`Ldb (S P N)`
All args are integers; S and P are non-negative. Performs the Common Lisp LDB operation with S and P being the size and position of the byte specifier.

`Mask-Field (S P N)`
performs the Common Lisp Mask-Field operation with S and P being the size and position of the byte specifier.

`Dpb (V S P N)`
performs the Common Lisp DPB operation with S and P being the size and position of the byte specifier.

`Deposit-Field (V S P N)`
performs the Common Lisp Deposit-Field operation with S and P as the size and position of the byte specifier.

`Lsh (N X)` returns a fixnum that is N shifted left by X bits, with 0's shifted in on the right. If X is negative, N is shifted to the right with 0's coming in on the left. Both N and X should be fixnums.

`Logldb (S P N)`
All args are fixnums. S and P specify a "byte" or bit-field of any length within N. This is extracted and is returned right-justified as a

fixnum. S is the length of the field in bits; P is the number of bits from the right of N to the beginning of the specified field. $P = 0$ means that the field starts at bit 0 of N, and so on. It is an error if the specified field is not entirely within the 26 bits of N

`Logdpb (V S P N)`
: All args are fixnums. Returns a number equal to N, but with the field specified by P and S replaced by the S low-order bits of V. It is an error if the field does not fit into the 26 bits of N.

`Sin(X), Cos(X), Tan(X), and Atan(X)`
: accept a single number X as argument and return the sine, cosine, tangent, and arctangent of the number respectively. These miscops take advantage of the hardware support provided on the IBM RT PC if it is available, otherwise they escape to Lisp code to calculate the appropriate result.

`Log(X)`
: returns the natural log of the number X. This miscop uses the hardware operation if it is available, otherwise it escapes to Lisp code to calculate the result.

`Exp(X)`
: returns e raised to the power X. This miscop uses the hardware operation if it is available, otherwise it escapes to Lisp code to calculate the result.

`Sqrt(X)`
: returns the square root of X. This miscop uses the hardware operation if it is available, otherwise it escapes to Lisp code to calculate the result.

18.1.8 [Branching]

All branching is done with [No value for "("]DinkyMachine) branch instructions. Instructions are generated to set the condition code bits appropriately, and a branch which tests the appropriate condition code bit is generated.

18.1.9 [Function Call and Return]

`Call()`
: A call frame for a function is opened. This is explained in more detail in the next chapter.

`Call-0 (F)`
: F must be an executable function, but is a function of 0 arguments. Thus, there is no need to collect arguments. The call frame is opened and activated in a single miscop.

`Call-Multiple ()`
: Just like a Call miscop, but it marks the frame to indicate that multiple values will be accepted. See section ⟨undefined⟩ "[", page ⟨undefined⟩Multi].

`Set-Up-Apply-Args ()`
: is called to handle the last argument of a function called by apply. All the other arguments will have been properly set up by this time. Set-

up-apply-args places the values of the list passed as the last argument to apply in their proper locations, whether they belong in argument registers or on the stack. It updates the NArgs register with the actual count of the arguments being passed to the function. When Set-up-apply-args returns, all the arguments to the function being applied are in their correct locations, and the function can be invoked normally.

`Start-Call-Interpreter (`*`NArgs`*`)`
is called from the interpreter to start a function call. It accepts the number of arguments that are pushed on the stack in register A0. Just below the arguments is the function to call; just below the function is the area to store the preserved registers. This miscop sets up the argument registers correctly, moves any other arguments down on the stack to their proper place, and invokes the function.

`Invoke1 (`*`Function Argument`*`)`
is similar to Start-Call-Interpreter, but is simpler, since the *Function* is being called with only a single *Argument*.

`Invoke1* (`*`Function Argument`*`)`
is similar to Invoke1, but the *Function* being called is called for one value, rather than multiple ones.

`Start-call-mc ()`
is called when the compiler generates code for the form multiple-value-call. Register A0 contains the function to be called, A1 contains a 0 if the call if for a single value, and 1 otherwise, NArgs contains the number of arguments that are stored on the stack. The argument registers are set up correctly, and the excess values moved down on the stack if necessary. Finally, the function is actually invoked.

`Push-Last ()`
closes the currently open call frame, and initiates a function call.

`Return (`*`X`*`)` Return from the current function call. After the current frame is popped off the stack, *X* is returned in register A0 as the result Being returned. See section ⟨undefined⟩ "[", page ⟨undefined⟩Return] for more details.

`Return-From (`*`X F`*`)`
is similar to Return, except it accepts the frame to return from as an additional argument.

`Return-1-Value-Any-Bind (`*`X`*`)`
is similar to return, except only one value is returned. Any number of bindings are undone during the return operation.

`Return-Mult-Value-0-Bind (`*`X`*`)`
is similar to return, except multiple values may be returned, but the binding stack does not have to be popped.

`Link-Address-Fixup` (*Symbol NArgs Code-Vector Offset*)
: finds the correct link table entry for *Symbol* with *NArgs* (*NArgs* specifies the fixed number of arguments and a flag if more may be passed). It smashes the *Code-Vector* at *Offset* to generate code to point at the absolute address of the link table entry.

`Miscop-Fixup` (*Code-Vector Offset Index*)
: smashes *Code-Vector* at *Offset* with the correct value for the miscop specified by *Index* in a transfer vector of all the miscops.

`Make-Compiled-Closure` (*env fcn offset*)
: returns a new function object that is a copy of the function object *fcn* which has the *env* information stored at *offset*. Compiled lexical closures are now represented as real function objects rather than as lists. This miscop is necessary to support this change.

`Reset-link-table` (*function*)
: resets all the link table entries for *function* to the default action. This is necessary because Portable Commonloops updates generic function objects by copying new information into the function object. The link table must be updated to reflect this or the wrong function will be called.

`Interrupt-Handler` (*Signal Code Signal-Context*)
: gets the first indication that a Unix signal has occurred. This miscop does not follow the normal Lisp calling conventions at all. Instead it follows the standard IBM RT PC calling conventions for C or other algorithmic languages. On entry the registers are as follows:

`R0` Pointer to C data area for Interrupt-Handler. Currently this data area only holds a pointer to the entry point for Interrupt-Handler and nothing else.

`R1` Pointer to a C stack that contains information about the signal.

`R2` Contains the *Signal* number that caused the interrupt to happen.

`R3` Contains the *Code* that further specifies what caused the interrupt (if necessary).

`R4` Contains a pointer to the *signal-context* which contains information about where the interrupt occurred, the saved registers, etc.

`R5-R14` Contain unknown values.

`R15` is the return PC which will return from the interrupt handler and restart the computation.

Interrupt-Handler determines whether it is safe to take the interrupt now, i.e., it is executing in Lisp code, C code, or an interruptible miscop. An interruptible miscop is one that has been specially written to make sure that it is safe to interrupt it at any point and is possible that it will never return of its own accord (e.g., length which could be passed a circular list, some of the system call miscops, etc.). If it is safe to take the interrupt, the signal-context is modified so that control will transfer to the miscop interrupt-routine when the interrupt-handler returns normally (i.e., after the kernel has done the necessary bookkeeping). If it is unsafe to take the interrupt (i.e., it is executing in an non-interruptible miscop), then the return PC of the miscop is modified to be interrupt-routine and interrupt-handler returns to the kernel. In either case interrupts are disabled and information is stored in a global Lisp data area, so that the interrupt-routine miscop can retrieve the important information about the interrupt.

`Interrupt-Routine ()`

gets control when it is safe to take an interrupt. It saves the current state of the computation on the appropriate stack (on the C stack if it was executing in C or on the Lisp stack if in Lisp) including all the registers, some control information specifying whether the computation was in C code, Lisp code, whether it should form a PC in register R15. When a PC has to be formed in R15, R14 will contain a pointer to the active function and R15 will contain an index into the code vector associated with the active function. Reforming the PC is necessary so it is possible to restart a computation even after a garbage collection may have moved the function. Once this information is stored, interrupt-routine invokes the Lisp function %sp-software-interrupt-routine which moves the processing of the interrupt to Lisp code.

`Break-Return ()`

returns from a function called by the interrupt-routine miscop. The only function that should ever do this is %sp-software-interrupt-routine. This miscop expects the stack to be in a format that is generated during an interrupt and should not be used for anything else.

`Catch (`*Tag PC*`)`

builds a catch frame. *Tag* is the tag caught by this catch frame, *PC* is a saved-format PC (i.e., an index into the current code vector). See section ⟨undefined⟩ "[", page ⟨undefined⟩Catch] for details.

`Catch-Multiple (`*Tag PC*`)`

builds a multiple-value catch frame. *Tag* is the tag caught by this catch frame, and *PC* is a saved-format PC. See section ⟨undefined⟩ "[", page ⟨undefined⟩Catch] for details.

`Catch-All (PC)`
: builds a catch frame whose tag is the special Catch-All object. *PC* is the saved-format PC, which is the address to branch to if this frame is thrown through. See section ⟨undefined⟩ "[", page ⟨undefined⟩Catch] for details.

`Throw (X Tag)`
: *Tag* is the throw-tag, normally a symbol. *X* is the value to be returned. See section ⟨undefined⟩ "[", page ⟨undefined⟩Catch] for a description of how this miscop works.

`Rest-Entry-0, Rest-Entry-1, Rest-Entry-2, Rest-Entry`
: are miscops that do the processing for a function at its &rest entry point. Rest-Entry-*i* are miscops that are invoked by functions that have 0, 1, or 2 arguments before the &rest argument. Rest-entry is invoked for all other cases, and is passed an additional argument in A3 which is the number of non-&rest arguments. These miscops form the &rest arg list and set up all the registers to have the appropriate values. In particular, the non-&rest arguments are copied into preserved registers, and the &rest arg list is built and stored in the appropriate preserved register or on the stack as appropriate.

`Call-Foreign (C-Function Arguments NArgs)`
: establishes the C environment so that C code can be called correctly. *C-Function* is a pointer to the data area for a C function, the first word of which is a pointer to the entry point of the C function. *Arguments* is a block of storage that contains the *NArgs* arguments to be passed to the C function. The first four of these arguments are passed in registers R2 through R5 respectively, the rest are moved onto the C stack in the proper location. When the C function returns, Call-Foreign restores the Lisp environment and returns as its value the integer in R2.

`Call-Lisp (Arg<1> ... Arg<2>)`
: is a Lisp miscop that gets control when a C function needs to call a Lisp function. Lisp provides a mechanism for setting up an object that looks like a C procedure pointer. The code pointer in this object always points at Call-Lisp. Additional data in this procedure pointer is the Lisp function to call and the number of arguments that it should be called with. Call-Lisp restores the Lisp environment, saves the state of the C computation, moves the C arguments into the correct places for a call to a Lisp function and then invokes the special Lisp function call-lisp-from-c. This Lisp function actually invokes the correct Lisp function. Call-Lisp never regains control.

`Return-To-C (C-Stack-Pointer Value)`
: is used in the function call-lisp-from-c to return control to C from a Lisp function called by C. *C-Stack-Pointer* is the C stack pointer when the call-lisp miscop got control. The C stack pointer argument is used to restore the C environment to what it was at the time the call to

Lisp was made. *Value* is the value returned from Lisp and is passed back to C in register R2. Currently, it is not possible to return other than a single 32 bit quantity.

`Reset-C-Stack ()`
is invoked when a Lisp function called by C throws out past where it should return to C. Reset-C-Stack restores the C stack to what it was before the original call to C happened. This is so that in the future, the C stack will not contain any garbage that should not be there.

`Set-C-Procedure-Pointer (`*`Sap I Proc`*`)`
sets the *I/2*'th element of *sap* to be the data part of the statically allocated g-vector *Proc*. This is used to set up a C procedure argument in the argument block that is passed to call-foreign.

18.1.10 [Miscellaneous]

`Eq (`*`X Y`*`)` sets the eq condition code bit to 1 if *X* and *Y* are the same object, 0 otherwise.

`Eql (`*`X Y`*`)` sets the eq condition code bit to 1 if *X* and *Y* are the same object or if *X* and *Y* are numbers of the same type with the same value, 0 otherwise.

`Make-Predicate (`*`X`*`)`
returns NIL if *X* is NIL or T if it is not.

`Not-Predicate (`*`X`*`)`
returns T if *X* is NIL or NIL if it is not.

`Values-To-N (`*`V`*`)`
V must be a Values-Marker. Returns the number of values indicated in the low 24 bits of *V* as a fixnum.

`N-To-Values (`*`N`*`)`
N is a fixnum. Returns a Values-Marker with the same low-order 24 bits as *N*.

`Force-Values (`*`VM`*`)`
If the *VM* is a Values-Marker, do nothing; if not, push *VM* and return a Values-Marker 1.

`Flush-Values ()`
is a no-op for the [No value for "("]DinkyMachine), since the only time that a Flush-Values miscop is generated is in some well-defined cases where all the values are wanted on the stack.

18.1.11 [System Hacking]

`Get-Type (`*`Object`*`)`
returns the five type bits of the *Object* as a fixnum.

`Get-Space (Object)`
returns the two space bits of *Object* as a fixnum.

`Make-Immediate-Type (X A)`
returns an object whose type bits are the integer A and whose other bits come from the immediate object or pointer X. A should be an immediate type code.

`8bit-System-Ref (X I)`
X must be a system area pointer, returns the I'th byte of X, indexing into X directly. I must be a fixnum.

`8bit-System-Set (X I V)`
X must be a system area pointer, sets the I'th element of X to V, indexing into X directly.

`16bit-System-Ref (X I)`
X must be a system area pointer, returns the I'th 16-bit word of X, indexing into X directly.

`Signed-16bit-System-Ref (X I)`
X must be a system area pointer, returns the I'th 16-bit word of X extending the high order bit as the sign bit.

`16bit-System-Set (X I V)`
X must be a system area pointer, sets the I'th element of X to V, indexing into X directly.

`Signed-32bit-System-Ref (X I)`
X must be a system area pointer and I an even fixnum, returns the $I/2$'th 32 bit word as a signed quantity.

`Unsigned-32bit-System-Ref (X I)`
X must be a system area pointer and I an even fixnum, returns the $I/2$'th 32 bit word as an unsigned quantity.

`Signed-32bit-System-Set (X I V)`
X must be a system area pointer, I an even fixnum, and V an integer, sets the $I/2$'th element of X to V.

`Sap-System-Ref (X I)`
X must be a system area pointer and I and even fixnum, returns the $I/2$'th element of X as a system area pointer.

`Sap-System-Set (X I V)`
X and V must be a system area pointers and I an even fixnum, sets the $I/2$'th element of X to V.

`Pointer-System-Set (X I)`
X must be a system area pointer, I an even fixnum, and V a pointer (either system area pointer or Lisp pointer), sets the $I/2$'th element of X to the pointer V. If the pointer is a Lisp pointer, the pointer stored is to the first word of data (i.e., the header word(s) are bypassed).

`Sap-Int (X)`
: *X* should be a system area pointer, returns a Lisp integer containing the system area pointer. This miscop is useful when it is necessary to do arithmetic on system area pointers.

`Int-Sap (X)`
: *X* should be an integer (fixnum or bignum), returns a system area pointer. This miscop performs the inverse operation of sap-int.

`Check-<= (X Y)`
: checks to make sure that *X* is less than or equal to *Y*. If not, then check-<= signals an error, otherwise it just returns.

`Collect-Garbage ()`
: causes a stop-and-copy GC to be performed.

`Purify ()`
: is similar to collect-garbage, except it copies Lisp objects into static or read-only space. This miscop needs Lisp level code to get the process started by putting some root structures into the correct space.

`Newspace-Bit ()`
: returns 0 if newspace is currently space 0 or 1 if it is 1.

`Save (*current-alien-free-pointer* Checksum memory)`
: Save takes a snap short of the current state of the Lisp computation. The value of the symbol *Current-alien-free-pointer* must be passed to save, so that it can save the static alien data structures. The parameter *checksum* specifies whether a checksum should be generated for the saved image. Currently, this parameter is ignored and no checksum is generated. The parameter *memory* should be be a pointer to a block of memory where the saved core image will be stored. Save returns the size of the core image generated.

Syscall0 Syscall1 Syscall2 Syscall3 Syscall4 Syscall (*number*

`arg<1> ... arg<n>)`
: is for making syscalls to the Mach kernel. The argument *number* should be the number of the syscall. Syscall0 accepts no arguments to the syscall; syscall1 accepts one argument to the syscall, etc. Syscall accepts five or more arguments to the syscall.

`Unix-Write (fd buffer offset length)`
: performs a Unix write syscall to the file descriptor *fd*. *Buffer* should contain the data to be written; *Offset* should be an offset into buffer from which to start writing; and *length* is the number of bytes of data to write.

`Unix-Fork ()`
: performs a Unix fork operation returning one or two values. If an error occurred, the value -1 and the error code is returned. If no error occurred, 0 is returned in the new process and the process id of the child process is returned in the parent process.

`Arg-In-Frame (N F)`
: N is a fixnum, F is a control stack pointer as returned by the Active-Call-Frame miscop. It returns the item in slot N of the args-and-locals area of call frame F.

`Active-Call-Frame ()`
: returns a control-stack pointer to the start of the currently active call frame. This will be of type Control-Stack-Pointer.

`Active-Catch-Frame ()`
: returns the control-stack pointer to the start of the currently active catch frame. This is Nil if there is no active catch.

`Set-Call-Frame (P)`
: P must be a control stack pointer. This becomes the current active call frame pointer.

`Current-Stack-Pointer ()`
: returns the Control-Stack-Pointer that points to the current top of the stack (before the result of this operation is pushed). Note: by definition, this points to the to the last thing pushed.

`Current-Binding-Pointer ()`
: returns a Binding-Stack-Pointer that points to the first word above the current top of the binding stack.

`Read-Control-Stack (F)`
: F must be a control stack pointer. Returns the Lisp object that resides at this location. If the addressed object is totally outside the current stack, this is an error.

`Write-Control-Stack (F V)`
: F is a stack pointer, V is any Lisp object. Writes V into the location addressed. If the addressed cell is totally outside the current stack, this is an error. Obviously, this should only be used by carefully written and debugged system code, since you can destroy the world by using this miscop.

`Read-Binding-Stack (B)`
: B must be a binding stack pointer. Reads and returns the Lisp object at this location. An error if the location specified is outside the current binding stack.

`Write-Binding-Stack (B V)`
: B must be a binding stack pointer. Writes V into the specified location. An error if the location specified is outside the current binding stack.

19 [Control Conventions]

19.1 [Function Calls]

On the Perq function calling is done by micro-coded instructions. The instructions perform a large number of operations, including determining whether the function being called is compiled or interpreted, determining that a legal number of arguments are passed, and branching to the correct entry point in the function. To do all this on the [No value for "("]DinkyMachine) would involve a large amount of computation. In the general case, it is necessary to do all this, but in some common cases, it is possible to short circuit most of this work.

To perform a function call in the general case, the following steps occur:

1. Allocate space on the control stack for the fix-sized part of a call frame. This space will be used to store all the registers that must be preserved across a function call.
2. Arguments to the function are now evaluated. The first three arguments are stored in the argument registers A0, A1, and A2. The rest of the arguments are stored on the stack as they are evaluated. Note that during the evaluation of arguments, the argument registers may be used and may have to be stored in local variables and restored just before the called function is invoked.
3. Load R0 with the argument count.
4. Load the PC register with the offset into the current code vector of the place to return to when the function call is complete.
5. If this call is for multiple values, mark the frame as accepting multiple values, by making the fixnum offset above negative by oring in the negative fixnum type code.
6. Store all the registers that must be preserved over the function call in the current frame.

At this point, all the arguments are set up and all the registers have been saved. All the code to this point is done inline. If the object being called as a function is a symbol, we get the definition from the definition cell of the symbol. If this definition is the trap object, an undefined symbol error is generated. The function calling mechanism diverges at this point depending on the type of function being called, i.e., whether it is a compiled function object or a list.

If we have a compiled function object, the following steps are performed (this code is out of line):

Load the active function register with a pointer to the compiled function object.

The active frame register is set to the start of the current frame.

Note the number of arguments evaluated. Let this be K. The correct entry point in the called function's code vector must be computed as a function of K and the number of arguments the called function wants:

If K < minimum number of arguments, signal an error.

If K > maximum number of arguments and there is no &rest argument, signal an error.

If K > maximum number of arguments and there is a &rest argument, start at offset 0 in the code vector. This entry point must collect the excess arguments into a list and leave the &rest argument in the appropriate argument register or on the stack as appropriate.

If K is between the minimum and maximum arguments (inclusive), get the starting offset from the appropriate slot of the called function's function object. This is stored as a fixnum in slot K - MIN + 6 of the function object.

Load one of the Non-Lisp temporary registers with the address of the code vector and add in the offset calculated above. Then do a branch register instruction with this register as the operand. The called function is now executing at the appropriate place.

If the function being called is a list, %SP-Internal-Apply must be called to interpret the function with the given arguments. Proceed as follows:

Note the number of arguments evaluated for the current open frame (call this N) and the frame pointer for the frame (call it F). Also remember the lambda expression in this frame (call it L).

Load the active function register with the list L.

Load the PC register with 0.

Allocate a frame on the control stack for the call to %SP-Internal-Apply.

Move the contents of the argument registers into the local registers L0, L1, and L2 respectively.

Store all the preserved registers in the frame.

Place N, F, and L into argument registers A0, A1, and A2 respectively.

Do the equivalent of a start call on %SP-Internal-Apply.

%SP-Internal-Apply, a function of three arguments, now evaluates the call to the lambda-expression or interpreted lexical closure L, obtaining the arguments from the frame pointed to by F. The first three arguments must be obtained from the frame that %SP-Internal-Apply runs in, since they are stored in its stack frame and not on the stack as the rest of the arguments are. Prior to returning %SP-Internal-Apply sets the Active-Frame register to F, so that it returns from frame F.

The above is the default calling mechanism. However, much of the overhead can be reduced. Most of the overhead is incurred by having to check the legality of the function call everytime the function is called. In many situations where the function being called is a symbol, this checking can be done only once per call site by introducing a data structure called a link table. The one exception to this rule is when the function apply is used with a symbol. In this situation, the argument count checks are still necessary, but checking for whether the function is a list or compiled function object can be bypassed.

The link table is a hash table whose key is based on the name of the function, the number of arguments supplied to the call and a flag specifying whether the call is done through apply or not. Each entry of the link table consists of two words:

> The address of the function object associated with the symbol being called. This is here, so that double indirection is not needed to access the function object which must be loaded into the active function register. Initially, the symbol is stored in this slot.
>
> The address of the instruction in the function being called to start executing when this table entry is used. Initially, this points to an out of line routine that checks the legality of the call and calculates the correct place to jump to in the called function. This out of line routine replaces the contents of this word with the correct address it calculated. In the case when the call is caused by apply, this will often be an out of line routine that checks the argument count and calculates where to jump. In the case where the called function accepts &rest arguments and the minimum number of arguments passed is guaranteed to be greater than the maximum number of arguments, then a direct branch to the &rest arg entry point is made.

When a compiled file is loaded into the lisp environment, all the entries for the newly loaded functions will be set to an out of line routine mentioned above. Also, during a garbage collection the entries in this table must be updated when a function object for a symbol is moved.

The [No value for "("]DinkyMachine) code to perform a function call using the link table becomes:

```
cal CS,CS,%Frame-Size ; Alloc. space on control st.

<Code to evaluate arguments to the function>

cau NL1,0,high-half-word(lte(function nargs flag))
oil NL1,0,low-half-word(lte(function nargs flag))
cal PC,0,return-tag  ; Offset into code vector.
       <oiu PC,PC,#xF800 ; Mark if call-multiple frame>
stm L0,CS,-(%Frame-Size-4) ; Save preserved regs.
lm AF,NL1,0  ; Link table entry contents.
bnbrx pz,R15 ; Branch to called routine.
cal FP,CS,-(%Frame-Size-4) ; Get pointer to frame.
return-tag:
```

The first two instructions after the arguments are evaled get the address of the link table entry into a register. The two 16-bit half word entries are filled in at load time. The rest of the instructions should be fairly straight forward.

19.2 (Returning from a Function Call)

Returning from a function call on the Perq is done by a micro-coded instruction. On the [No value for "("]DinkyMachine), return has to do the following:

Pop the binding stack back to the binding stack pointer stored in the frame we're returning from. For each symbol/value pair popped of the binding stack, restore that value for the symbol.

Save the current value of the frame pointer in a temporary registers. This will be used to restore the control stack pointer at the end.

Restore all the registers that are preserved across a function call.

Get a pointer to the code vector for the function we're returning to. This is retrieved from the code slot of what is now the active function.

Make sure the relative PC (which is now in a register) is positive and add it to the code vector pointer above, giving the address of the instruction to return to.

If the function is returning multiple values do a block transfer of all the return values down over the stack frame just released, i.e., the first return value should be stored where the temporarily saved frame pointer points to. In effect the return values can be pushed onto the stack using the saved frame pointer above as a stack pointer that is incremented everytime a value is pushed. Register A0 can be examined to determine the number of values that must be transferred.

Set the control stack register to the saved frame pointer above. NB: it may have been updated if multiple values are being returned.

Resume execution of the calling function.

Again, it is not always necessary to use the general return code. At compile time it is often possible to determine that no special symbols have to be unbound and/or only one value is being returned. For example the code to perform a return when only one value is returned and it is unnecessary to unbind any special symbols is:

```
cas NL1,FP,0 ; Save frame register.
lm L0,FP,0 ; Restore all preserved regs.
ls A3,AF,%function-code ; Get pointer to code vector.
niuo PC,PC,#x07FF ; Make relative PC positive.
cas PC,A3,PC ; Get addr. of instruction
bnbrx pz,PC ; to return to and do so while
cas CS,NL1,0 ; updating control stack reg.
```

19.2.1 [Returning Multiple-Values]

If the current frame can accept multiple values and a values marker is in register A0 indicating N values on top of the stack, it is necessary to copy the N return values down to the top of the control stack after the current frame is popped off. Thus returning multiple values is similar to the above, but a block transfer is necessary to move the returned values down to the correct location on the control stack.

In tail recursive situations, such as in the last form of a PROGN, one function, FOO, may want to call another function, BAR, and return "whatever BAR returns." Call-Multiple is used in this case. If BAR returns multiple values, they will all be passed to FOO. If FOO's caller wants multiple values, the values will be returned. If not, FOO's Return instruction will see that there are multiple values on the stack,

but that multiple values will not be accepted by FOO's caller. So Return will return only the first value.

19.3 [Non-Local Exits]

The Catch and Unwind-Protect special forms are implemented using catch frames. Unwind-Protect builds a catch frame whose tag is the Catch-All object. The Catch miscop creates a catch frame for a given tag and PC to branch to in the current instruction. The Throw miscop looks up the stack by following the chain of catch frames until it finds a frame with a matching tag or a frame with the Catch-All object as its tag. If it finds a frame with a matching tag, that frame is "returned from," and that function is resumed. If it finds a frame with the Catch-All object as its tag, that frame is "returned from," and in addition, %SP-Internal-Throw-Tag is set to the tag being searched for. So that interrupted cleanup forms behave correctly, %SP-Internal-Throw-Tag should be bound to the Catch-All object before the Catch-All frame is built. The protected forms are then executed, and if %SP-Internal-Throw-Tag is not the Catch-All object, its value is thrown to. Exactly what we do is this:

Put the contents of the Active-Catch register into a register, A. Put NIL into another register, B.

If A is NIL, the tag we seek isn't on the stack. Signal an Unseen-Throw-Tag error.

Look at the tag for the catch frame in register A. If it's the tag we're looking for, go to step 4. If it's the Catch-All object and B is NIL, copy A to B. Set A to the previous catch frame and go back to step 2.

If B is non-NIL, we need to execute some cleanup forms. Return into B's frame and bind %SP-Internal-Throw-Tag to the tag we're searching for. When the cleanup forms are finished executing, they'll throw to this tag again.

If B is NIL, return into this frame, pushing the return value (or BLTing the multiple values if this frame accepts multiple values and there are multiple values).

If no form inside of a Catch results in a Throw, the catch frame needs to be removed from the stack before execution of the function containing the throw is resumed. For now, the value produced by the forms inside the Catch form are thrown to the tag. Some sort of specialized miscop could be used for this, but right now we'll just go with the throw. The branch PC specified by a Catch miscop is part of the constants area of the function object, much like the function's entry points.

19.4 [Escaping to Lisp code]

Escaping to Lisp code is fairly straight forward. If a miscop discovers that it needs to call a Lisp function, it creates a call frame on the control stack and sets it up so that the called function returns to the function that called the miscop. This means it is impossible to return control to a miscop from a Lisp function.

19.5 [Errors]

When an error occurs during the execution of a miscop, a call to %SP-Internal-Error is performed. This call is a break-type call, so if the error is proceeded (with a Break-Return instruction), no value will be returned.

%SP-Internal-Error is passed a fixnum error code as its first argument. The second argument is a fixnum offset into the current code vector that points to the location immediately following the instruction that encountered the trouble. From this offset, the Lisp-level error handler can reconstruct the PC of the losing instruction, which is not readily available in the micro-machine. Following the offset, there may be 0 - 2 additional arguments that provide information of possible use to the error handler. For example, an unbound-symbol error will pass the symbol in question as the third arg.

The following error codes are currently defined. Unless otherwise specified, only the error code and the code-vector offset are passed as arguments.

`1 Object Not List`
: The object is passed as the third argument.

`2 Object Not Symbol`
: The object is passed as the third argument.

`3 Object Not Number`
: The object is passed as the third argument.

`4 Object Not Integer`
: The object is passed as the third argument.

`5 Object Not Ratio`
: The object is passed as the third argument.

`6 Object Not Complex`
: The object is passed as the third argument.

`7 Object Not Vector`
: The object is passed as the third argument.

`8 Object Not Simple Vector`
: The object is passed as the third argument.

`9 Illegal Function Object`
: The object is passed as the third argument.

`10 Object Not Header`
: The object (which is not an array or function header) is passed as the third argument.

`11 Object Not I-Vector`
: The object is passed as the third argument.

`12 Object Not Simple Bit Vector`
: The object is passed as the third argument.

`13 Object Not Simple String`
The object is passed as the third argument.

`14 Object Not Character`
The object is passed as the third argument.

`15 Object Not Control Stack Pointer`
The object is passed as the third argument.

`16 Object Not Binding Stack Pointer`
The object is passed as the third argument.

`17 Object Not Array`
The object is passed as the third argument.

`18 Object Not Non-negative Fixnum`
The object is passed as the third argument.

`19 Object Not System Area Pointer`
The object is passed as the third argument.

`20 Object Not System Pointer`
The object is passed as the third argument.

`21 Object Not Float`
The object is passed as the third argument.

`22 Object Not Rational`
The object is passed as the third argument.

`23 Object Not Non-Complex Number`
A complex number has been passed to the comparison routine for `<` or `>`. The complex number is passed as the third argument.

`25 Unbound Symbol`
Attempted access to the special value of an unbound symbol. Passes the symbol as the third argument to %Sp-Internal-Error.

`26 Undefined Symbol`
Attempted access to the definition cell of an undefined symbol. Passes the symbol as the third argument to %Sp-Internal-Error.

`27 Altering NIL`
Attempt to bind or setq the special value of NIL.

`28 Altering T`
Attempt to bind or setq the special value of T.

`30 Illegal Vector Access Type`
The specified access type is returned as the third argument.

`31 Illegal Vector Size`
Attempt to allocate a vector with negative size or size too large for vectors of this type. Passes the requested size as the third argument.

`32 Vector Index Out of Range`
The specified index is out of bounds for this vector. The bad index is passed as the third argument.

`33 Illegal Vector Index`
The specified index is not a positive fixnum. The bad index is passed as the third argument.

`34 Illegal Shrink Vector Value`
The specified value to shrink a vector to is not a positive fixnum. The bad value is passed as the third argument.

`35 Not A Shrink`
The specified value is greater than the current size of the vector being shrunk. The bad value is passed as the third argument.

`36 Illegal Data Vector`
The data vector of an array is illegal. The bad vector is passed as the third value.

`37 Array has Too Few Indices`
An attempt has been made to access an array as a two or three dimensional array when it has fewer than two or three dimensions, respectively.

`38 Array has Too Many Indices`
An attempt has been made to access an array as a two or three dimensional array when it has more than two or three dimensions, respectively.

`40 Illegal Byte Specifier`
A bad byte specifier has been passed to one of the byte manipulation miscops. The offending byte specifier is passed as the third argument.

`41 Illegal Position in Byte Specifier`
A bad position has been given in a byte specifier that has been passed to one of the byte manipulation miscops. The offending byte specifier is passed as the third argument.

`42 Illegal Size in Byte Specifier`
A bad size has been given in a byte specifier that has been passed to one of the byte manipulation miscops. The offending byte specifier is passed as the third argument.

`43 Illegal Shift Count`
A shift miscop has encountered non fixnum shift count. The offending shift count is passed as the third argument.

`44 Illegal Boole Operation`
The operation code passed to the boole miscop is either not a fixnum or is out of range. The operation code is passed as the third argument.

`50 Too Few Arguments`
Too few arguments have been passed to a function. The number of arguments actually passed is passed as the third argument, and the function is passed as the fourth.

`51 Too Many Arguments`
Too many arguments have been passed to a function. The number of arguments actually passed is passed as the third argument, and the function is passed as the fourth.

`52 Last Apply Arg Not a List`
The last argument to a function being invoked by apply is not a list. The last argument is passed as the third argument.

`53 Deleted Link Table Entry`
An attempt has been made to call a function through a link table entry which no longer exists. This is a serious internal error and should never happen.

`55 Error Not <=`
The check-<= miscop will invoke this error if the condition is false. The two arguments are passed as the third and fourth arguments to %SP-internal-error.

`60 Divide by 0`
An division operation has done a division by zero. The two operands are passed as the third and fourth arguments.

`61 Unseen Throw Tag`
An attempt has been made to throw to a tag that is not in the current catch hierarchy. The offending tag is passed as the third argument.

`62 Short Float Underflow`
A short float operation has resulted in underflow. The two arguments to the operation are passed as the third and fourth arguments.

`63 Short Float Overflow`
A short float operation has resulted in overflow. The two arguments to the operation are passed as the third and fourth arguments.

`64 Single Float Underflow`
A single float operation has resulted in underflow. The two arguments to the operation are passed as the third and fourth arguments.

`65 Single Float Overflow`
A single float operation has resulted in overflow. The two arguments to the operation are passed as the third and fourth arguments.

`66 Long Float Underflow`
A long float operation has resulted in underflow. The two arguments to the operation are passed as the third and fourth arguments.

`67 Long Float Overflow`
A long float operation has resulted in overflow. The two arguments to the operation are passed as the third and fourth arguments.

`68 Monadic Short Float Underflow`
A short float operation has resulted in underflow. The argument to the operation is passed as the third argument.

`69 Monadic Short Float Overflow`
A short float operation has resulted in overflow. The argument to the operation is passed as the third argument.

`70 Monadic Long Float Underflow`
A long float operation has resulted in underflow. The argument to the operation is passed as the third argument.

`71 Monadic Long Float Overflow`
A long float operation has resulted in overflow. The argument to the operation is passed as the third argument.

19.6 [Trapping to the Mach Kernel]

Trapping to the Mach kernel is done through one of the syscall0, syscall1, syscall2, syscall3, syscall4, or syscall miscops. The first argument to these miscops is the number of the Unix syscall that is to be invoked. Any other arguments the syscall requires are passed in order after the first one. Syscall0 accepts only the syscall number and no other arguments; syscall1 accepts the syscall number and a single argument to the syscall; etc. Syscall accepts the syscall number and five or more arguments to the Unix syscall. These syscalls generally return two values: the result twice if the syscall succeeded and a -1 and the Unix error code if the syscall failed.

19.7 [Interrupts]

An interface has been built to the general signal mechanism defined by the Unix operating system. As mentioned in the section on function call and return miscops, several miscops are defined that support the lowest level interface to the Unix signal mechanism. The manual *CMU Common Lisp User's Manual, Mach/IBM RT PC Edition* contains descriptions of functions that allow a user to set up interrupt handlers for any of the Unix signals from within Lisp.

Appendix A [Fasload File Format]

A.1 [General]

The purpose of Fasload files is to allow concise storage and rapid loading of Lisp data, particularly function definitions. The intent is that loading a Fasload file has the same effect as loading the ASCII file from which the Fasload file was compiled, but accomplishes the tasks more efficiently. One noticeable difference, of course, is that function definitions may be in compiled form rather than S-expression form. Another is that Fasload files may specify in what parts of memory the Lisp data should be allocated. For example, constant lists used by compiled code may be regarded as read-only.

In some Lisp implementations, Fasload file formats are designed to allow sharing of code parts of the file, possibly by direct mapping of pages of the file into the address space of a process. This technique produces great performance improvements in a paged time-sharing system. Since the Mach project is to produce a distributed personal-computer network system rather than a time-sharing system, efficiencies of this type are explicitly *not* a goal for the CMU Common Lisp Fasload file format.

On the other hand, CMU Common Lisp is intended to be portable, as it will eventually run on a variety of machines. Therefore an explicit goal is that Fasload files shall be transportable among various implementations, to permit efficient distribution of programs in compiled form. The representations of data objects in Fasload files shall be relatively independent of such considerations as word length, number of type bits, and so on. If two implementations interpret the same macrocode (compiled code format), then Fasload files should be completely compatible. If they do not, then files not containing compiled code (so-called "Fasdump" data files) should still be compatible. While this may lead to a format which is not maximally efficient for a particular implementation, the sacrifice of a small amount of performance is deemed a worthwhile price to pay to achieve portability.

The primary assumption about data format compatibility is that all implementations can support I/O on finite streams of eight-bit bytes. By "finite" we mean that a definite end-of-file point can be detected irrespective of the content of the data stream. A Fasload file will be regarded as such a byte stream.

A.2 [Strategy]

A Fasload file may be regarded as a human-readable prefix followed by code in a funny little language. When interpreted, this code will cause the construction of the encoded data structures. The virtual machine which interprets this code has a *stack* and a *table*, both initially empty. The table may be thought of as an expandable register file; it is used to remember quantities which are needed more than once. The elements of both the stack and the table are Lisp data objects. Operators of the funny language may take as operands following bytes of the data stream, or items popped from the stack. Results may be pushed back onto the stack or pushed onto the table. The table is an indexable stack that is never popped; it is indexed

relative to the base, not the top, so that an item once pushed always has the same index.

More precisely, a Fasload file has the following macroscopic organization. It is a sequence of zero or more groups concatenated together. End-of-file must occur at the end of the last group. Each group begins with a series of seven-bit ASCII characters terminated by one or more bytes of all ones (FF(16)); this is called the *header*. Following the bytes which terminate the header is the *body*, a stream of bytes in the funny binary language. The body of necessity begins with a byte other than FF(16). The body is terminated by the operation `[FOP-END-GROUP]`.

The first nine characters of the header must be `"FASL FILE"` in upper-case letters. The rest may be any ASCII text, but by convention it is formatted in a certain way. The header is divided into lines, which are grouped into paragraphs. A paragraph begins with a line which does *not* begin with a space or tab character, and contains all lines up to, but not including, the next such line. The first word of a paragraph, defined to be all characters up to but not including the first space, tab, or end-of-line character, is the *name* of the paragraph. A Fasload file header might look something like this:

```
FASL FILE >SteelesPerq>User>Guy>IoHacks>Pretty-Print.Slisp
Package Pretty-Print
Compiled 31-Mar-1988 09:01:32 by some random luser
Compiler Version 1.6, Lisp Version 3.0.
Functions: INITIALIZE DRIVER HACK HACK1 MUNGE MUNGE1 GAZORCH
            MINGLE MUDDLE PERTURB OVERDRIVE GOBBLE-KEYBOARD
            FRY-USER DROP-DEAD HELP CLEAR-MICROCODE
             %AOS-TRIANGLE %HARASS-READTABLE-MAYBE
Macros:    PUSH POP FROB TWIDDLE
@r[<one or more bytes of FF@-(16)>]
```

The particular paragraph names and contents shown here are only intended as suggestions.

A.3 [Fasload Language]

Each operation in the binary Fasload language is an eight-bit (one-byte) opcode. Each has a name beginning with `"FOP-"`. In the following descriptions, the name is followed by operand descriptors. Each descriptor denotes operands that follow the opcode in the input stream. A quantity in parentheses indicates the number of bytes of data from the stream making up the operand. Operands which implicitly come from the stack are noted in the text. The notation "@PushArrow stack" means that the result is pushed onto the stack; "@PushArrow table" similarly means that the result is added to the table. A construction like "*n*(1) *value*(*n*)" means that first a single byte *n* is read from the input stream, and this byte specifies how many bytes to read as the operand named *value*. All numeric values are unsigned binary integers unless otherwise specified. Values described as "signed" are in two's-complement form unless otherwise specified. When an integer read from the stream occupies more than one byte, the first byte read is the least significant byte, and the last

byte read is the most significant (and contains the sign bit as its high-order bit if the entire integer is signed).

Some of the operations are not necessary, but are rather special cases of or combinations of others. These are included to reduce the size of the file or to speed up important cases. As an example, nearly all strings are less than 256 bytes long, and so a special form of string operation might take a one-byte length rather than a four-byte length. As another example, some implementations may choose to store bits in an array in a left-to-right format within each word, rather than right-to-left. The Fasload file format may support both formats, with one being significantly more efficient than the other for a given implementation. The compiler for any implementation may generate the more efficient form for that implementation, and yet compatibility can be maintained by requiring all implementations to support both formats in Fasload files.

Measurements are to be made to determine which operation codes are worthwhile; little-used operations may be discarded and new ones added. After a point the definition will be "frozen", meaning that existing operations may not be deleted (though new ones may be added; some operations codes will be reserved for that purpose).

0 *FOP-NOP* No operation. (This is included because it is recognized that some implementations may benefit from alignment of operands to some operations, for example to 32-bit boundaries. This operation can be used to pad the instruction stream to a desired boundary.)

1 *FOP-POP* `@PushArrow table`
One item is popped from the stack and added to the table.

2 *FOP-PUSH* *index*(4) `@PushArrow stack`
Item number *index* of the table is pushed onto the stack. The first element of the table is item number zero.

3 *FOP-BYTE-PUSH* *index*(1) `@PushArrow stack`
Item number *index* of the table is pushed onto the stack. The first element of the table is item number zero.

4 *FOP-EMPTY-LIST* `@PushArrow stack`
The empty list (*()*) is pushed onto the stack.

5 *FOP-TRUTH* `@PushArrow stack`
The standard truth value (*T*) is pushed onto the stack.

6 *FOP-SYMBOL-SAVE* *n*(4) *name*(*n*)

`@PushArrow stack & table`
The four-byte operand *n* specifies the length of the print name of a symbol. The name follows, one character per byte, with the first byte of the print name being the first read. The name is interned in the default package, and the resulting symbol is both pushed onto the stack and added to the table.

7 *FOP-SMALL-SYMBOL-SAVE* `n(1)` *name*(*n*) `@PushArrow stack & table`

The one-byte operand *n* specifies the length of the print name of a symbol. The name follows, one character per byte, with the first byte of the print name being the first read. The name is interned in the default package, and the resulting symbol is both pushed onto the stack and added to the table.

8 *FOP-SYMBOL-IN-PACKAGE-SAVE* *index*(4) *n*(4) *name*(*n*)

`@PushArrow stack & table`

The four-byte *index* specifies a package stored in the table. The four-byte operand *n* specifies the length of the print name of a symbol. The name follows, one character per byte, with the first byte of the print name being the first read. The name is interned in the specified package, and the resulting symbol is both pushed onto the stack and added to the table.

9 *FOP-SMALL-SYMBOL-IN-PACKAGE-SAVE* *index*(4) *n*(1) *name*(*n*)

`@PushArrow stack & table`

The four-byte *index* specifies a package stored in the table. The one-byte operand *n* specifies the length of the print name of a symbol. The name follows, one character per byte, with the first byte of the print name being the first read. The name is interned in the specified package, and the resulting symbol is both pushed onto the stack and added to the table.

10 *FOP-SYMBOL-IN-BYTE-PACKAGE-SAVE* *index*(1) *n*(4) *name*(*n*)

`@PushArrow stack & table`

The one-byte *index* specifies a package stored in the table. The four-byte operand *n* specifies the length of the print name of a symbol. The name follows, one character per byte, with the first byte of the print name being the first read. The name is interned in the specified package, and the resulting symbol is both pushed onto the stack and added to the table.

11 *FOP-SMALL-SYMBOL-IN-BYTE-PACKAGE-SAVE* *index*(1) *n*(1) *name*(*n*)

`@PushArrow stack & table`

The one-byte *index* specifies a package stored in the table. The one-byte operand *n* specifies the length of the print name of a symbol. The name follows, one character per byte, with the first byte of the print name being the first read. The name is interned in the specified package, and the resulting symbol is both pushed onto the stack and added to the table.

12 Unused.

13 *FOP-DEFAULT-PACKAGE* `index(4)`

A package stored in the table entry specified by *index* is made the default package for future *FOP-SYMBOL* and *FOP-SMALL-SYMBOL* interning operations. (These package FOPs may change or disappear as the package system is determined.)

14 *FOP-PACKAGE* `@PushArrow table`

An item is popped from the stack; it must be a symbol. The package of that name is located and pushed onto the table.

15 *FOP-LIST* `length(1) @PushArrow stack`

The unsigned operand *length* specifies a number of operands to be popped from the stack. These are made into a list of that length, and the list is pushed onto the stack. The first item popped from the stack becomes the last element of the list, and so on. Hence an iterative loop can start with the empty list and perform "pop an item and cons it onto the list" *length* times. (Lists of length greater than 255 can be made by using *FOP-LIST** repeatedly.)

16 *FOP-LIST** `length(1) @PushArrow stack`

This is like *FOP-LIST* except that the constructed list is terminated not by *()* (the empty list), but by an item popped from the stack before any others are. Therefore *length*+1 items are popped in all. Hence an iterative loop can start with a popped item and perform "pop an item and cons it onto the list" *length*+1 times.

17-24 *FOP-LIST-1, FOP-LIST-2, ..., FOP-LIST-8*

FOP-LIST-k is like *FOP-LIST* with a byte containing *k* following it. These exist purely to reduce the size of Fasload files. Measurements need to be made to determine the useful values of *k*.

25-32 *FOP-LIST*-1, FOP-LIST*-2, ..., FOP-LIST*-8*

FOP-LIST-k* is like *FOP-LIST** with a byte containing *k* following it. These exist purely to reduce the size of Fasload files. Measurements need to be made to determine the useful values of *k*.

33 *FOP-INTEGER* *n*(4) *value*(*n*)

`@PushArrow stack`

A four-byte unsigned operand specifies the number of following bytes. These bytes define the value of a signed integer in two's-complement form. The first byte of the value is the least significant byte.

34 *FOP-SMALL-INTEGER* *n*(1) *value*(*n*)

`@PushArrow stack`

A one-byte unsigned operand specifies the number of following bytes. These bytes define the value of a signed integer in two's-complement form. The first byte of the value is the least significant byte.

35 *FOP-WORD-INTEGER* `value(4)` `@PushArrow stack`

A four-byte signed integer (in the range -2[31] to 2[31]-1) follows the operation code. A LISP integer (fixnum or bignum) with that value is constructed and pushed onto the stack.

36 *FOP-BYTE-INTEGER* `value(1)` `@PushArrow stack`

A one-byte signed integer (in the range -128 to 127) follows the operation code. A LISP integer (fixnum or bignum) with that value is constructed and pushed onto the stack.

37 *FOP-STRING* *n*(4) *name*(*n*)

`@PushArrow stack`

The four-byte operand *n* specifies the length of a string to construct. The characters of the string follow, one per byte. The constructed string is pushed onto the stack.

38 *FOP-SMALL-STRING* `n(1)` *name*(*n*) `@PushArrow stack`

The one-byte operand *n* specifies the length of a string to construct. The characters of the string follow, one per byte. The constructed string is pushed onto the stack.

39 *FOP-VECTOR* `n(4)` `@PushArrow stack`

The four-byte operand *n* specifies the length of a vector of LISP objects to construct. The elements of the vector are popped off the stack; the first one popped becomes the last element of the vector. The constructed vector is pushed onto the stack.

40 *FOP-SMALL-VECTOR* `n(1)` `@PushArrow stack`

The one-byte operand *n* specifies the length of a vector of LISP objects to construct. The elements of the vector are popped off the stack; the first one popped becomes the last element of the vector. The constructed vector is pushed onto the stack.

41 *FOP-UNIFORM-VECTOR* `n(4)` `@PushArrow stack`

The four-byte operand *n* specifies the length of a vector of LISP objects to construct. A single item is popped from the stack and used to initialize all elements of the vector. The constructed vector is pushed onto the stack.

42 *FOP-SMALL-UNIFORM-VECTOR* `n(1)` `@PushArrow stack`

The one-byte operand *n* specifies the length of a vector of LISP objects to construct. A single item is popped from the stack and used to initialize all elements of the vector. The constructed vector is pushed onto the stack.

43 *FOP-INT-VECTOR* *n*(4) *size*(1) *count*(1)

data(*ceiling*<*n*/*count*>*ceiling*<*size***count*/8>)

`@PushArrow stack`

The four-byte operand *n* specifies the length of a vector of unsigned integers to be constructed. Each integer is *size* bits big, and are packed

in the data stream in sections of *count* apiece. Each section occupies an integral number of bytes. If the bytes of a section are lined up in a row, with the first byte read at the right, and successive bytes placed to the left, with the bits within a byte being arranged so that the low-order bit is to the right, then the integers of the section are successive groups of *size* bits, starting from the right and running across byte boundaries. (In other words, this is a consistent right-to-left convention.) Any bits wasted at the left end of a section are ignored, and any wasted groups in the last section are ignored. It is permitted for the loading implementation to use a vector format providing more precision than is required by *size*. For example, if *size* were 3, it would be permitted to use a vector of 4-bit integers, or even vector of general LISP objects filled with integer LISP objects. However, an implementation is expected to use the most restrictive format that will suffice, and is expected to reconstruct objects identical to those output if the Fasload file was produced by the same implementation. (For the PERQ U-vector formats, one would have *size* an element of 1, 2, 4, 8, 16, and *count*=32/*size*; words could be read directly into the U-vector. This operation provides a very general format whereby almost any conceivable implementation can output in its preferred packed format, and another can read it meaningfully; by checking at the beginning for good cases, loading can still proceed quickly.) The constructed vector is pushed onto the stack.

44 *FOP-UNIFORM-INT-VECTOR* *n*(4) *size*(1)

value(*ceiling*<*size*/8>) @PushArrow stack

The four-byte operand *n* specifies the length of a vector of unsigned integers to construct. Each integer is *size* bits big, and is initialized to the value of the operand *value*. The constructed vector is pushed onto the stack.

45 *FOP-FLOAT* *n*(1) *exponent*(*ceiling*<*n*/8>)

m(1) *mantissa*(*ceiling*<*m*/8>) @PushArrow stack

The first operand *n* is one unsigned byte, and describes the number of *bits* in the second operand *exponent*, which is a signed integer in two's-complement format. The high-order bits of the last (most significant) byte of *exponent* shall equal the sign bit. Similar remarks apply to *m* and *mantissa*. The value denoted by these four operands is *mantissa*x2*exponent*-length(*mantissa*). A floating-point number shall be constructed which has this value, and then pushed onto the stack. That floating-point format should be used which is the smallest (most compact) provided by the implementation which nevertheless provides enough accuracy to represent both the exponent and the mantissa correctly.

46-51 Unused

52 *FOP-ALTER* `index(1)`

Two items are popped from the stack; call the first *newval* and the second *object*. The component of *object* specified by *index* is altered to contain *newval*. The precise operation depends on the type of *object*:

`List` A zero *index* means alter the car (perform *RPLACA*), and *index*=1 means alter the cdr (*RPLACD*).

`Symbol` By definition these indices have the following meaning, and have nothing to do with the actual representation of symbols in a given implementation:

- `0` Alter value cell.
- `1` Alter function cell.
- `2` Alter property list (!).

`Vector (of any kind)` Alter component number *index* of the vector.

`String` Alter character number *index* of the string.

53 *FOP-EVAL* `@PushArrow stack`

Pop an item from the stack and evaluate it (give it to *EVAL*). Push the result back onto the stack.

54 *FOP-EVAL-FOR-EFFECT*

Pop an item from the stack and evaluate it (give it to *EVAL*). The result is ignored.

55 *FOP-FUNCALL* `nargs(1) @PushArrow stack`

Pop *nargs*+1 items from the stack and apply the last one popped as a function to all the rest as arguments (the first one popped being the last argument). Push the result back onto the stack.

56 *FOP-FUNCALL-FOR-EFFECT* `nargs(1)`

Pop *nargs*+1 items from the stack and apply the last one popped as a function to all the rest as arguments (the first one popped being the last argument). The result is ignored.

57 *FOP-CODE-FORMAT* `id(1)`

The operand *id* is a unique identifier specifying the format for following code objects. The operations *FOP-CODE* and its relatives may not occur in a group until after *FOP-CODE-FORMAT* has appeared; there is no default format. This is provided so that several compiled code formats may co-exist in a file, and so that a loader can determine whether or not code was compiled by the correct compiler for the implementation being loaded into. So far the following code format identifiers are defined:

0 PERQ

1 VAX

3 [No value for "("]DinkyMachine)

58 *FOP-CODE* *nitems*(4) *size*(4)

code(*size*) `@PushArrow` `stack`

A compiled function is constructed and pushed onto the stack. This object is in the format specified by the most recent occurrence of *FOP-CODE-FORMAT*. The operand *nitems* specifies a number of items to pop off the stack to use in the "boxed storage" section. The operand *code* is a string of bytes constituting the compiled executable code.

59 *FOP-SMALL-CODE* *nitems*(1) *size*(2)

code(*size*) `@PushArrow` `stack`

A compiled function is constructed and pushed onto the stack. This object is in the format specified by the most recent occurrence of *FOP-CODE-FORMAT*. The operand *nitems* specifies a number of items to pop off the stack to use in the "boxed storage" section. The operand *code* is a string of bytes constituting the compiled executable code.

60 *FOP-STATIC-HEAP*

Until further notice operations which allocate data structures may allocate them in the static area rather than the dynamic area. (The default area for allocation is the dynamic area; this default is reset whenever a new group is begun. This command is of an advisory nature; implementations with no static heap can ignore it.)

61 *FOP-DYNAMIC-HEAP*

Following storage allocation should be in the dynamic area.

62 *FOP-VERIFY-TABLE-SIZE* *size*(4)

If the current size of the table is not equal to *size*, then an inconsistency has been detected. This operation is inserted into a Fasload file purely for error-checking purposes. It is good practice for a compiler to output this at least at the end of every group, if not more often.

63 *FOP-VERIFY-EMPTY-STACK*

If the stack is not currently empty, then an inconsistency has been detected. This operation is inserted into a Fasload file purely for error-checking purposes. It is good practice for a compiler to output this at least at the end of every group, if not more often.

64 *FOP-END-GROUP*

This is the last operation of a group. If this is not the last byte of the file, then a new group follows; the next nine bytes must be "*FASL FILE*".

65 *FOP-POP-FOR-EFFECT* `stack` `@PushArrow`

One item is popped from the stack.

66 *FOP-MISC-TRAP* `@PushArrow stack`

A trap object is pushed onto the stack.

67 *FOP-READ-ONLY-HEAP*

Following storage allocation may be in a read-only heap. (For symbols, the symbol itself may not be in a read-only area, but its print name (a string) may be. This command is of an advisory nature; implementations with no read-only heap can ignore it, or use a static heap.)

68 *FOP-CHARACTER character*(3) `@PushArrow stack`

The three bytes specify the 24 bits of a CMU Common Lisp character object. The bytes, lowest first, represent the code, control, and font bits. A character is constructed and pushed onto the stack.

69 *FOP-SHORT-CHARACTER character*(1)

`@PushArrow stack`

The one byte specifies the lower eight bits of a CMU Common Lisp character object (the code). A character is constructed with zero control and zero font attributes and pushed onto the stack.

70 *FOP-RATIO* `@PushArrow stack`

Creates a ratio from two integers popped from the stack. The denominator is popped first, the numerator second.

71 *FOP-COMPLEX* `@PushArrow stack`

Creates a complex number from two numbers popped from the stack. The imaginary part is popped first, the real part second.

72 *FOP-LINK-ADDRESS-FIXUP nargs*(1) *restp*(1)

offset(4) `@PushArrow stack`

Valid only for when FOP-CODE-FORMAT corresponds to the Vax or the [No value for "("]DinkyMachine). This operation pops a symbol and a code object from the stack and pushes a modified code object back onto the stack according to the needs of the runtime code linker on the Vax or [No value for "("]DinkyMachine).

73 *FOP-LINK-FUNCTION-FIXUP offset*(4)

`@PushArrow stack`

Valid only for when FOP-CODE-FORMAT corresponds to the Vax or the [No value for "("]DinkyMachine). This operation pops a symbol and a code object from the stack and pushes a modified code object back onto the stack according to the needs of the runtime code linker on the Vax or the [No value for "("]DinkyMachine).

74 *FOP-FSET*

Pops the top two things off of the stack and uses them as arguments to FSET (i.e. SETF of SYMBOL-FUNCTION).

128 *FOP-LINK-ADDRESS-FIXUP nargs flag*

offset Valid only when FOP-CODE-FORMAT corresponds to the [No value for "("]DinkyMachine). This operation pops a symbol and a function object off the stack. The code vector in the function object is modified according to the needs of the runtime code linker of the [No value for "("]DinkyMachine) and pushed back on the stack. This FOP links in calls to other functions.

129 *FOP-MISCOP-FIXUP* *index*(2) *offset*(4)
Valid only when FOP-CODE-FORMAT corresponds to the [No value for "("]DinkyMachine). This operation pops a code object from the stack and pushes a modified code object back onto the stack according to the needs of the runtime code linker on the [No value for "("]DinkyMachine). This FOP links in calls to the assembler language support routines.

130 *FOP-ASSEMBLER-ROUTINE* *code-length*
Valid only when FOP-CODE-FORMAT corresponds to the [No value for "("]DinkyMachine). This operation loads assembler code into the assembler code space of the currently running Lisp.

131 *FOP-FIXUP-MISCOP-ROUTINE*
Valid only when FOP-CODE-FORMAT corresponds to the [No value for "("]DinkyMachine). This operation pops a list of external references, a list of external labels defined, the name, and the code address off the stack. This information is saved, so that after everything is loaded, all the external references can be resolved.

132 *FOP-FIXUP-ASSEMBLER-ROUTINE*
is similar to FOP-FIXUP-MISCOP-ROUTINE, except it is for internal assembler routines rather than ones visible to Lisp.

133 *FOP-FIXUP-USER-MISCOP-ROUTINE*
is similar to FOP-FIXUP-MISCOP-ROUTINE, except it is for routines written by users who have an extremely good understanding of the system internals.

134 *FOP-USER-MISCOP-FIXUP* *offset*(4)
is similar to FOP-MISCOP-FIXUP, but is used to link in user defined miscops.

255 *FOP-END-HEADER*
Indicates the end of a group header, as described above.

20 Appendix – Building CMU Common Lisp

20.1 (Introduction)

This document explains how to build a working Common Lisp from source code on the IBM RT PC under the Mach operating system. You should already have a working Common Lisp running on an IBM RT PC before trying to build a new Common Lisp.

Throughout this document the following terms are used:

Core file A core file is a file containing an image of a Lisp system. The core file contains header information describing where the data in the rest of the file should be placed in memory. There is a simple C program which reads a core file into memory at the correct locations and then jumps to a location determined by the contents of the core file. The C code includes the X window system version 10 release 4 which may be called from Lisp.

Cold core file
A cold core file contains enough of the Lisp system to make it possible to load in the rest of the code necessary to generate a full Common Lisp. A cold core file is generated by the program Genesis.

Miscops Miscops are assembler language routines that are used to support compiled Lisp code. A Lisp macro assembler provides a convenient mechanism for writing these assembler language routines.

Matchmaker
Matchmaker is a program developed to automatically generate remote procedure call interfaces between programs. Matchmaker accepts a description of a remote procedure call interface and generates code that implements it.

There are many steps required to go from sources to a working Common Lisp system. Each step will be explained in detail in the following sections. It is possible to perform more than one step with one invocation of Lisp. However, I recommend that each step be started with a fresh Lisp. There is some small chance that something done in one step will adversely affect a following step if the same Lisp is used. The scripts for each step assume that you are in the user package which is the default when Lisp first starts up. If you change to some other package, some of these steps may not work correctly.

In many of the following steps, there are lines setting up search lists so that command files know where to find the sources. What I have done is create a init.lisp file that sets up these search lists for me. This file is automatically loaded from the user's home directory (as determined by the [HOME] environment variable) when you start up Lisp. Note that my init.lisp file is included with the sources. You may have to modify it, if you change where the lisp sources are.

20.2 (Installing Source Code)

With this document, you should also have received a tape cartridge in tar format containing the complete Common Lisp source code. You should create some directory where you want to put the source code. For the following discussion, I will assume that the source code lives in the directory /usr/lisp. To install the source code on your machine, issue the following commands:

```
cd /usr/lisp
tar xvf <tape device>
```

The first command puts you into the directory where you want the source code, and the second extracts all the files and sub-directories from the tape into the current directory. <Tape device> should be the name of the tape device on your machine, usually /dev/st0.

The following sub-directories will be created by tar:

`bin` contains a single executable file, lisp, which is a C program used to start up Common Lisp.

`clc` contains the Lisp source code for the Common Lisp compiler and assembler.

`code` contains the Lisp source code that corresponds to all the functions, variables, macros, and special forms described in *Common Lisp: The Language* by Guy L. Steele Jr., as well as some Mach specific files.

`hemlock` contains the Lisp source code for Hemlock, an emacs-like editor written completely in Common Lisp.

`icode` contains Matchmaker generated code for interfaces to Inter Process Communication (IPC) routines. This code is used to communicate with other processes using a remote procedure call mechanism. Under Mach, all the facilities provided by Mach beyond the normal Berkeley Unix 4.3 system calls are accessed from Lisp using this IPC mechanism. Currently, the code for the Mach, name server, Lisp typescript, and Lisp eval server interfaces reside in this directory.

`idefs` contains the Matchmaker definition files used to generate the Lisp code in the icode directory.

`lib` contains files needed to run Lisp. The file lisp.core is known as a Lisp core file and is loaded into memory by the lisp program mentioned above in the entry for the bin directory. This file has a format which allows it to be mapped into memory at the correct locations. The files spell-dictionary.text and spell-dictionary.bin are the text and binary form of a dictionary, respectively, used by Hemlock's spelling checker and corrector. The two files hemlock.cursor and hemlock.mask are used by Hemlock when running under the X window system.

`miscops` contains the Lisp assembler source code for all the miscops that support low level Lisp functions, such as storage allocation, complex operations that can not performed in-line, garbage collection, and other

operations. These routines are written in assembler, so that they are as efficient as possible. These routines use a very short calling sequence, so calling them is very cheap compared to a normal Lisp function call.

`mm` contains the Lisp source code for the Matchmaker program. This program is used to generate the Lisp source code files in icode from the corresponding matchmaker definitions in idefs.

`pcl` contains the Lisp source code for a version of the Common Lisp Object System (originally Portable Common Loops), an object oriented programming language built on top of Common Lisp.

`X` contains the C object files for X version 10 release 4 C library routines. These are linked with the lisp startup code, so that X is available from Lisp.

`scribe` contains Scribe source and postscript output for the manuals describing various aspects of the CMU Common Lisp implementation.

`demos` contains the Lisp source code for various demonstration programs. This directory contains the Gabriel benchmark set (bmarks.lisp) and a sub-directory containing the Soar program which is also used for benchmarking purposes. There may be other programs and/or subdirectories here that you may look at.

These directories contain source files as well as Lisp object files. This means it is not necessary to go through all the steps to build a new a Common Lisp, only those steps that are affected by a modification to the sources. For example, modifying the compiler will require recompiling everything. Modifying a miscop file should require only reassembling that particular file and rebuilding the cold core file and full core file.

As well as the directories mentioned above, there are also several files contained in the top-level directory. These are:

`init.lisp` is a Lisp init file I use. This sets up some standard search lists, as well as defines a Hemlock mode for editing miscop source files.

lisp.c contains the C code used to start up the lisp core image under Mach.

lispstart.s contains some assembler language code that is invoked by lisp.c to finish the process of starting up the lisp process.

makefile contains make definitions for compiling lisp.c and lispstart.s into the lisp program.

rg contains some adb commands that can be read into adb while debugging a lisp process. It prints out all the registers, the name of the currently executing Lisp function, and sets an adb variable to the current stack frame which is used by the following file.

st contains some adb commands that can be read into adb while debugging a lisp process. It prints out a Lisp stack frame and the name

of the function associated with the stack frame. It also updates the adb variable mentioned above to point to the next stack frame. Repeatedly reading this file into adb will produce a backtrace of all the active call frames on the Lisp stack.

ac contains some adb commands that print out the current values of the active catch pointer. This points to the head of a list of catch frames that exist on the control stack.

cs contains some adb commands that print out the contents of a catch frame. Reading cs into adb several times in a row (after reading ac once) will print out the catch frames in order.

20.3 (Compiling the Lisp Startup Program)

To compile the lisp start up program, you should be in the top level directory of the sources (/usr/lisp) and type:

```
make lisp
```

This will compile the file lisp.c, assemble the file lispstart.s and produce an executable file lisp. Currently the default location for the lisp core file is /usr/misc/.lisp/lib/lisp.core. If you want to change this default location, edit the file lisp.c and change the line

```
#define COREFILE "/usr/misc/.lisp/lib/lisp.core"
```

to refer to the file where you intend to put the core file.

This step takes a few seconds.

20.4 (Assembling Assembler routines)

The standard core image includes a Lisp macro assembler. To assemble all the miscops, the following steps should be performed:

```
(compile-file "/usr/lisp/clc/miscasm.lisp")
(load "/usr/lisp/clc/miscasm.fasl")
(setf (search-list "msc:") '("/usr/lisp/miscops/"))
(clc::asm-files)
```

The first line compiles a file that contains a couple of functions used to assemble miscop source files. The second line loads the resulting compiled file into the currently executing core image. The third line defines the msc search list which is used by the function clc::asm-files to locate the miscop sources. The terminal will display information as each file is assembled. For each file a .fasl, a .list, and an .err file will be generated in /usr/lisp/miscops.

This step takes about half an hour.

20.5 (Compiling the Compiler)

To compile the compiler is simple:

```
(setf (search-list "clc:") '("/usr/lisp/clc/"))
(load "clc:compclc.lisp")
```

The first line just sets up a search list variable clc, so that the file compclc.lisp can find the compiler sources. The terminal will display information as each file is assembled. For each file a .fasl and an .err file will be generated. A log of the compiler output is also displayed on the terminal.

This step takes about forty-five minutes.

20.6 (Compiling the Lisp Sources)

Compiling the Lisp source code is also easy:

```
(setf (search-list "code:") '("/usr/lisp/code/"))
(load "code:worldcom.lisp")
```

Again, the first line defines a search list variable, so that the file worldcom.lisp can find the Lisp sources. As each file is compiled, the name of the file is printed on the terminal. For each file a .fasl will be generated. Also, a single error log will be generated in the file code:compile-lisp.log.

This step takes about an hour and a half.

20.7 (Compiling Hemlock)

Compiling the Hemlock source code is done as follows:

```
(setf (search-list "hem:") '("/usr/lisp/hemlock/"))
(load "hem:ctw.lisp")
```

Again, the first line defines a search list variable, so that ctw.lisp can find the Hemlock sources. As each file is compiled, the name of the file is printed on the terminal. For each file a .fasl will be generated. Also, a single error log will be generated in the file hem:lossage.log.

This step takes about forty-five minutes.

20.8 (Compiling Matchmaker)

Compiling the matchmaker sources is done as follows:

```
(setf (search-list "mm:") '("/usr/lisp/mm"))
(compile-file "mm:mm.lisp")
(load "mm:mm.fasl")
(compile-mm)
```

The first line sets up a search list, so that the matchmaker sources can be found. The second line compiles the file containing a function for compiling the matchmaker sources. The third line loads the file just compiled, and the final line invokes the function compile-mm which compiles the matchmaker sources. For each file, a .fasl and .err file is generated. Also, a log of the compiler output is printed to the terminal.

This step takes about 15 minutes

20.9 (Generating Lisp Source Files from Matchmaker Definition Files)

The following sequence of commands is necessary to generate the Lisp files for the Mach interface:

```
(setf (search-list "mm:") '("/usr/lisp/mm/"))
(setf (search-list "idefs:") '("/usr/lisp/idefs/"))
(setf (search-list "icode:") '("/usr/lisp/icode/"))
(setf (search-list "code:") '("/usr/lisp/code/"))
(setf (default-directory) "/usr/lisp/icode/")
(load "code:mm-interfaces.lisp")
```

The first four lines set up search lists for mm (matchmaker sources), idefs (matchmaker interface definition files), icode (Lisp matchmaker interface sources), and code (Lisp code sources). The fifth line changes the current working directory to be /usr/lisp/icode. This is where the output from matchmaker will be placed. And finally, the last line invokes matchmaker on the matchmaker definition files for all the interfaces.

Matchmaker generates three files for each interface XXX:

`XXXdefs.lisp`
: contains constants and record definitions for the interface.

`XXXmsgdefs.lisp`
: contains definitions of offsets to important fields in the messages that are sent to and received from the interface.

`XXXuser.lisp`
: contains code for each remote procedure, that sends a message to the server and receives the reply from the server (if appropriate). Each of these functions returns one or more values. The first value returned is a general return which specifies whether the remote procedure call succeeded or gives an indication of why it failed. Other values may be returned depending on the particular remote procedure. These values are returned using the multiple value mechanism of Common Lisp.

This step takes about five minutes.

20.10 (Compiling Matchmaker Generated Lisp Files)

To compile the matchmaker generated Lisp files the following steps should be performed:

```
(setf (search-list "code:") '("/usr/lisp/code/"))
(setf (search-list "icode:") '("/usr/lisp/icode/"))
(load "code:comutil.lisp")
```

The first two lines set up search lists for the code and icode directory. The final line loads a command file that compiles the Mach interface definition in the correct order. Note that once the files are compiled, the XXXmsgdefs files are no longer needed. The file /usr/lisp/icode/lossage.log contains a listing of all the error messages generated by the compiler.

This step takes about fifteen minutes.

20.11 (Compiling the Common Lisp Object System)

To compile the Common Lisp Object System (CLOS) do the following:

```
(setf (search-list "pcl:") '("/usr/lisp/pcl/"))
(rename-package (find-package "CLOS") "OLD-CLOS")
(compile-file "pcl:defsys.lisp")
(load "pcl:defsys.fasl")
(clos::compile-pcl)
```

The first line sets up a search list as usual. The second line renames the CLOS package to be the OLD-CLOS package. This is so that the current version of CLOS doesn't interfere with the compilation process. The third line compiles a file containing some functions for building CLOS. The fourth line loads in the result of the previous compilation. The final line compiles all the CLOS files necessary for a working CLOS system.

The file /usr/lisp/pcl/test.lisp is a file that contains some test functions. To run it through CLOS build a new Lisp and start up a fresh Lisp resulting from the build and do the following:

```
(in-package 'clos)
(compile-file "/usr/lisp/pcl/test.lisp")
(load "/usr/lisp/pcl/test.fasl")
```

This sequence of function calls puts you in the CLOS package, compiles the test file and then loads it. As the test file is loaded, it executes several tests. It will print out a message specifying whether each test passed or failed.

Currently, CLOS is built into the standard core.

This step takes about 30 minutes.

20.12 (Compiling Genesis)

To compile genesis do the following:

```
(compile-file "/usr/lisp/clc/genesis.lisp")
```

Genesis is used to build a cold core file. Compiling Genesis takes about five minutes.

20.13 (Building a Cold Core File)

Once all the files have been assembled or compiled as described above, it is necessary to build a cold core file as follows:

```
(setf (search-list "code:") '("/usr/lisp/code/"))
(setf (search-list "icode:") '("/usr/lisp/icode/"))
(setf (search-list "msc:") '("/usr/lisp/miscops/"))
(load "/usr/lisp/clc/genesis.fasl")
(load "code:worldbuild.lisp")
```

The first three lines set up search lists for the code, icode, and miscops subdirectories. The fourth line loads in the program Genesis which builds the cold core

file. The last line calls Genesis on a list of the files that are necessary to build the cold core file. As each file is being processed, its name is printed to the terminal. Genesis generates two files: /usr/lisp/ilisp.core and /usr/lisp/lisp.map. Ilisp.core is the cold core file and lisp.map is a file containing the location of all the functions and miscops in the cold core file. Lisp.map is useful for debugging the cold core file.

This step takes from about fifteen minutes.

20.14 (Building a Full Common Lisp)

The cold core file built above does not contain some of the more useful programs such as the compiler and hemlock. To build these into a core, it is necessary to do the following:

```
lisp -c /usr/lisp/ilisp.core
(in-package "USER")
(load (open "/usr/lisp/code/worldload.lisp"))
```

The first line invokes the lisp startup program specifying the cold core file just built as the core file to load. This cold core file is set up to do a significant amount of initialization and it is quite possible that some bug will occur during this initialization process. After about a minute, you should get a prompt of the form:

```
CMU Common Lisp kernel core image 2.7(?).
[You are in the Lisp Package.]
*
```

The following two lines should then be entered. The first of these puts you into the User package which is the package you should be in when the full core is first started up. It is necessary to add this line, because the current package is rebound while a file is loaded. The last line loads in a file that loads in the compiler, hemlock, and some other files not yet loaded. The open call is [essential] otherwise when the full core is started up, load will try to close the file and probably invalidate memory that is needed. When load is passed a stream, it does not automatically close the stream. With a file name it now does after a recent bug fix. This file prompts for the versions of the Lisp system, the compiler, and hemlock. You should enter versions that make sense for your installation. It then purifies the core image. Up to this point most of the Lisp system has been loaded into dynamic space. Only a few symbols and some other data structures are in static space. The process of purification moves Lisp objects into static and read-only space, leaving very little in dynamic space. Having the Lisp system in static and read-only space reduces the amount of work the garbage collector has to do. Only those objects needed in the final core file are retained. Finally, a new core file is generated and is written to the file /usr/lisp/nlisp.core. Also, the currently running Lisp should go through the default initialization process, finally prompting for input with an asterisk. At this point you have successfully built a new core file containing a complete Common Lisp implementation.

This step takes about thirty minutes.

20.15 (Debugging)

Debugging Lisp code is much easier with a fully functional Lisp. However, it is quite possible that a change made in the system can cause a bug to happen while running the cold core file. If this happens, it is best to use adb to track down the problem. Unfortunately, the core file (i.e., the remains of a process normally created by Unix when a process dies) generated by such a bug will be of no use. To get some useful information, follow these steps:

Look at the file /usr/lisp/lisp.map and find the entry points for the miscop routines error0, error1, and error2. These entry points are used to invoke the Lisp error system from the miscops. Write down the numbers beside these names. They are the addresses (in hex) of where the miscops are located when the cold core file is loaded into memory.

Run adb on the lisp file, i.e.:

```
adb lisp
```

Set a breakpoint at the lispstart entry point:

```
lispstart:b
```

Start the lisp program running, telling it to use ilisp.core (I'm assuming you're in /usr/lisp):

```
:r -c ilisp.core
```

After a while, you will hit the lispstart breakpoint. The core file has been mapped into memory, but control is still in the C startup code. At this point, you should enter breakpoints for all the error entry points described above.

Continue running the program by typing :c. Shortly after this, the C lisp program will give up control to Lisp proper. Lisp will start doing its initialization and will probably hit one of the error break points. At that point you can look around at the state and try and discover what has gone wrong. Note that the two files rg and st are useful at this point. Also, you should look at the document *Internal Design of Common Lisp on the IBM RT PC* by David B. McDonald, Scott E. Fahlman, and Skef Wholey so that you know the internal data structures.

20.16 (Running the Soar Benchmark)

To compile the soar benchmark, you should do the following:

```
(compile-file "/usr/lisp/demos/soar/soar.lisp")
```

To run the benchmark, you should start up a fresh Lisp and do the following:

```
(load "/usr/lisp/demos/soar/soar.fasl")
(load "/usr/lisp/demos/soar/default.soar")
(load "/usr/lisp/demos/soar/eight.soar")
(user-select 'first)
(init-soar)
(time (run))
```

The first two lines load in the standard Soar system. The third line loads in information about the eight puzzle which is a standard Soar puzzle that has been

run on several different machines. The fourth line sets up the puzzle conditions so that it will select a goal to work on automatically. The fifth line initializes Soar's working memory, etc. The final line is the one that actually runs the benchmark. Soar prints out a fair amount of information as it solves the puzzle. The final state should be numbered 143 when it finishes. The time macro prints out information about information various resources after the eight puzzle has run.

20.17 (Summary)

I have tried to present sufficient information here to allow anyone to be able to build a Common Lisp system under Mach from the sources. I am sure there are many tricks that I have learned to use to reduce the amount of grief necessary to build a system. My best recommendation is to go slowly. Start by building a system from the sources provided on the tape. Make sure you are comfortable doing that before you try modifying anything.

Some hints on building the system which you may find useful:

- If you change the compiler, you will have to recompile all the sources before the change is reflected in a system. Changing the compiler is probably the most dangerous change you can make, since an error here means that nothing will work. In particular, this is the time you are going to need to get familiar with adb and the internal structure of the Lisp, since a serious error in the compiler will show up during the initialization of the cold core file.
- Changing the miscops should be done with care. They follow a fairly rigid convention and you should understand all the information provided in *Internal Design of Common Lisp on the IBM RT PC* before making any changes to miscops. You will probably need to get familiar with adb to debug some of the changes. Note that this requires building a new cold core file and a final core file before the change is reflected in the system.
- Changing sources in the code directory should be fairly straight forward. The only time this will cause trouble is if you change something that a lot of files depend on in which case you will have to recompile everything and build a new cold core file and a core file.
- Changing hemlock should have no adverse effect on system integrity.
- If you make a fairly major change, it is a good idea to go through the complete process of building a core file at least two or three times. If things are still working at the end of this, your change is probably correct and shouldn't cause any serious trouble.
- Finally, always keep at least one backup copy of a good core image around. If you build a bad core file over an existing one and can't back up, it is possible that you may not be able to recover from a serious error.

Todo: fasl.tex In good shape.

object.tex Fairly good, but should probably be integrated with description of primitives in vm.tex.

front.tex Needs updating cleanup scan. Not too bad.

middle.tex Need VMR overview. New names for GTN/LTN? Needs general cleanup, but not too bad. NLX and stack are the worst.

back.tex Pack and assembler need more info. General cleanup.

compiler-overview.tex Adapt introductory material from /../fred/usr/ram/comp.mss, pap:talk.mss Division between ICR overview and ICR convert needs work.

debugger.tex Needs much work. Merge much info from debug-info and debug-int. Duplicating a fair amount of stuff in the source may make sense where, since this is a part of the system that is generally interesting. And also, a part that people building on CMU CL might want to understand.

glossary.tex Finish, integrate w/ main text?

interpreter.tex Very sketchy and tentative. Needs to be fleshed out from the code.

retargeting.tex Very rough. Needs to be merged with parts of vm.tex (call vops). Needs some additional text. Documentation of assembler, and all other exported interfaces. (Generate defined VOP descriptions from the core, keyed to files?)

vm.tex This file should probably cease to exist, going into object, retargeting and introductory material. [Also other scrap in stuff/]

[VMR and ICR overview also needed...]

architecture.tex Missing sections on startup code, compiling, building.

environment.tex Needs to be written: type system and info database interfaces.

interface.tex Needs to be written: source paths and error message utilities.

lowlev.tex Needs to be written. All manner of low-level stuff: memory layout and management, core file format, C interface, low-level debugging (and ldb.)

Several different audiences: – Curious compiler implementors (not a big priority. Downplay academic aspects, i.e. comparisons to other techniques, analysis of limitations, future work...) Compiler part can be more academic, and include some justifications of other design decisions. – System maintainers. – People retargeting the compiler. – People bringing up the system in a new environment.

Sys arch part: Package + file structure [system.txt] system building [compiling.txt] bootstrapping & cross compiling

Compiler design: Overview (mirror structure of rest of the part) ICR data structure Front end [front.tex] Basic VMR data structures (no back-end stuff) Middle end [middle.tex] Back end + data structures [back.tex]

Error system interface Source tracking

Compiler retargeting: VM definition concepts [porting.txt, mail.txt, retargeting.tex] SCs, SBs, primitive-types Defining VOPS time specification defining and using the assembler Required VOPs [internal.txt, lowlev.txt, vm.mss] Standard primitives [vm.mss] (broken down by type, parallels object format section structure.) Customizing VMR conversion multiple hardware constant operands VM specific transforms special-case IR2 convert methods

Run-time system: type system info database Data format [object.tex] Debugger: Info format [debug.txt] Stack parsing [debug.txt] Breakpoints Internal errors Signals Memory management: [William] heap Layout stacks GC misc implementation stuff:

foreign call, assembly routines [lowlev.txt] LDB and low-level debugging core file format [William] fasl format [fasl.tex]

Index

%

%character-code-byte 78
%character-control-byte 78
%character-font-byte 78
%set-symbol-package 79
%symbol-name-slot 79
%symbol-package-slot 79
%symbol-plist-slot 79

[

[%Initial-function] 142
[%Link-table-header] 142
[%SP-1+complex] 143
[%SP-1+ratio] 143
[%SP-1-complex] 144
[%SP-abs-complex] 143
[%SP-abs-ratio] 142
[%SP-bignum/bignum] 142
[%SP-bignum/fixnum] 142
[%SP-complex*complex] 144
[%SP-complex*number] 144
[%SP-complex+complex] 143
[%SP-complex+number] 143
[%SP-complex-complex] 144
[%SP-complex-number] 143
[%SP-complex-truncate-complex] 145
[%SP-complex-truncate-number] 145
[%SP-complex/complex] 144
[%SP-complex/number] 144
[%SP-fixnum/bignum] 142
[%SP-integer+ratio] 143
[%SP-integer-truncate-ratio] 145
[%SP-integer/ratio] 144
[%SP-Internal-Apply] 142
[%SP-Internal-Error] 142
[%SP-Internal-Throw-Tag] 142
[%SP-negate-complex] 143
[%SP-negate-ratio] 143
[%SP-number*complex] 144
[%SP-number+complex] 143
[%SP-number-complex] 144
[%SP-number-truncate-complex] 145
[%SP-number/complex] 144
[%SP-ratio*ratio] 144
[%SP-ratio+ratio] 143
[%SP-ratio-integer] 143
[%SP-ratio-ratio] 143
[%SP-ratio-truncate-integer] 145
[%SP-ratio-truncate-ratio] 145
[%SP-ratio/integer] 144
[%SP-ratio/ratio] 144
[%SP-Software-Interrupt-Handler] 142
[*] 164
[*Ignore-Floating-Point-Underflow*] 145
[*Nameserverport*] 145
[+] 164
[-] 164
[/] 164
[=] 163
[1+] 164
[1-] 164
[16bit-System-Ref] 172
[16bit-System-Set] 172
[8bit-System-Ref] 172
[8bit-System-Set] 172
[Abs] 164
[Access-type codes] 140
[Active frame] 149
[Active-Call-Frame] 174
[Active-Catch-Frame] 174
[Active-Frame-Pointer register] 147
[Active-Function-Pointer register] 147
[Alloc-Array] 155
[Alloc-Bignum] 155
[Alloc-Bit-Vector] 155
[Alloc-Function] 155
[Alloc-G-Vector] 155
[Alloc-I-Vector] 155
[Alloc-String] 155
[Alloc-Symbol] 155
[Aref1] 160
[Arg-In-Frame] 173
[Argument registers] 147
[Array format] 137, 139
[Array header format] 140
[ArrayP] 162
[Arrays] 140
[Aset1] 160
[Ash] 165
[Assembler Support Routines] 154
[Assoc] 157
[Assq] 157
[Atan] 166
[Bignum Format] 139
[Bignum format] 136
[BignumP] 161

[Bind-Null] 156
[Bind] 156
[Binding stack format] 151
[Binding stack space] 138
[Binding-Stack-Pointer register] 147
[Bit-Bash] 160
[Bit-Vector format] 136
[Bit-Vector-P] 161
[Boole] 165
[Boundp] 158
[Break-Return] 169
[Byte-BLT] 160
[Caar] 157
[Cadr] 157
[Call-0] 166, 175
[Call-Foreign] 170
[Call-lisp-from-c] 145
[Call-Lisp] 170
[Call-Multiple] 166, 175
[Call] 166, 175
[Car] 157
[CAref2] 160
[CAref3] 160
[CAset2] 160
[CAset3] 160
[Catch frames] 150
[Catch-All object] 135, 179
[Catch-All] 169
[Catch-Multiple] 169
[Catch] 150, 169, 179
[Cdar] 157
[Cddr] 157
[Cdr] 157
[Character object] 134
[CharacterP] 163
[Check-<=] 173
[Clean-Space pointer] 152
[Code vector] 148
[Collect-Garbage] 173
[Compare] 164
[Compiled-Function-P] 162
[Complex number format] 137
[Complex-Array-P] 162
[ComplexP] 161
[Cons] 156
[ConsP] 162
[Constants in code] The function object contains a vector of 148
[Control stack space] 138
[Control-stack format] 149
[Control-Stack-Pointer register] 147
[Cos] 166
[Current-allocation-space] 142
[Current-Binding-Pointer] 174
[Current-Stack-Pointer] 174
[Decode-Float] 163
[Defined-From String Format] 149
[Definition cell] 136
[DEFSTRUCT] 138
[Denominator] 163
[Deposit-Field] 165
[Dpb] 165
[Endp] 157
[Eq] 171
[Eql] 171
[Errors] 180
[Escape to Lisp code convention] 179
[Exp] 166
[FBoundp] 158
[Find-Character-With-Attribute] 160
[Find-Character] 160
[Fixnum*Fixnum] 164
[Fixnum/Fixnum] 164
[FixnumP] 163
[Float-Long] 163
[Float-Short] 163
[Floating point formats] 136
[FloatP] 162
[Flonum format] 136
[Flonum formats] 136
[Flush-Values] 171
[Force-Values] 171
[Forwarding pointers] 137
[Free-Storage pointer] 152
[Function object format] 137, 139
[G-Vector format] 136
[G-Vector-Length] 158
[Garbage Collection] 152
[GC-Forward pointer] 137
[GCD] 164
[General-Vector format] 136, 138
[General-Vector-P] 162
[Get-Allocation-Space] 154
[Get-Definition] 157
[Get-Package] 158
[Get-Plist] 158
[Get-Pname] 158
[Get-Space] 171
[Get-Type] 171
[Get-Value] 157
[Get-Vector-Access-Code] 159
[Get-Vector-Subtype] 159
[Get] 158
[GetF] 157

[Hairy stuff] 175
[Hash tables] 138
[Header-Length] 159
[Header-Ref] 159
[Header-Set] 159
[I-Vector format] 136
[Illegal object trap] 135
[Imagpart] 163
[Immediate object format] 133
[Int-Sap] 173
[Integer-Length] 163
[Integer-Vector format] 136, 139
[IntegerP] 162
[Interrupt-Handler] 168
[Interruptible Marker] 135
[Interrupts] 184
[Invoke1*] 167
[Invoke1] 167
[Kernel traps] 184
[Ldb] 165
[Link-Address-Fixup] 167
[Lisp objects] 133
[Lisp-command-line-list] 145
[Lisp-environment-list] 145
[List cell] 136
[List*] 156
[List] 156
[ListP] 162
[Local registers] 147
[Log] 166
[Logand] 164
[Logcount] 163
[Logdpb] 166
[Logior] 164
[Logldb] 165
[Lognot] 165
[Logxor] 164
[Long-Float-P] 161
[Lsh] 165
[Make-Compiled-Closure] 168
[Make-Complex] 155
[Make-Immediate-Type] 172
[Make-Predicate] 171
[Make-Ratio] 155
[Mask-Field] 165
[maybe-gc] 145
[Member] 157
[Memq] 157
[Miscop argument register] 147
[Miscop-Fixup] 168
[Multiple values] 178
[mv-list] 156
[N-To-Values] 171
[Negate] 164
[Newspace-Bit] 173
[NIL] 141
[Non-Lisp temporary registers] 148
[Non-Local Exits] 179
[Not-Predicate] 171
[NPop] 156
[NumberP] 162
[Numerator] 163
[Open frame] 149
[Package cell] 136
[Plist cell] 136
[Pname cell] 136
[Pointer object format] 133, 135
[Pointer-System-Set] 172
[Pop] 156
[Print name cell] 136
[Program-Counter register] 147
[Property list cell] 136
[Purification] 153
[Purify] 173
[Push-Last] 167
[Push] 156
[Put] 158
[Ratio format] 137
[RationalP] 162
[RatioP] 162
[Read-Binding-Stack] 174
[Read-Control-Stack] 174
[Read-only space] 135
[Realpart] 163
[Register allocation] 147
[Replace-Car] 157
[Replace-Cdr] 157
[Reset-C-Stack] 171
[Reset-link-table] 168
[Rest-Entry-0] 170
[Rest-Entry-1] 170
[Rest-Entry-2] 170
[Rest-Entry] 170
[Return-1-Value-Any-Bind] 167
[Return-From] 167
[Return-Mult-Value-0-Bind] 167
[Return-To-C] 170
[Return] 167
[Runtime Environment] 147
[Sap-Int] 172
[Sap-System-Ref] 172
[Sap-System-Set] 172
[Save] 173
[SBit] 160

[SBitset] 160
[Scale-Float] 163
[Scavenger] 153
[SChar] 160
[SCharset] 160
[Set-Allocation-Space] 155
[Set-C-Procedure-Pointer] 171
[Set-Call-Frame] 174
[Set-Cddr] 157
[Set-Cdr] 157
[Set-Definition] 158
[Set-Lpop] 157
[Set-Package] 158
[Set-Plist] 158
[Set-Up-Apply-Args] 166
[Set-Value] 157
[Set-Vector-Subtype] 159
[Short float format] 134
[Shrink-Vector] 159
[Signed-16bit-System-Ref] 172
[Signed-32bit-System-Ref] 172
[Signed-32bit-System-Set] 172
[Simple-Bit-Vector-Length]
  Vector-Length 158
[Simple-Bit-Vector-P] 161
[Simple-Integer-Vector-P] 161
[Simple-String-Length] 158
[Simple-String-P] 161
[Simple-Vector-P] 162
[Sin] 166
[Single-Float-P] 163
[Space codes] 134, 135
[Special binding stack space] 138
[Spread] 157
[Sqrt] 166
[Stack spaces] 138
[Start-Call-Interpreter] 167
[Start-call-mc] 167
[Static space] 135
[Static-Alloc-G-Vector] 155
[Storage management] 152
[String format] 137, 140
[StringP] 161
[SVref] 160
[SVset] 160
[SXHash-Simple-String] 161
[Symbol] 136
[SymbolP] 162
[Syscall] 173
[Syscall0] 173
[Syscall1] 173
[Syscall2] 173
[Syscall3] 173
[Syscall4] 173
[System table space] 138
[T] 142
[Tan] 166
[Throw] 170, 179
[Transporter] 152
[Trap] 135
`[Trap] Illegal object trap` 77
[Trapping to the kernel] 184
[Truncate] 164
[Type codes] 133
[Typed-Vref] 159
[Typed-Vset] 159
[Unbind] 156
[Unix-fork] 173
[Unix-write] 173
[Unsigned-32bit-System-Ref] 172
[Unwind-Protect] 179
[Value cell] 136
[Values-Marker] 134
[Values-To-N] 171
[Vector format] 136
[Vector-Length] 158
[Vector] 155
[VectorP] 162
[Vectors] 138
[Virtual memory] 135
[Write-Binding-Stack] 174
[Write-Control-Stack] 174

{

`{small, fast}` 76

A

accessor VOP 79
`alloc-symbol` 79
array 25
assembly 13

B

`bind` 80
Bit numbering 131
block compilation 19
Byte numbering 131

C

canonical form 25
`change-type` 77
`char-int` 78
character 78
code generation 13
Compiler Overview 11
control analysis 13
copy propagation 13, 57

D

data type 75
dumping 13

E

entry analysis 13
entry point 20

F

find components 11
Fixnum format 134

G

`get` 80
`get-type` 77
global TN assignment 44
GTN 12

H

Hairy function representation 17

I

ICR 11
ICR conversion 25
ICR representation 18
`if-char<` 78
`if-char=` 78
`if-char>` 78
`if-characterp` 76
`if-consp` 76
`if-fixnump` 76
`if-floatp` 76
`if-functionp` 77
`if-integerp` 76
`if-listp` 76
`if-numberp` 76
`if-rationalp` 76
`if-short-float-p` 76
`if-type` 76
`if-type-range` 76
`if-vector-subtype` 76
`if-vectorp` 77
`int-char` 78
intermediate representation 11
IR 11

L

lifetime analysis 13
lisp object 75
local call analysis 11
LTN 12

N

`name` 102
NLX 18
non-local exits 18

P

pack 13
pipeline reorganization 13
property list 80
`put` 80

R

representation selection 13, 58
return 177

S

SB 103
SBCL History 1
`small, fast` 79
special binding 80
stack analysis 13
storage base 102
Storage Base (SB) 103
storage class 102
storage resources 75
sub-primitive 79
symbol 79
`symbol-function-slot` 79
`symbol-value-slot` 79

T

tail sets 16
The Implicit Continuation Representation 14
type constraint propagation 12
type sub-primitive 77
type VOP 76

U

`unbind` 80

V

virtual machine representation 43
VM 71
VMR 11

X

XEP 20

About the Author

Robert J. Chassell (1946–2017) started working with GNU Emacs in 1985. He wrote and edited, taught Emacs and Emacs Lisp, and spoke throughout the world on software freedom. Chassell was a founding Director and Treasurer of the Free Software Foundation, Inc. He was co-author of the *Texinfo* manual, and edited more than a dozen other books. He graduated from Cambridge University, in England. He had an abiding interest in social and economic history and flew his own airplane.

"Goodbye to Bob Chassell" (`https://www.fsf.org/blogs/community/goodbye-to-bob-chassell`)